

Regresión Lineal

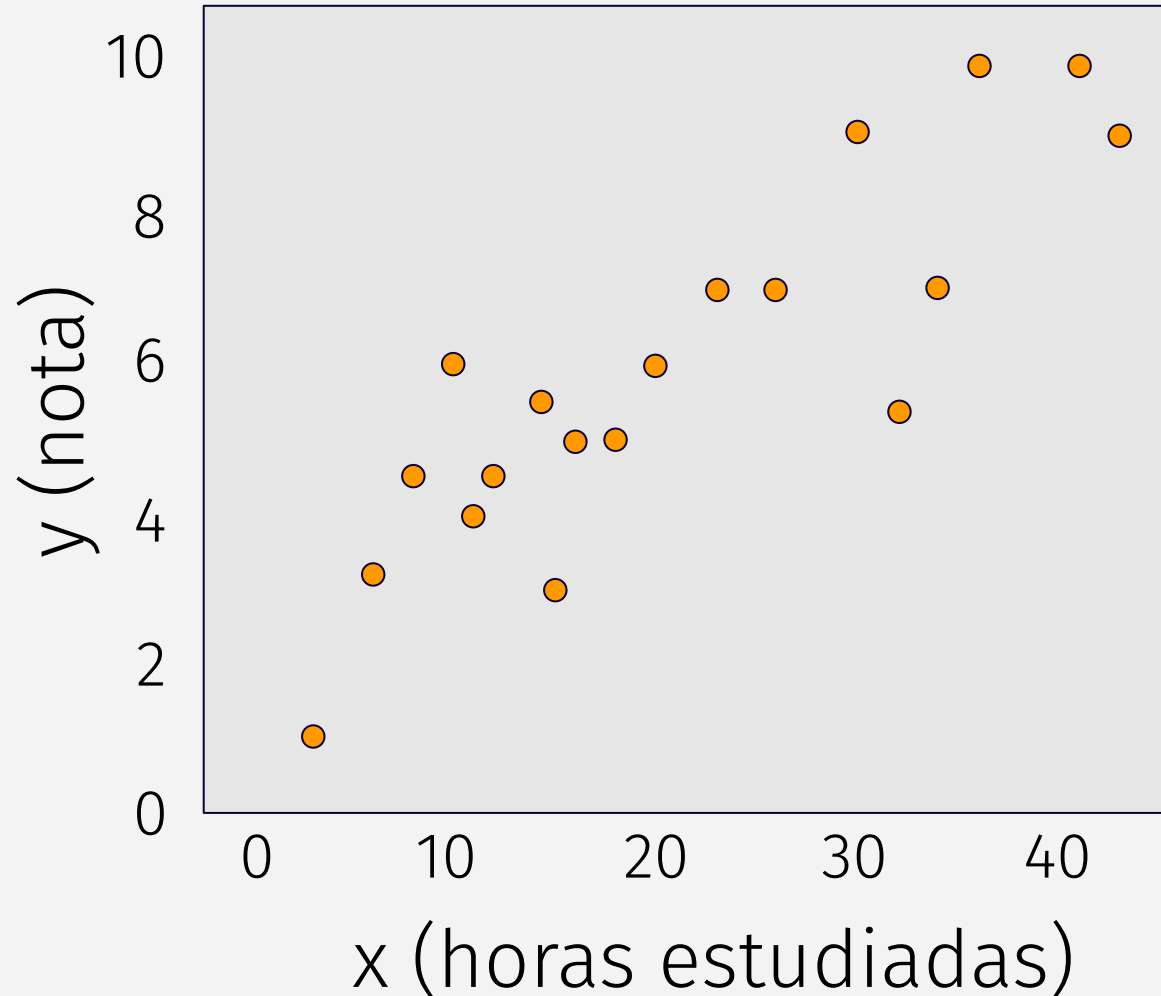
Introducción

Problema de Regresión

Ejemplos

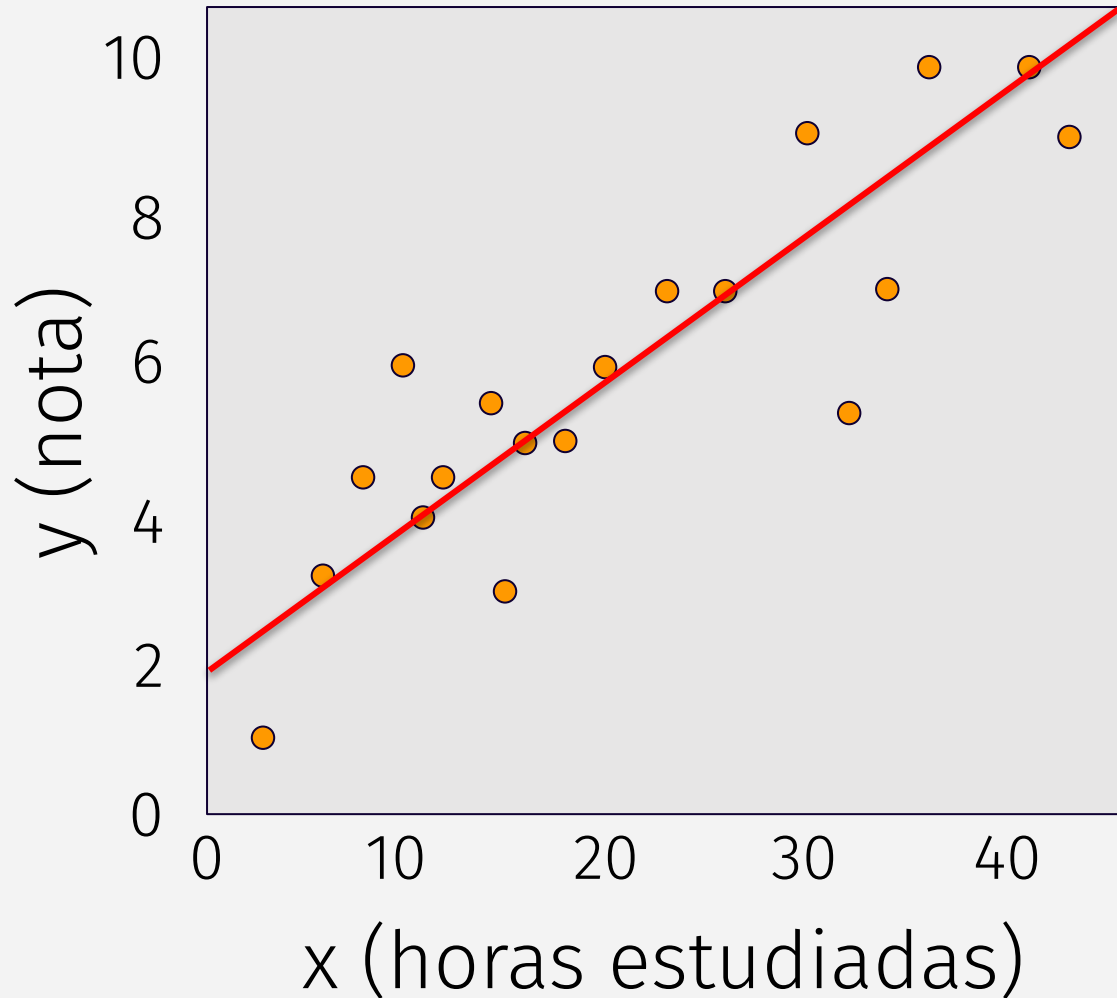
Horas	Nota
2	1
5	3.2
7	4.5
9	6
10	4
11	4.5
13.4	5.5
14	3
15	5

Visualización



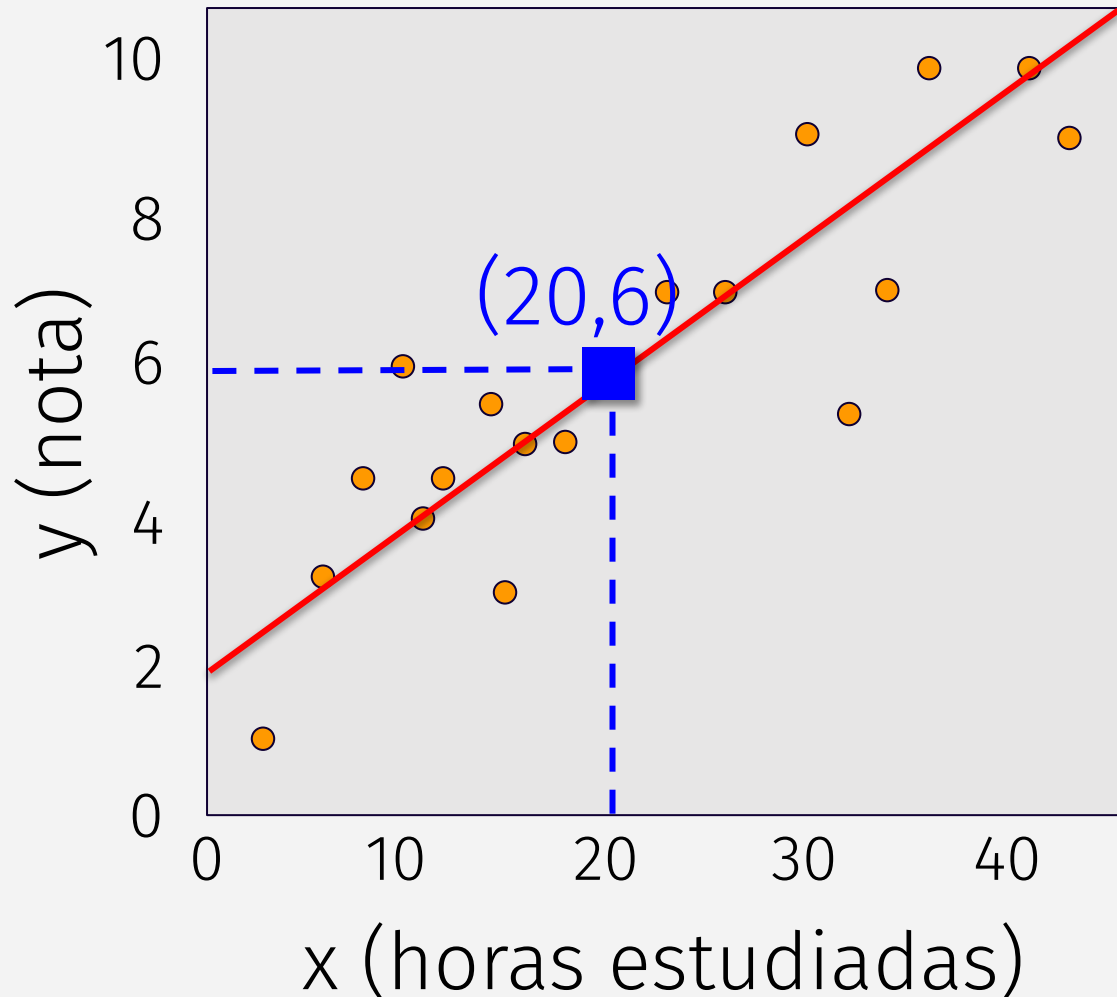
Si un nuevo alumno estudió $x = 20hs$, ¿cuál será su nota?

Modelo de Regresión Lineal (RL)



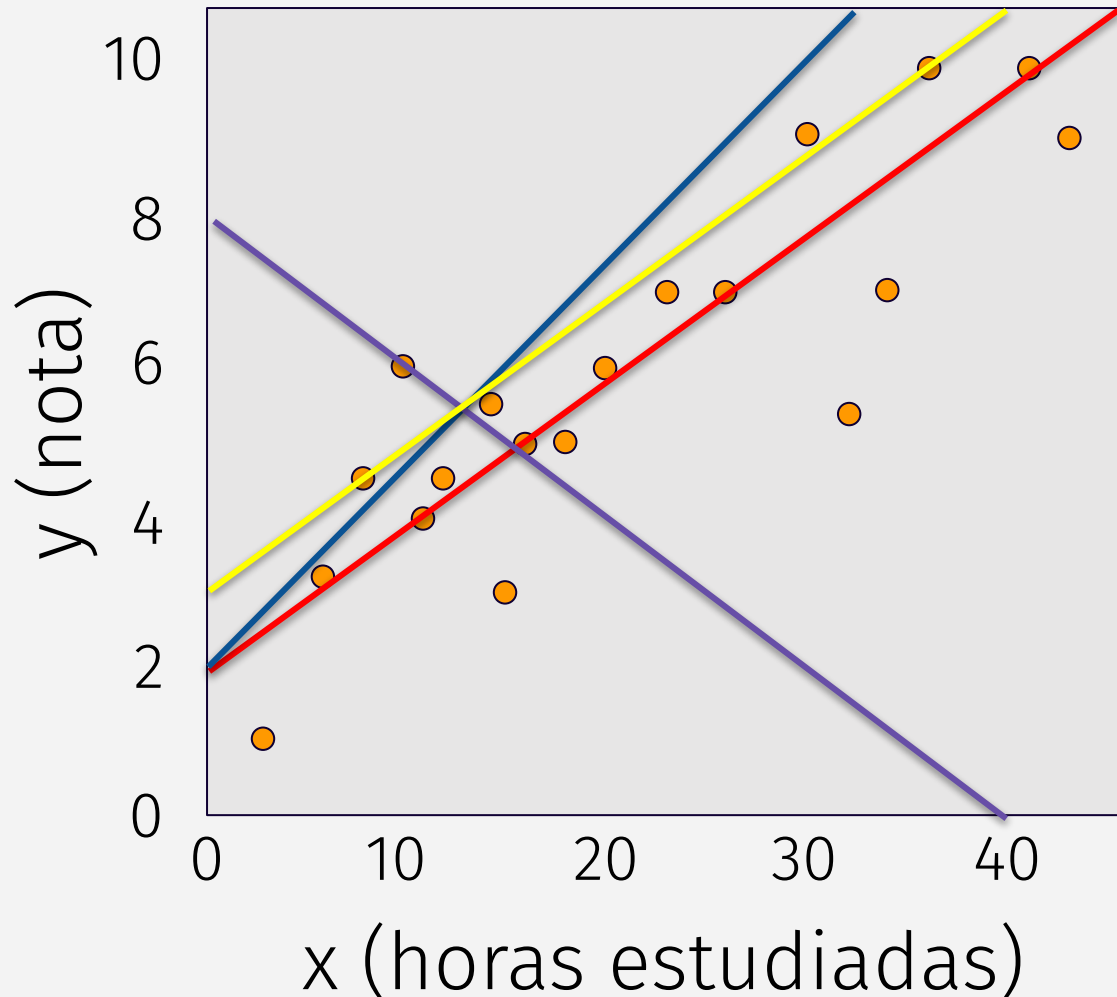
- Asunción
 - Relación $x - y$ es lineal
 - Modelo más simple
- Modelo
 - $y = m \times x + b$
 - 0
 - $f(x) = m \times x + b$

¿Cómo predecir con modelo de Regresión Lineal?



- Supongamos
 - $m = 0.5$
 - $b = 2$
 - $f(x) = 0,2 \times + 2$
- ¿Qué nota **predice** el modelo si **$x=20$** ?
 - $f(20) = m \times 20 + b$
 - $= 0,2 \times 20 + 2$
 - $= 6$
- Predice **$y=nota=6$**

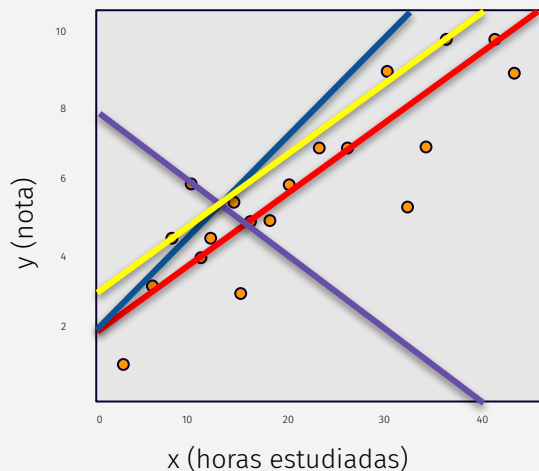
Valores de **m** y **b**



- Valores de **m** y **b** definen la recta
 - **m** = 0.20, **b** = 2
 - **m** = 0.20, **b** = 3
 - **m** = 0.26, **b** = 2
 - **m** = -0.20, **b** = 8
- **parámetros** del modelo
 - **m** indica la pendiente
 - **b** la ordenada a la origen

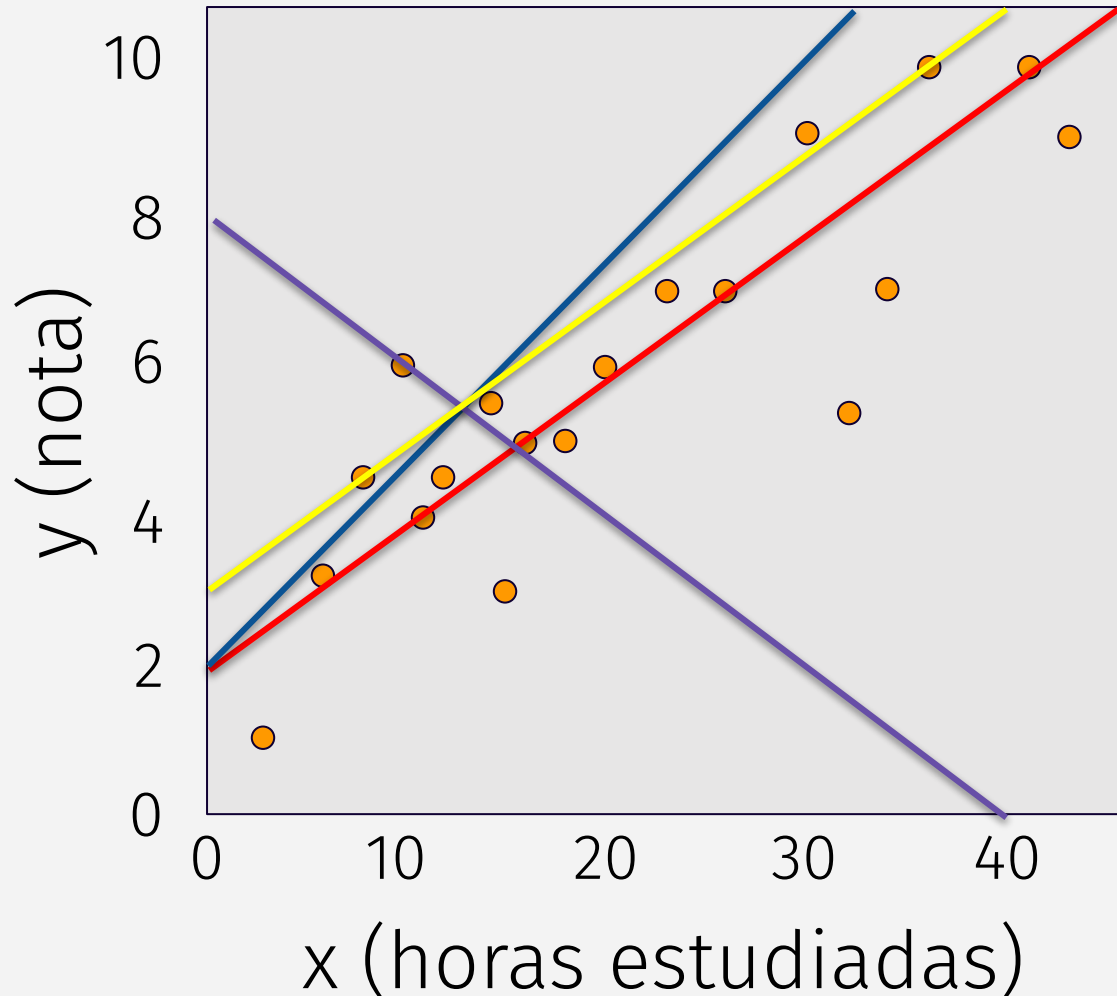
Valores de **m** y **b**

- Valores de **m** y **b** definen la recta
 - - **m** = 0.20, **b** = 2
 - - **m** = 0.20, **b** = 3
 - - **m** = 0.26, **b** = 2
 - - **m** = -0.20, **b** = 8



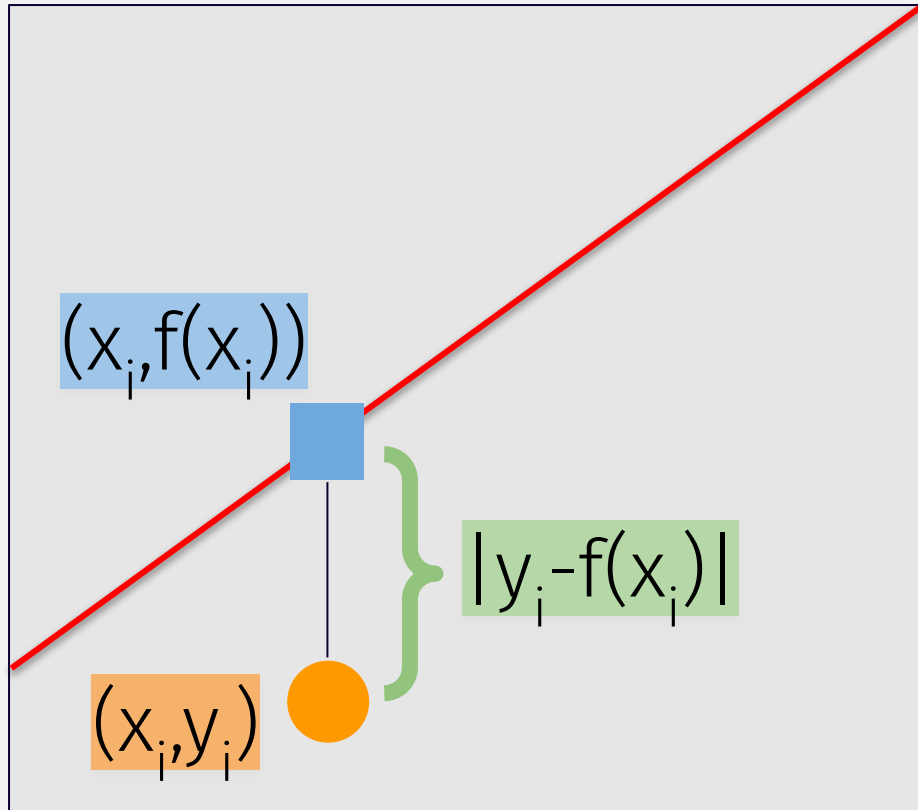
- Valores de **m** y **b** definen la recta
 - - $f(x) = 0.20 \times x + 2$
 - - $f(x) = 0.20 \times x + 3$
 - - $f(x) = 0.26 \times x + 2$
 - - $f(x) = 0.20 \times x + 8$
- Cada recta es un modelo **distinto**
- Misma familia de modelos
 - **Regresión Lineal**

Elección de **m** y **b**. Función de **error**



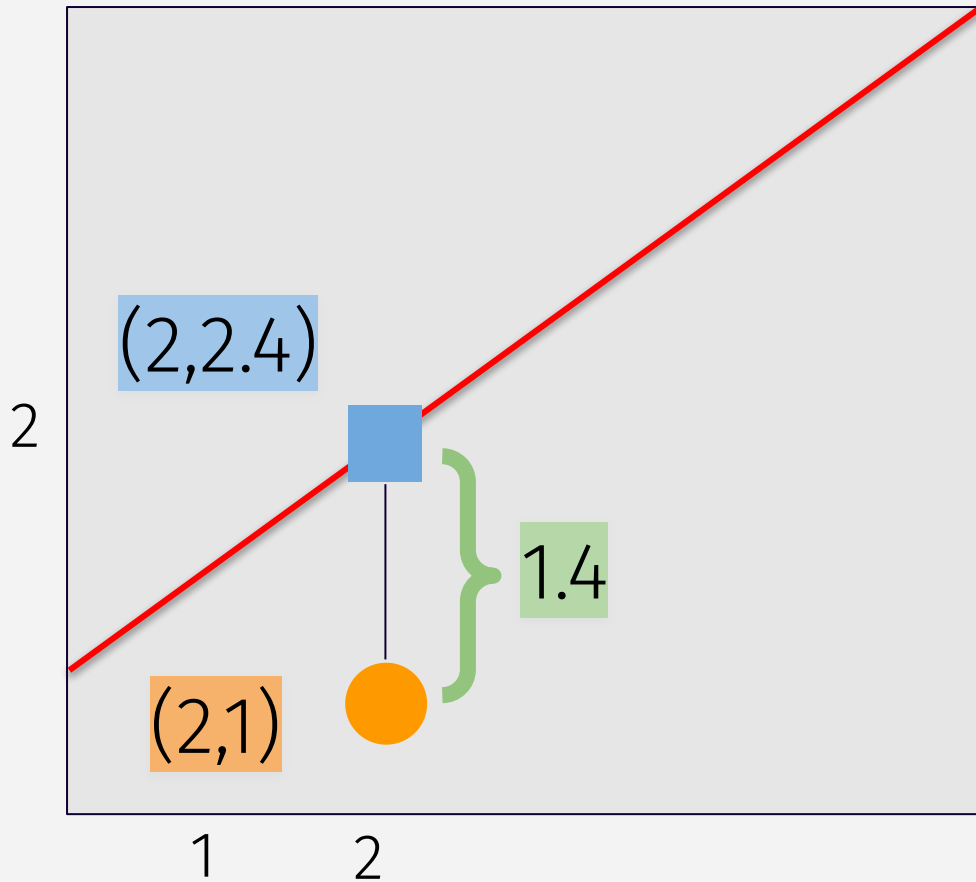
- Dado un conjunto de ejemplos
 - ¿Cómo elegir **m** y **b**?
 - Función de **puntaje** para cada **m** y **b**
 - Función de **error** **$E(m,b)$**
 - Ejemplo
 - $E(-) = 1.62$
 - $E(-) = 1.72$
 - $E(-) = 5.12$
 - $E(-) = 1.44$

$E_i(m,b)$: Error cuadrático del modelo para el ejemplo i



- Primero, definimos el **error** para **un solo ejemplo**
- $E_i(m,b)$
 - Error del ejemplo i para m y b
 - Distancia cuadrática entre
 - y_i : valor esperado
 - $f(x_i)$: valor predicho
- $E_i(m,b) = (y_i - f(x_i))^2$
 $= (y_i - m x_i + b)^2$

$E_i(m,b)$: Error cuadrático del modelo para el ejemplo i



- $E_i = (y_i - f(x_i))^2$

- Ejemplo

- Estudió 2 horas, nota 1

- $x_i = 2$

- $y_i = 1$

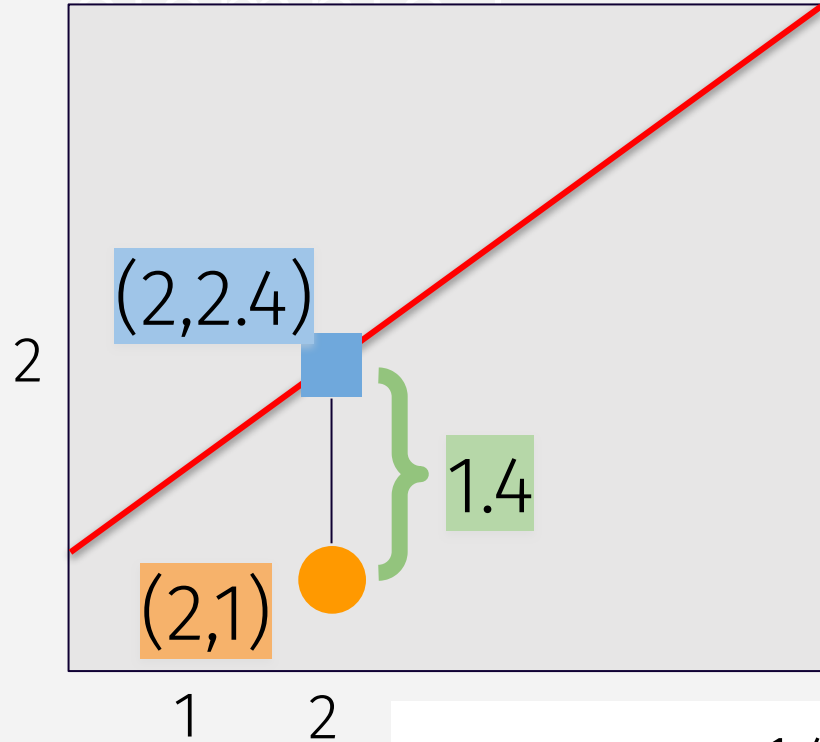
- $f(x_i) = f(2) = 0.2 \times 2 + 2$
 $= 2.4$

- $y_i - f(x_i) = 1 - 2.4 = -1.4$

- $(y_i - f(x_i))^2 = (-1.4)^2 = 1.96$

- $E_i = 1.96$

$E_i(m,b)$: Error cuadrático del modelo para el



$$y_i - f(x_i) = -1.4$$

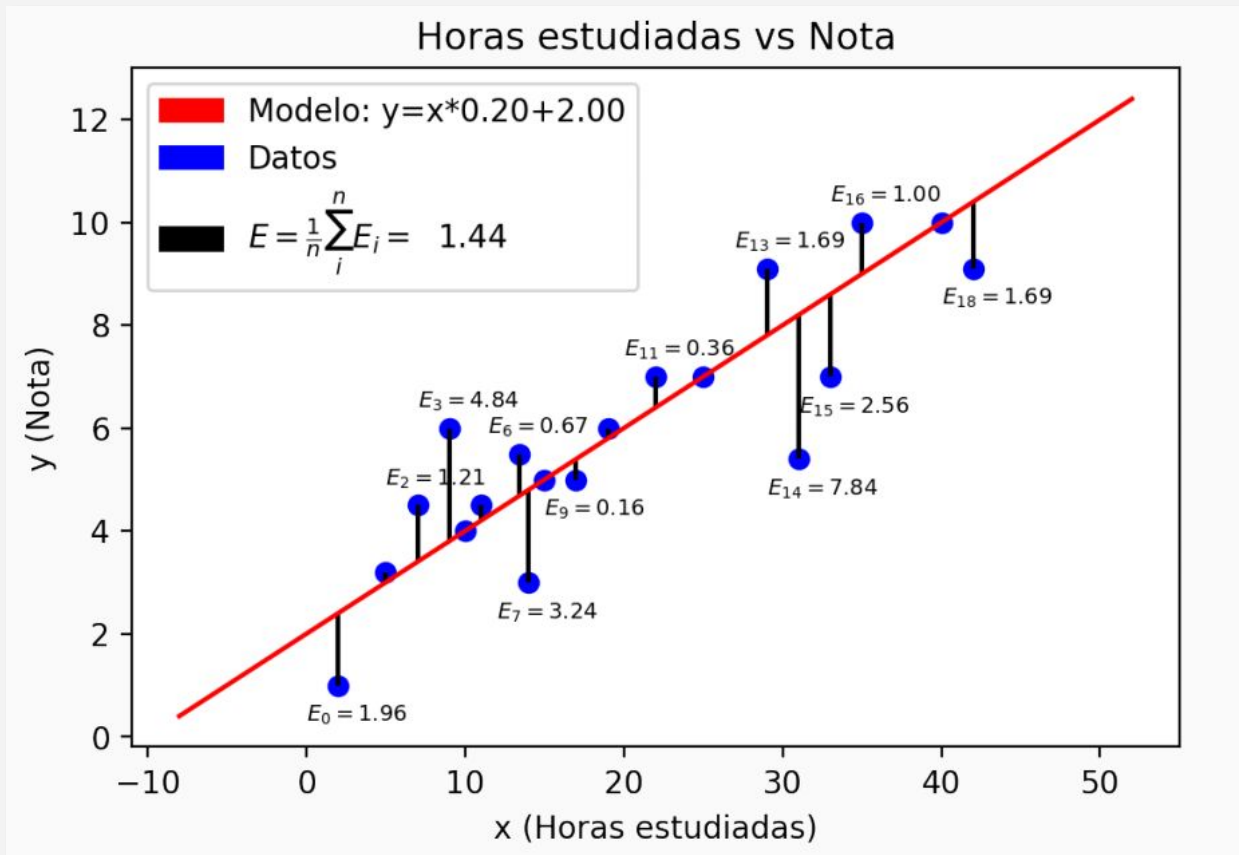
$$|y_i - f(x_i)| = 1.4$$

$$(y_i - f(x_i))^2 = 1.96$$

$$(y_i - f(x_i))^4 = 3.84$$

- $E_i = (y_i - f(x_i))^2$
 - ¿Por qué esta función de error?
- ¿Por qué no usar $y_i - f(x_i)$?
 - Valores negativos
- ¿Por qué no usar $|y_i - f(x_i)|$?
 - Valor absoluto
 - No es una función derivable
 - Difícil de optimizar
- ¿Qué efecto tiene el **2**?
 - Penaliza más errores grandes
 - $0.5^2 = 0.25$, $1^2 = 1$, $5^2 = 25$
- ¿Por qué no usar $(y_i - f(x_i))^4$?
 - Posible, pero penalizaría demasiado

$E(m,b)$: Error cuadrático medio de los ejemplos



- $E(m,b)$: Error cuadrático medio
 - Función de error **natural** para RL
 - Promedio de los E_i
- $E(m,b) = (1/n) \sum_i^n E_i(m,b)$

Ejemplo de cálculo de **E** para **m=0.2** y **b=2**

$$m = 0.2$$

$$b = 2$$

estudio nota

2

1

10

4

14

5

30

9

40

10

$$f(x_i)$$

$$f(x_i) - y_i$$

$$E_i$$

$$0.2 * 2 + 2 = 2.4$$

$$2.4 - 1 = 1.4$$

$$(1.4)^2 = 1.96$$

$$0.2 * 10 + 2 = 4$$

$$4 - 4 = 0$$

$$(0)^2 = 0$$

$$0.2 * 14 + 2 = 4.8$$

$$4.8 - 5 = -0.2$$

$$(-0.2)^2 = 0.04$$

$$0.2 * 30 + 2 = 8$$

$$8 - 9 = -1$$

$$(-1)^2 = 1$$

$$0.2 * 40 + 2 = 10$$

$$10 - 10 = 0$$

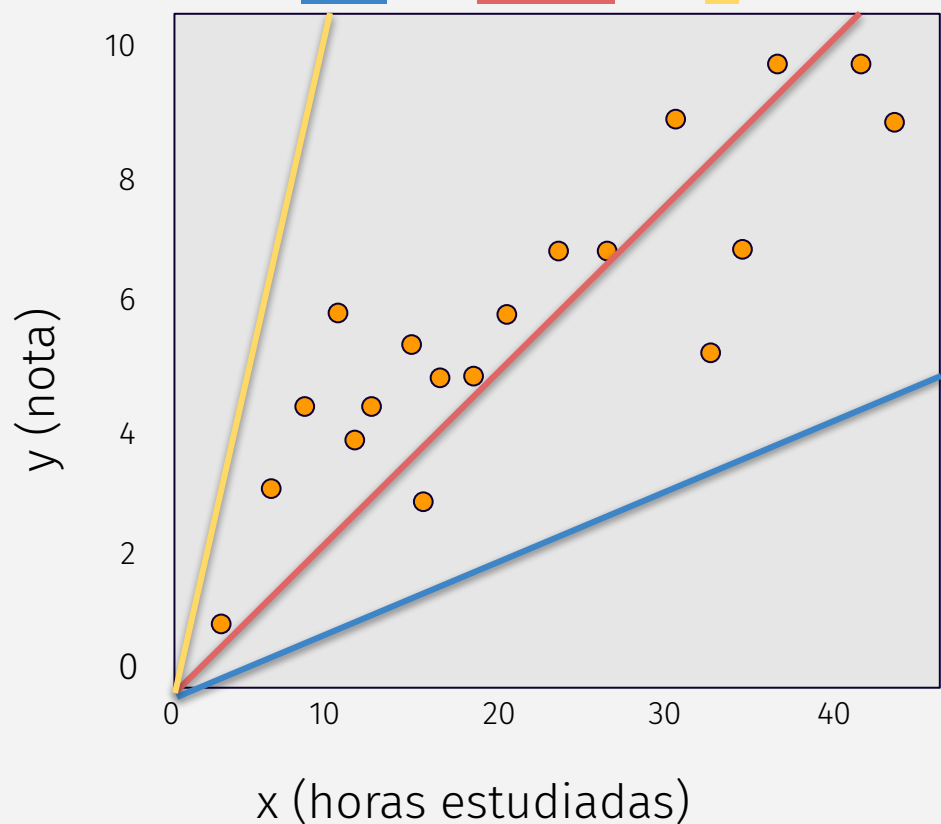
$$(0)^2 = 0$$

Error cuadrático medio

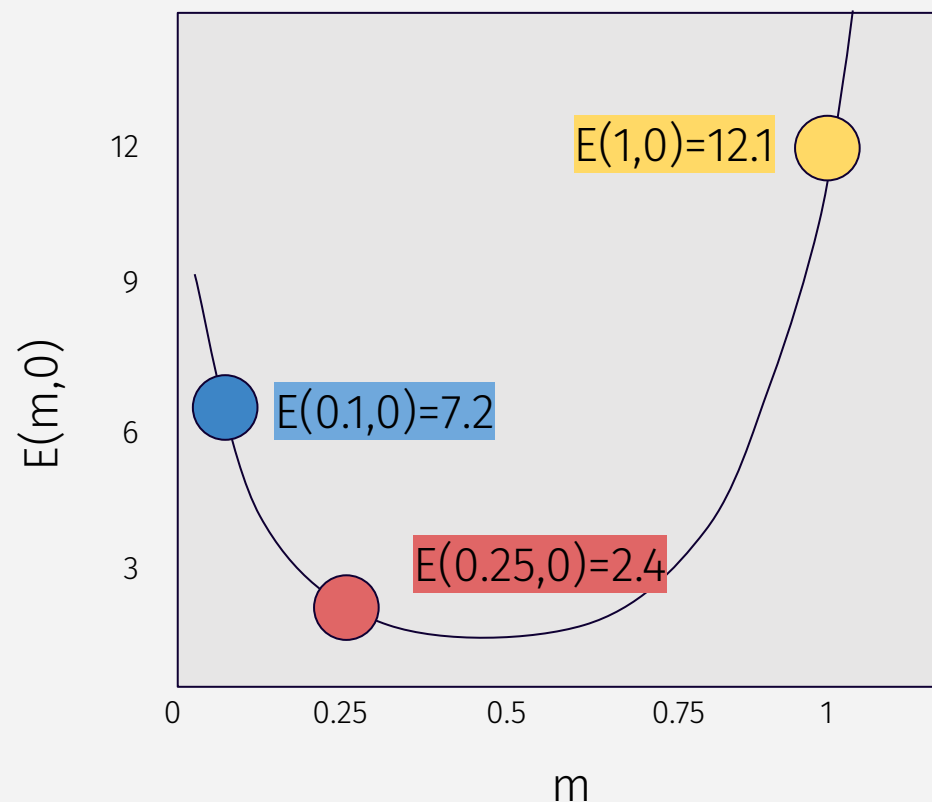
$$E = \frac{1}{n} \sum_i^n E_i = \frac{1.96 + 0 + 0.04 + 1 + 0}{5} = \frac{3}{5} = 0.6$$

Comprendiendo $E(m,b)$

- Asumamos $b=0$
 - Queda 1 parámetro: m
 - Probamos con $m=...$
 - 0.1 o 0.25 o 1

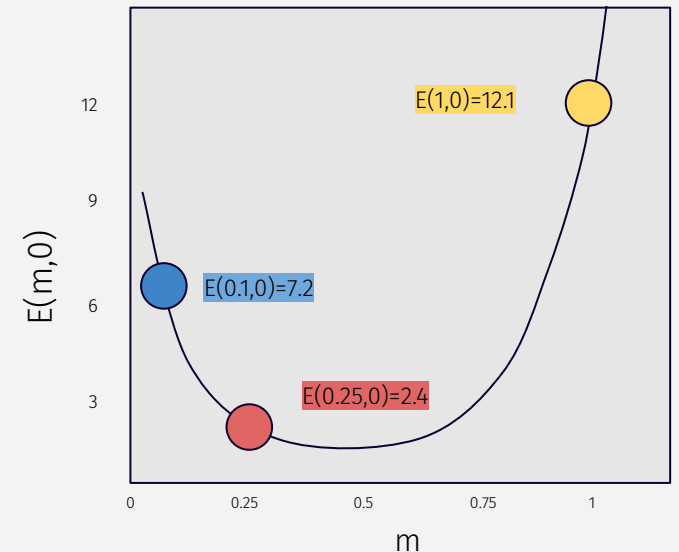
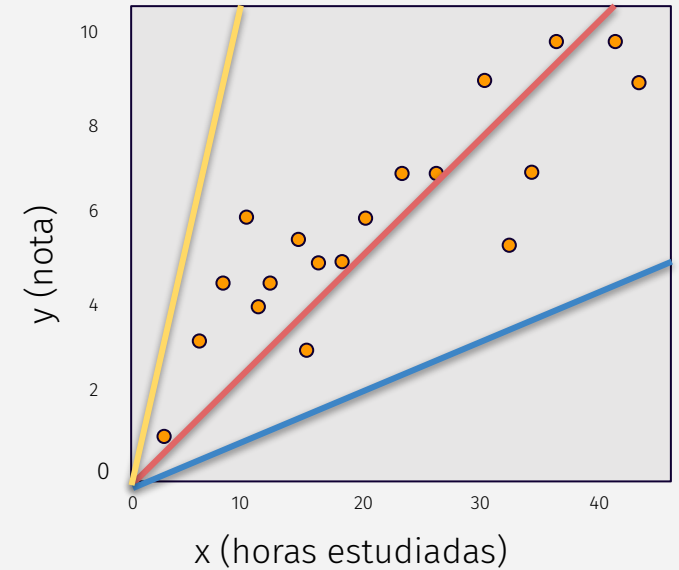


- Error $E(m,b) = E(m,0)$
 - 1D
 - Forma una parábola



Resumen

- Regresión Lineal
 - $f(x) = m x + b$
 - Modelo más simple
 - Asume relación lineal entre x e y
 - Aproximada
- Parámetros
 - **m** pendiente
 - **b** ordenada al origen
- Función de error **E**
 - Error cuadrático medio
 - Promedio de error de cada ejemplo E_i
 - Permite elegir valores óptimos de **m** y **b**



Ejercicio: Archivo **Regresión Lineal - Modelo.ipynb**

- **Probar**

- Modificar los parámetros **m** y **b**. Observar como cambia la recta y el error
- Modificar el valor de la variable **new_x**, observar los valores de $f(\mathbf{new_x})$ que predice el modelo (punto verde)

- **Interpretar**

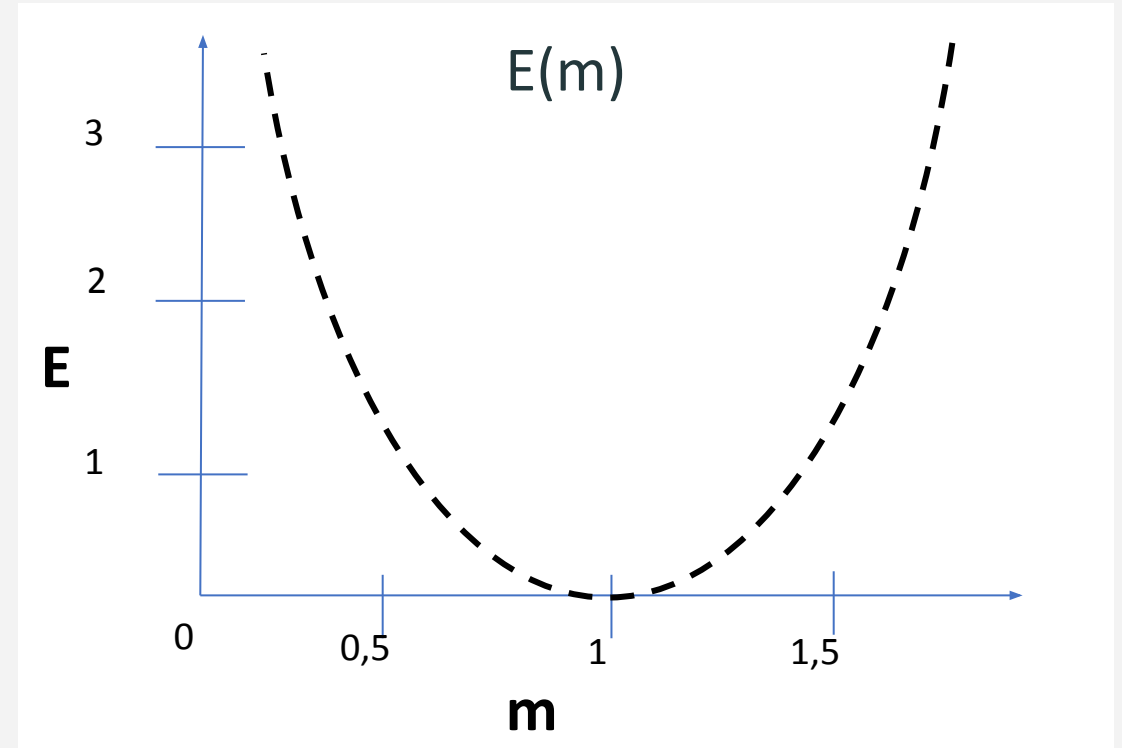
- ¿Qué me dice el modelo respecto a la nota que obtienen los alumnos que no estudiaron nada?
- ¿Cuánto puedo esperar que aumente mi nota por cada hora de estudio?
- ¿Cuándo es más fácil la materia? (en relación a m)
- ¿Cuándo baja el error? ¿Puede ser 0?

Regresión Lineal

Forma de la función de error

Prueba de que $E(\mathbf{m})$ es una parábola

- Error cuadrático medio
 - Función de error más común para RL
- 2 parámetros: $\mathbf{m}$ y $\mathbf{b}$
 - $E(\mathbf{m}, \mathbf{b}) = (1/n) \sum_i^n E_i(\mathbf{m}, \mathbf{b})$
 - $E_i(\mathbf{m}, \mathbf{b}) = (y_i - \mathbf{m} x_i + \mathbf{b})^2$
- Si $\mathbf{b}$ está fijo, 1 parámetro
 - $E(\mathbf{m})$ es una parábola
 - Único mínimo (global)



Prueba de que $E_i(m)$ es una parábola

- Si b está fijo, 1 parámetro

- $E_i(m) = (y_i - m x_i + b)^2$

- $= y_i^2 - 2(m x_i + b) y_i + (m x_i + b)^2$

- $= y_i^2 - 2 m x_i + 2b y_i + (m x_i)^2 + 2m x_i b + b^2$

- Si $C_i = y_i^2 + 2b y_i + b^2$

- $= -2 m x_i + (m x_i)^2 + 2m x_i b + C_i$

- Agrupando las m

- $= (-2+2b) m x_i + m^2 x_i^2 + C_i$

- Si $D_i = (-2+2b) x_i$ y $F_i = x_i^2$

- $E_i(m) = D_i m + F_i m^2 + C_i$

- Función cuadrática: parábola

- Sólo para el ejemplo i

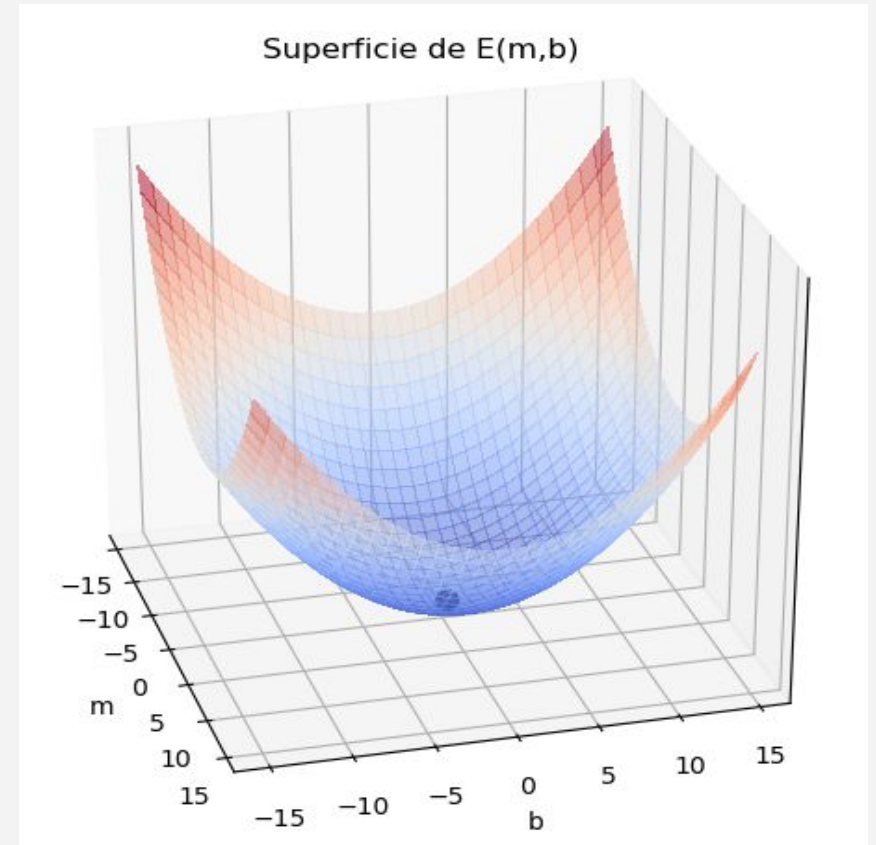
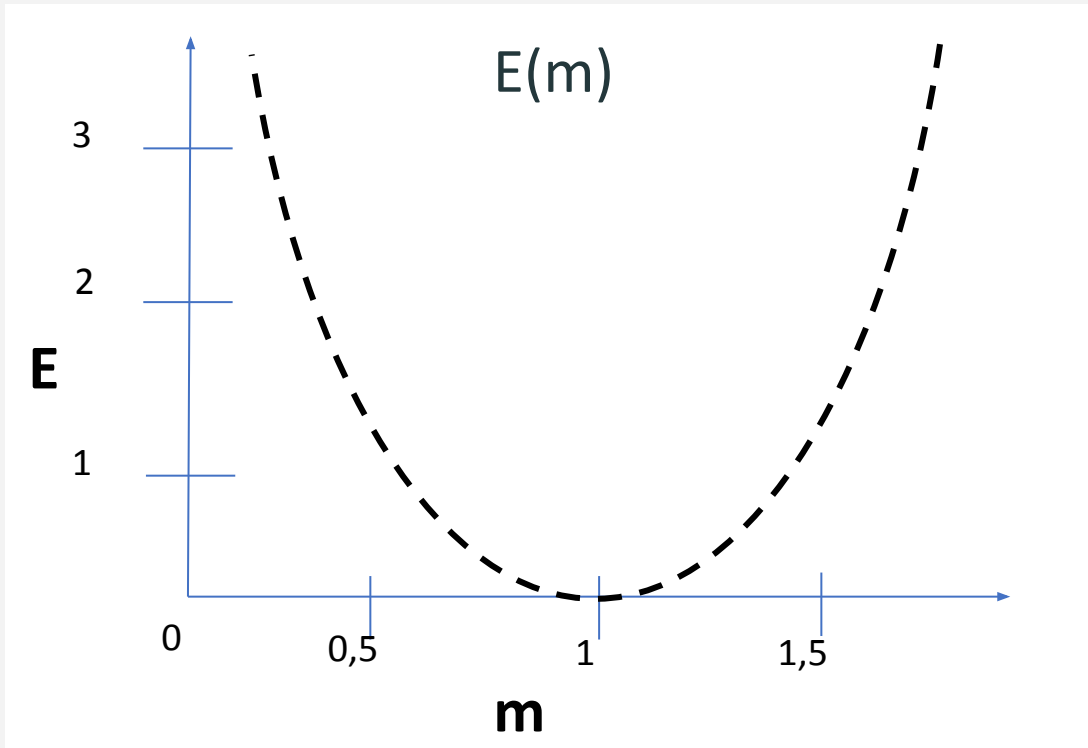
Prueba de que $E(\mathbf{m})$ es una parábola

- $E_i(\mathbf{m}) = D_i \mathbf{m} + F_i \mathbf{m}^2 + C_i$
 - Reemplazamos en $E(\mathbf{m})$
- Entonces
 - $E(\mathbf{m}) = (1/n) \sum_i^n E_i(\mathbf{m})$
 - $= (1/n) \sum_i^n D_i \mathbf{m} + F_i \mathbf{m}^2 + C_i$
 - $= (1/n) [\sum_i^n D_i \mathbf{m} + \sum_i^n F_i \mathbf{m}^2 + \sum_i^n C_i]$
 - $= \mathbf{m} (1/n \sum_i^n D_i) + \mathbf{m}^2 (1/n \sum_i^n F_i) + 1/n \sum_i^n C_i$
 - Si $D = 1/n \sum_i^n D_i$
 - Si $F = 1/n \sum_i^n F_i$
 - Si $C = 1/n \sum_i^n C_i$
 - $E(\mathbf{m}) = \mathbf{m} D + \mathbf{m}^2 F + C$
 - $E(\mathbf{m})$ es una parábola

$E(m,b)$ es un **paraboloide**

- Si b está fijo, 1 parámetro
 - E es una parábola
 - Único mínimo (global)

- Si b no está fijo, 2 parámetros
 - E es un paraboloide
 - (parábola en 2D)
 - Único mínimo (global)



Introducción a la Optimización de Funciones

Entrenamiento = Aprendizaje = Optimización

- Aprendizaje Supervisado
 - Función de error **E**
 - Ejemplos con:
 - Valores de entrada $\mathbf{x}_i$
 - Valores de salida $\mathbf{y}_i$
 - Buscar **parámetros óptimos** en base a **ejemplos** y **E**

Ejemplos

Entrada	Salida
2	1
5	3.2
7	4.5
9	6

**Aprendizaje =
Optimización** de **E**

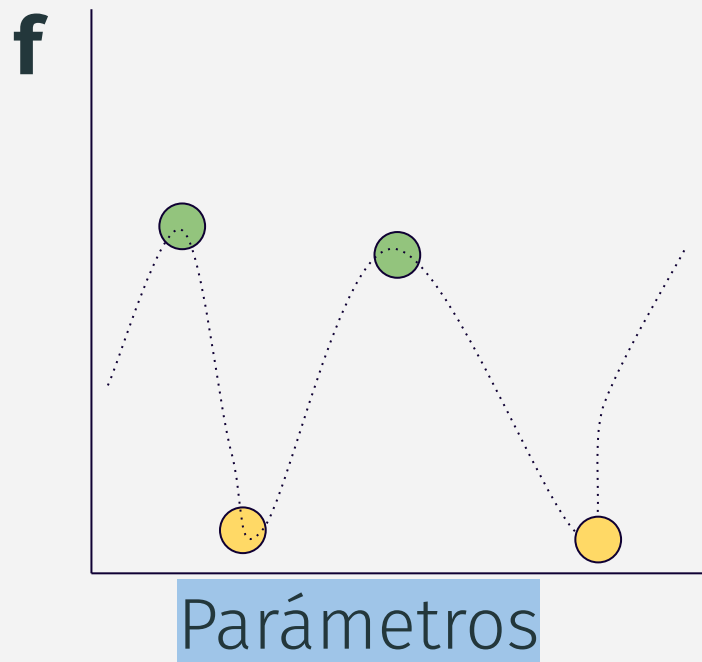


Modelo

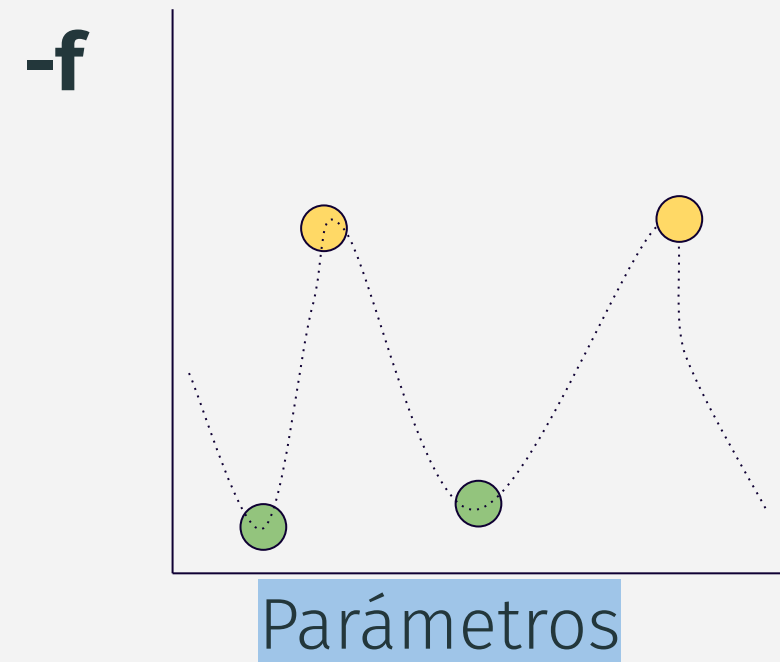
*Parámetros
óptimos*

Optimización de funciones

- ¿Qué significa **optimizar** una función **f**?
 - Buscar **algún** mínimo
 - Variando sus parámetros



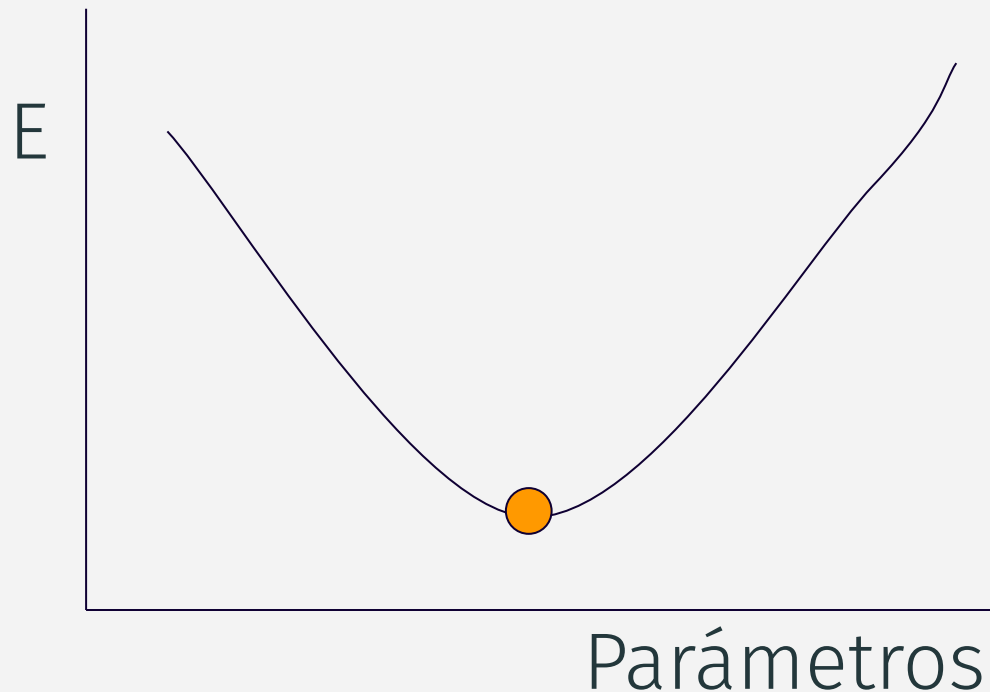
- ¿Por qué *minimizar*?
 - *Maximizar* **f** es equivalente a *minimizar* **-f**
 - Puedo buscar máximos de **f** en los mínimos de **-f**



Clasificación de funciones por convexidad

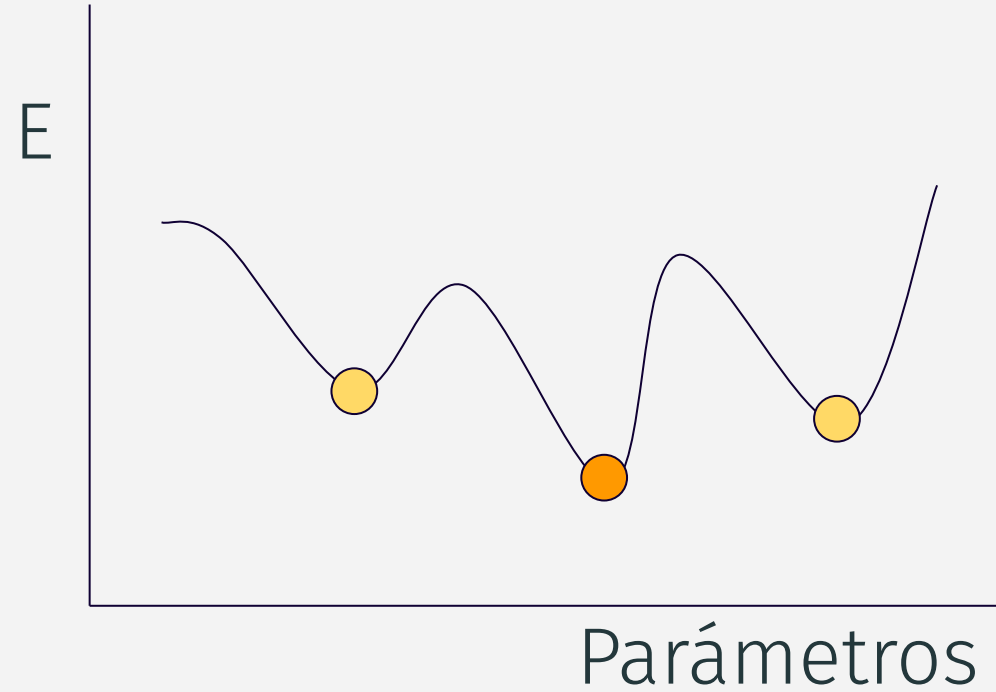
- Función **convexa**

- **Único** mínimo (global)
- **E** para **regresión lineal** siempre es convexa :)



- Función **no-convexa**

- **Varios** mínimos
 - Global (medio)
 - Local (otros dos)
- **E** para **redes** es no-convexa :(



Dificultades principales

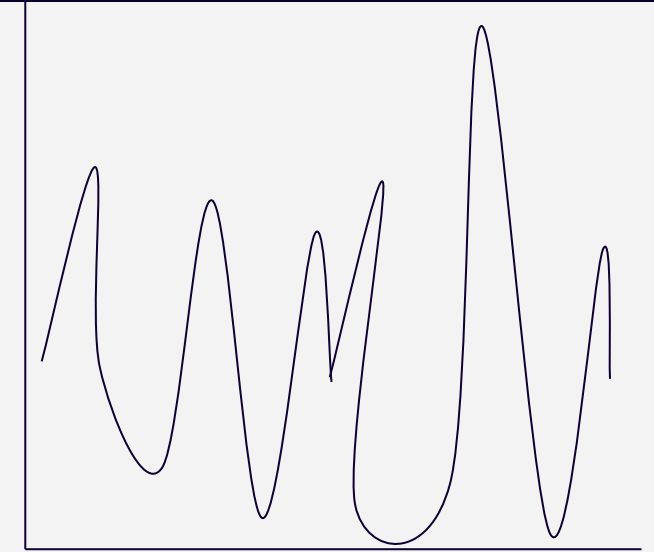
- Más de 2 parámetros (99.9% de los problemas)
 - No se puede visualizar!
 - **Algoritmos de Optimización**



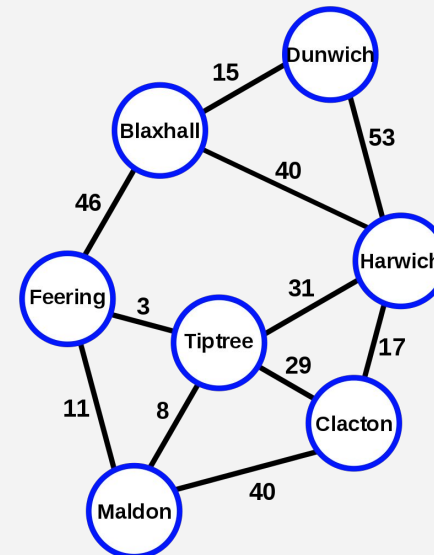
Tipos de Algoritmos de Optimización

- Generales
 - Pocas asunciones sobre **f**
 - Mayor tiempo de ejecución
 - Pocas garantías
 - Ejemplos
 - Fuerza bruta
 - Búsqueda aleatoria
- Especializados
 - Básicamente, todo lo contrario
 - **Pero no siempre existen/son aplicables**
 - Ejemplo
 - Camino mínimo en un grafo

f

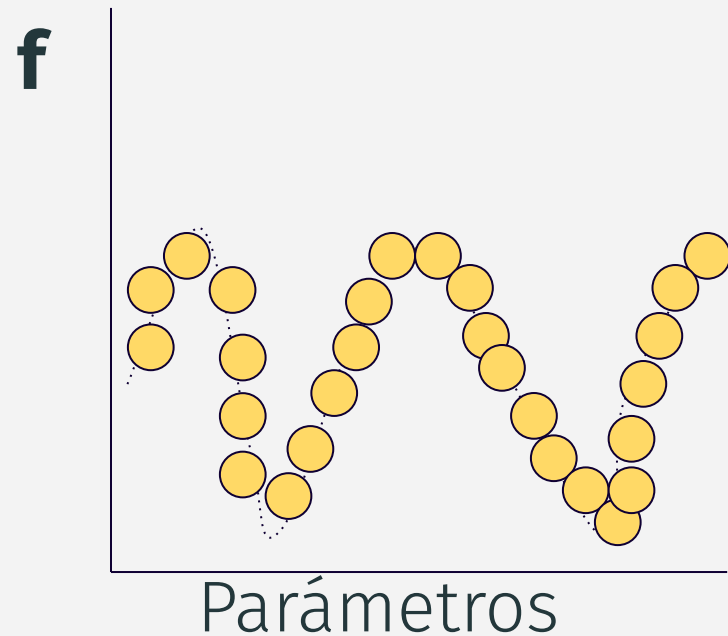


Parámetros

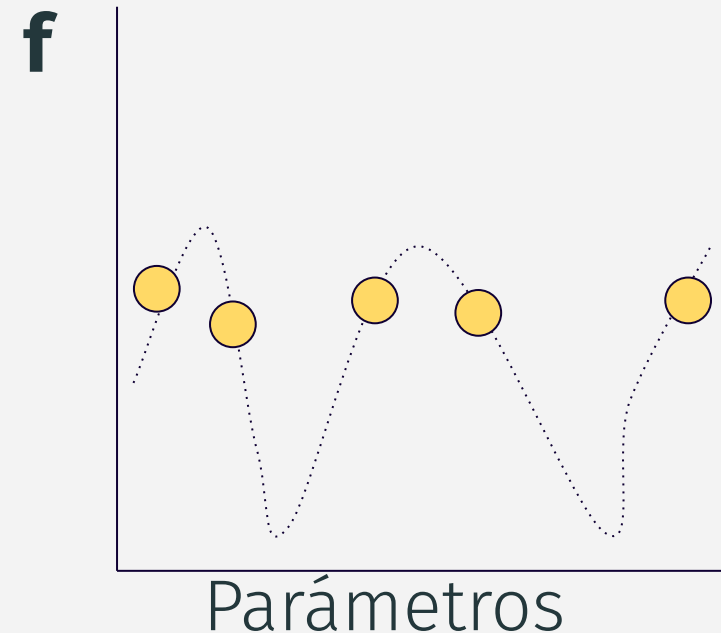


Algoritmos generales **básicos** de optimización

- Fuerza bruta
 - Probar “todos” los valores
 - Quedarse con el mejor
 - ¿Parámetros continuos?
 - ¿Muchos parámetros?

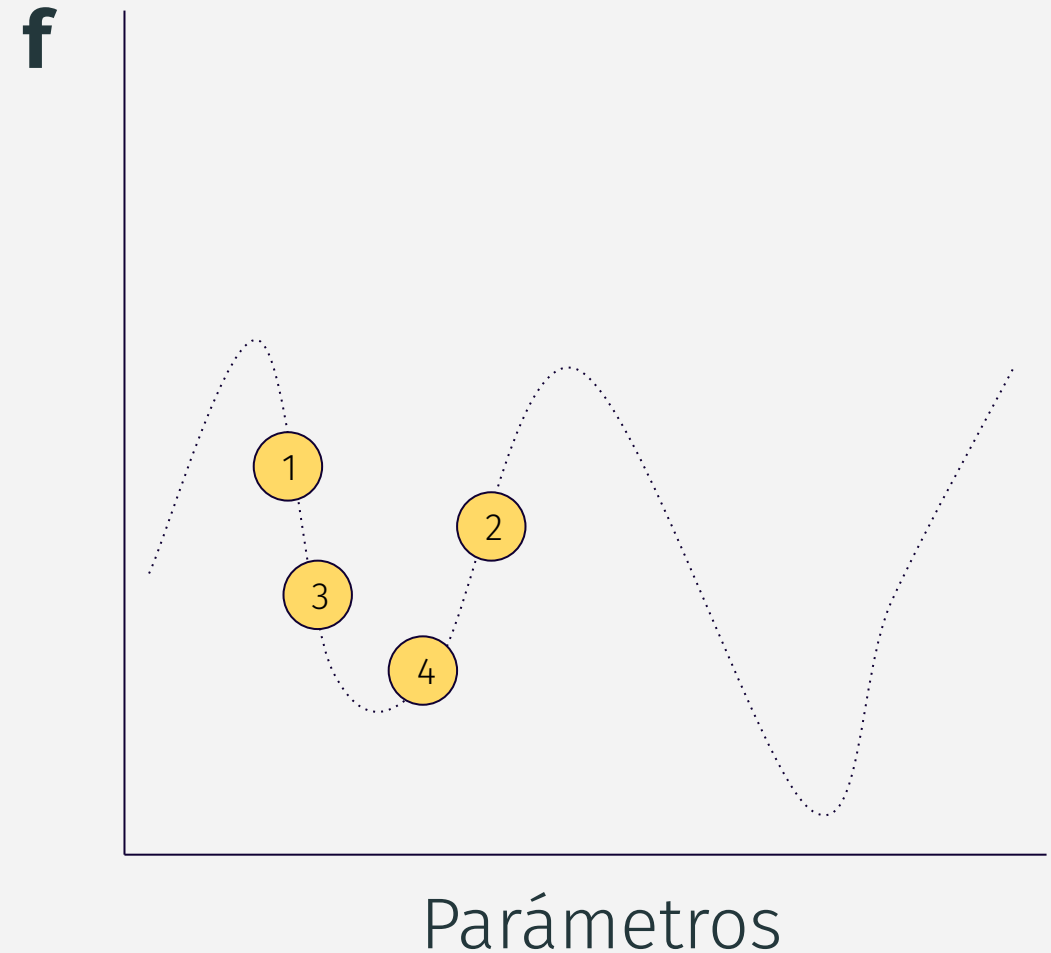


- Búsqueda aleatoria
 - Evaluar valores aleatorios
 - Quedarse con el mejor
 - ¿Cómo generar valores aleatorios?



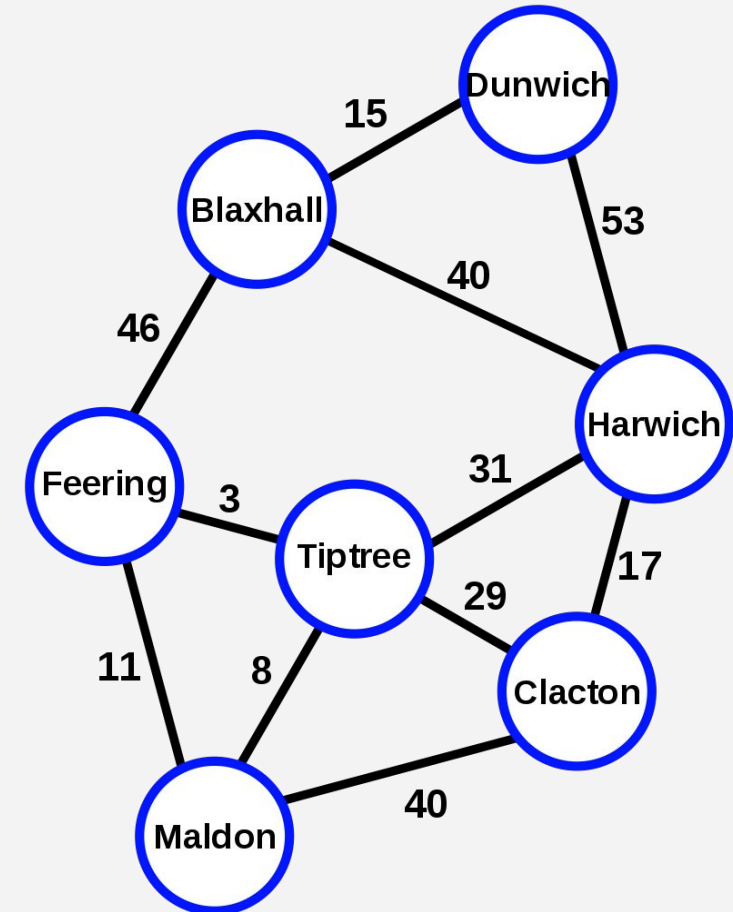
Dificultades principales

- Coste computacional de evaluar la función de error
- Criterio de convergencia
 - Convergencia = terminar
 - Explotación vs exploración
 - 4 evaluaciones
 - ¿continuar o parar?
 - Varios mínimos



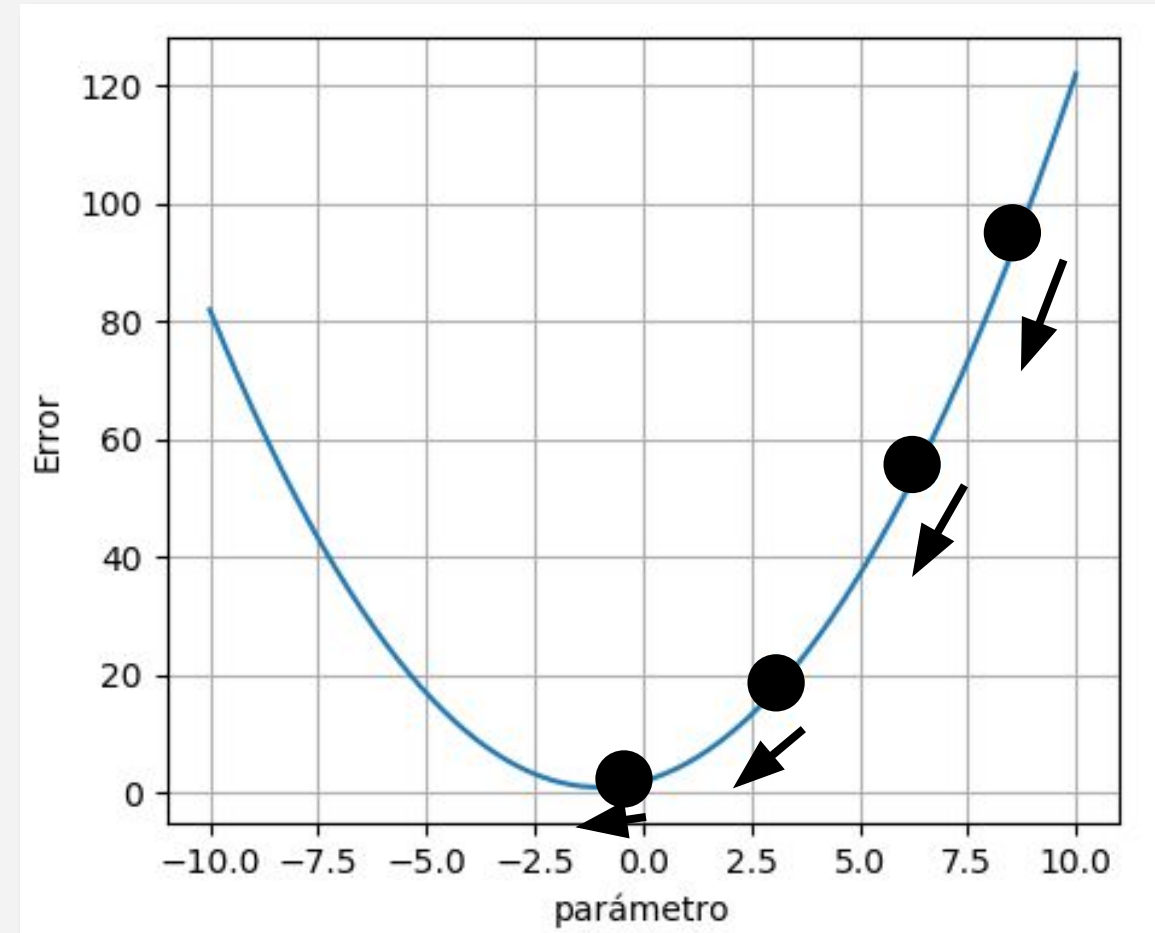
Algoritmos de Optimización Especializados

- Ejemplo: buscar el camino más corto en un grafo
 - Datos: son el grafo **G**
 - Parámetros: el camino **C** a tomar
 - Función de error: Longitud camino **L(C)**
 - Encontrar camino **C** $\in$ **G**
 - Con el menor valor de **L(C)**
 - Solución **única**
 - *Varios* algoritmos **especializados**
 - Dijkstra
 - Bellman-Ford
 - etc..



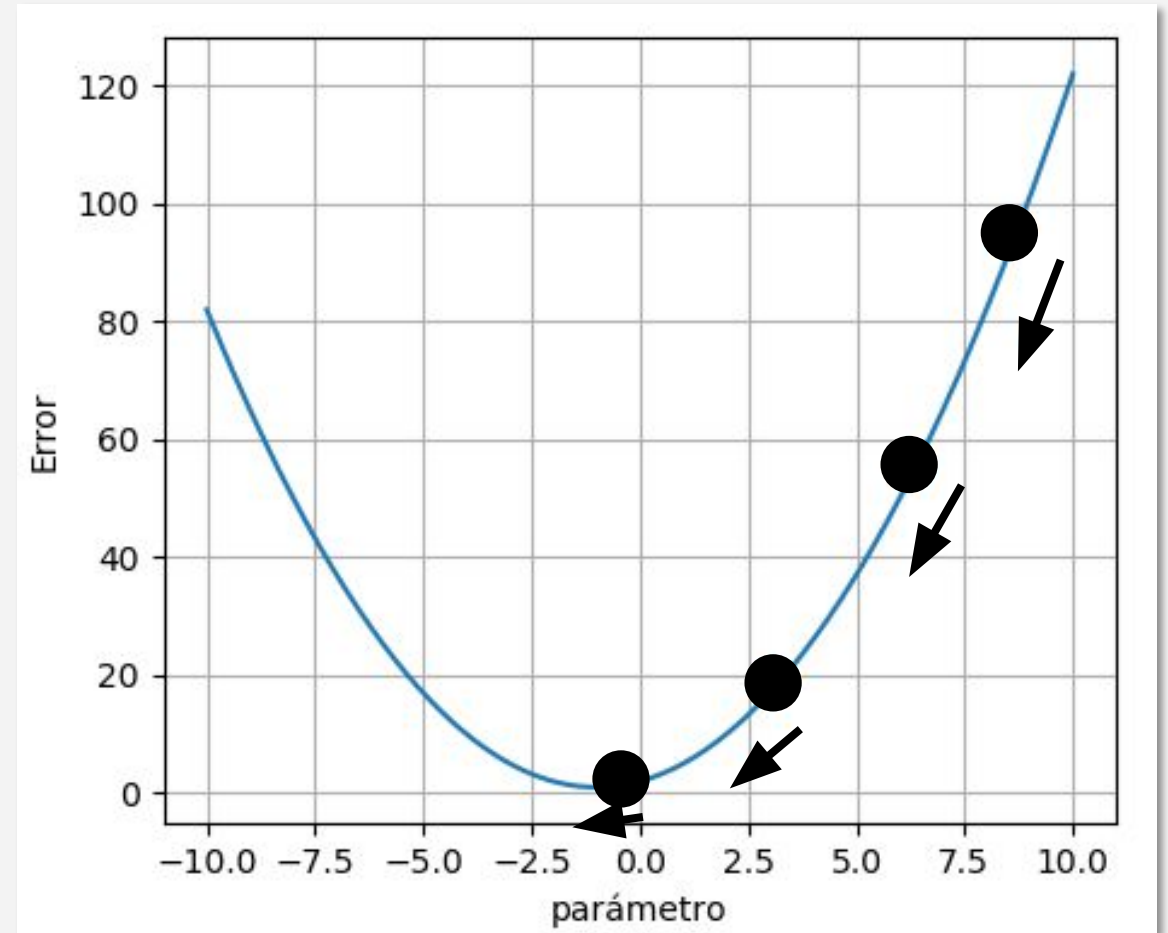
Descenso de Gradiente (DG)

- Algoritmo general
 - Asunción: **f** es **derivable**
 - Puede tener puntos de discontinuidad
 - Utilizar el **gradiente** o **derivada** para guiar la optimización
 - **Gradiente** en un punto
 - **Dirección** de crecimiento de la función
 - Problemas
 - Mínimos locales
 - Escalable a $n > \text{millones}$



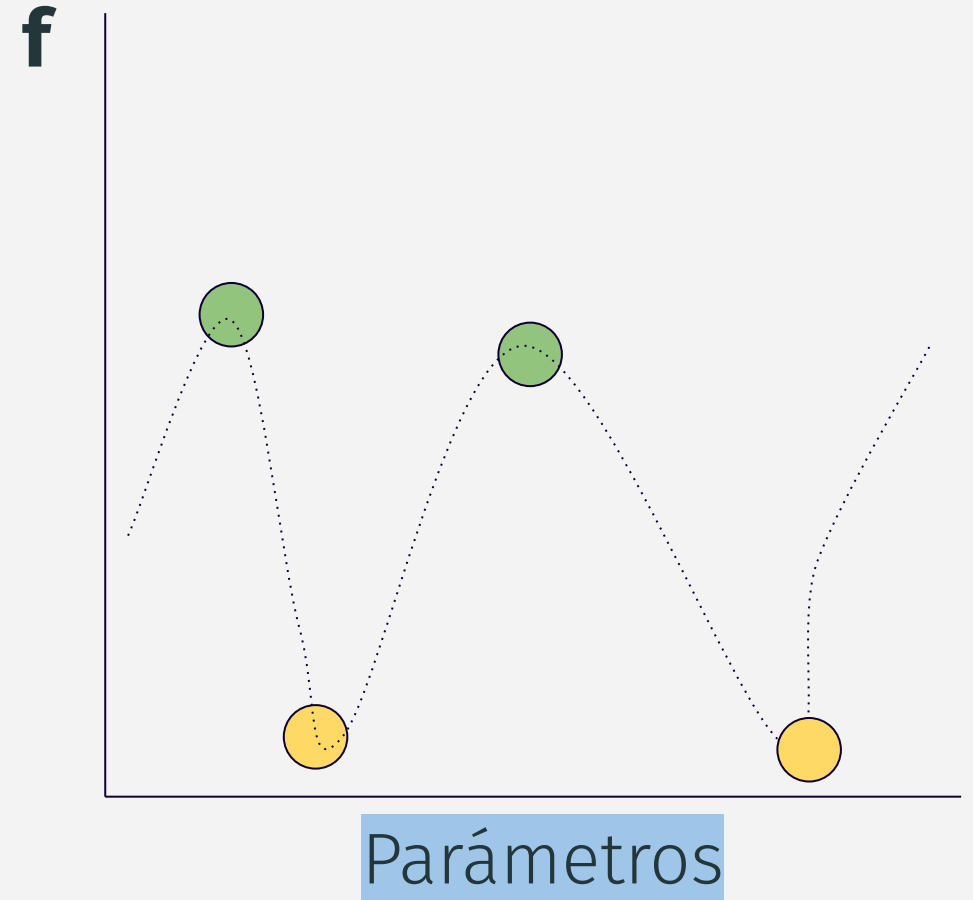
Descenso de gradiente (DG) : idea básica

- Empezar en un **valor aleatorio** del parámetro
 - Iterar hasta que derivada=0
 - Calcular **gradiente** o **derivada**
 - Indica dirección de maximización
 - Moverse en dirección opuesta
- Valor final: mínimo local (posiblemente global)



Resumen

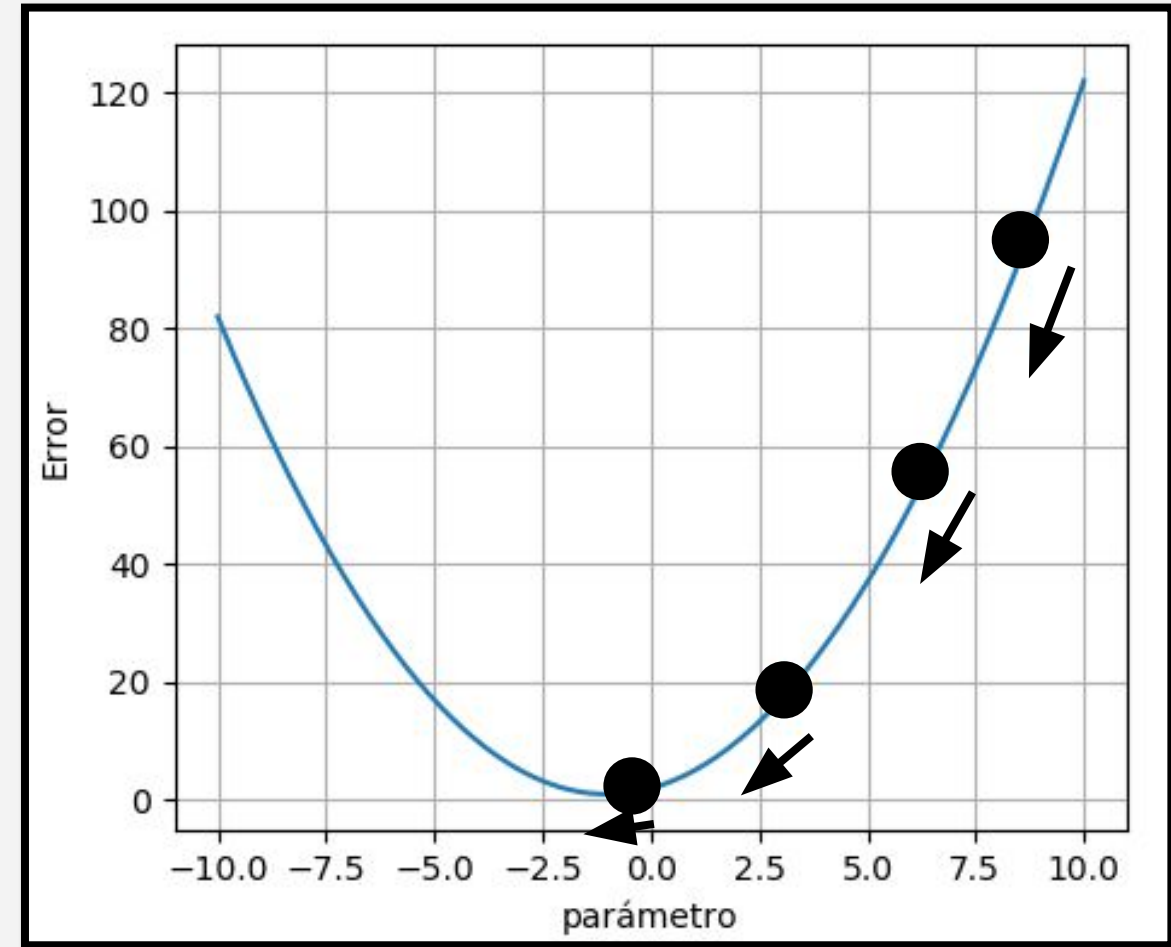
- ¿Qué significa **optimizar** una función **f**?
 - Buscar **algún** mínimo
 - Variando sus parámetros
- Algoritmos de Optimización
 - Analíticos vs Iterativos
 - Generales vs Especializados
 - Criterios de convergencia
 - Coste computacional
- Descenso de gradiente
 - Más utilizado para Redes Neuronales



Descenso de Gradiente

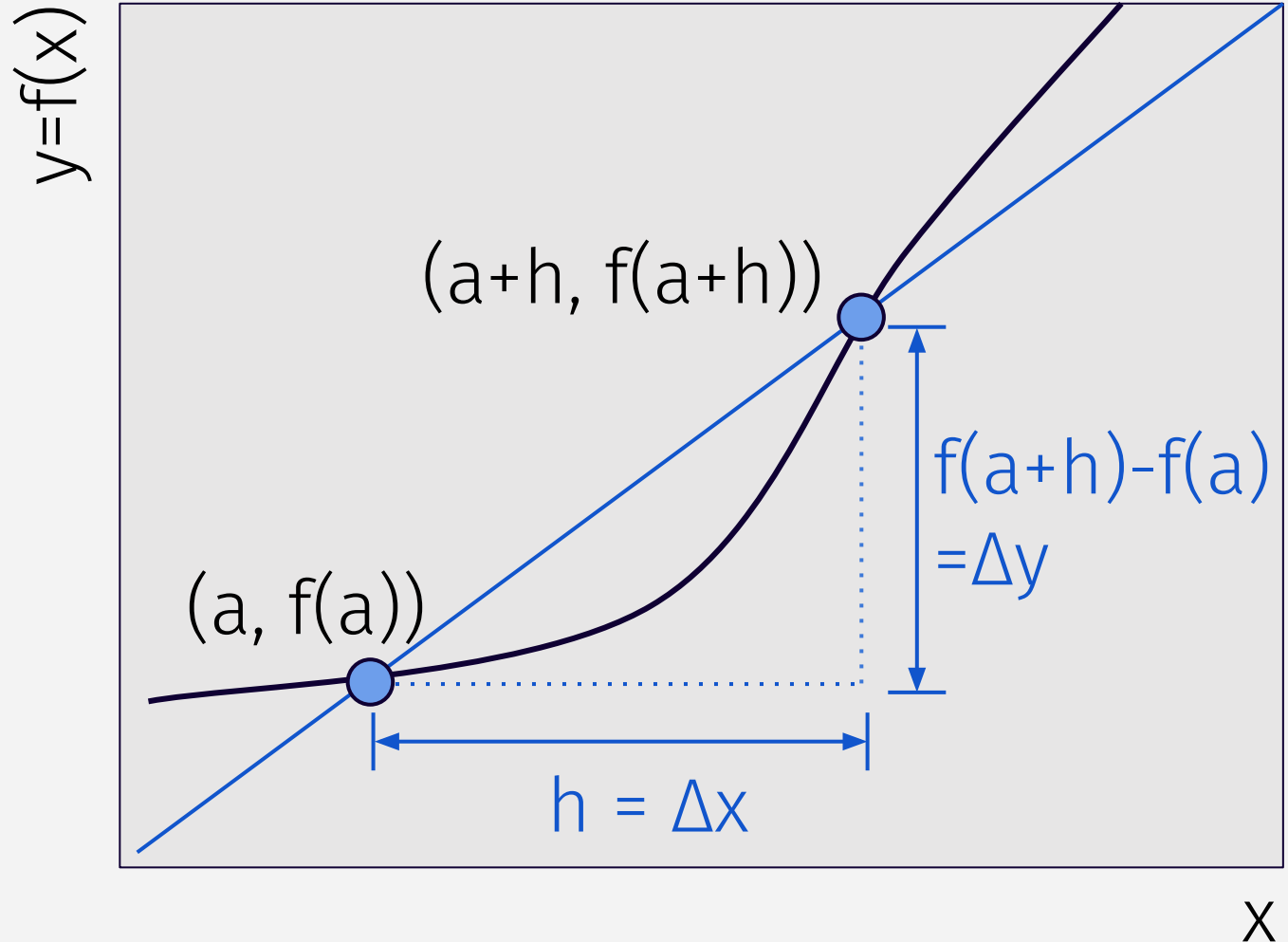
Descenso de gradiente

- Descenso de gradiente
 - Iterativo
 - Generalizable
 - Regresión Lineal
 - Regresión Logística
 - Redes Neuronales
 - Máquinas de Vectores de Soporte
 - Cualquier modelo con E derivable
 - Escalable (con modificaciones)
 - Millones de ejemplos



Derivadas

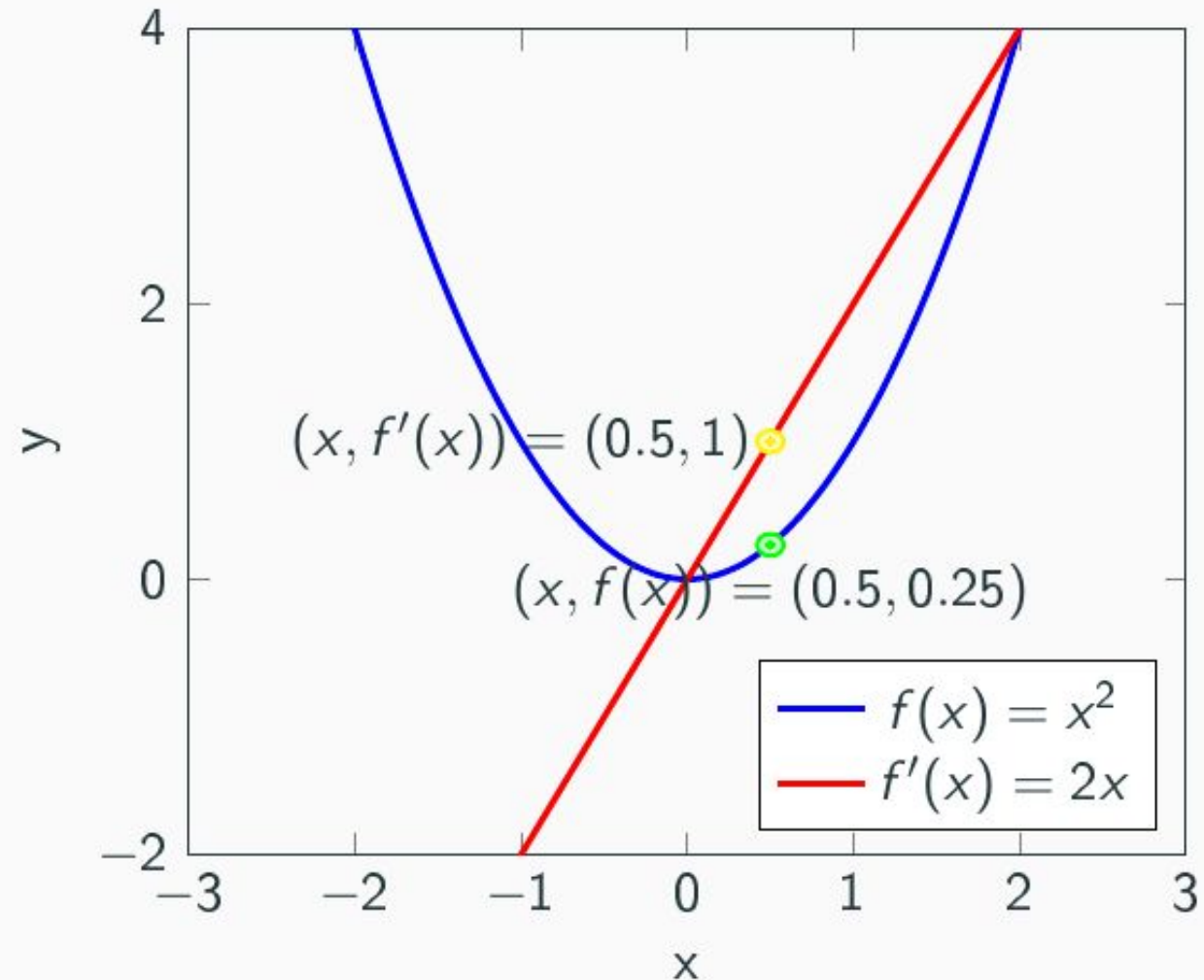
- Función $f(x) = \dots$
 - Alternativamente $y = f(x)$
 - Derivable $\rightarrow$ suave
- Derivada
 - Función calculada a partir de otra función
- Derivada 1D: $y' = f'(x)$
 - $f'(\mathbf{x})$ = pendiente a la recta tangente en el punto $\mathbf{x}$
 - Razón de cambio **dy/dx**
 - Signo indica dirección de crecimiento



Derivada 1D como indicador de crecimiento/decrecimiento

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} = \begin{cases} > 0 & \text{si } f \text{ crece} \\ < 0 & \text{si } f \text{ decrece} \\ = 0 & \text{pto crítico} \end{cases}$$

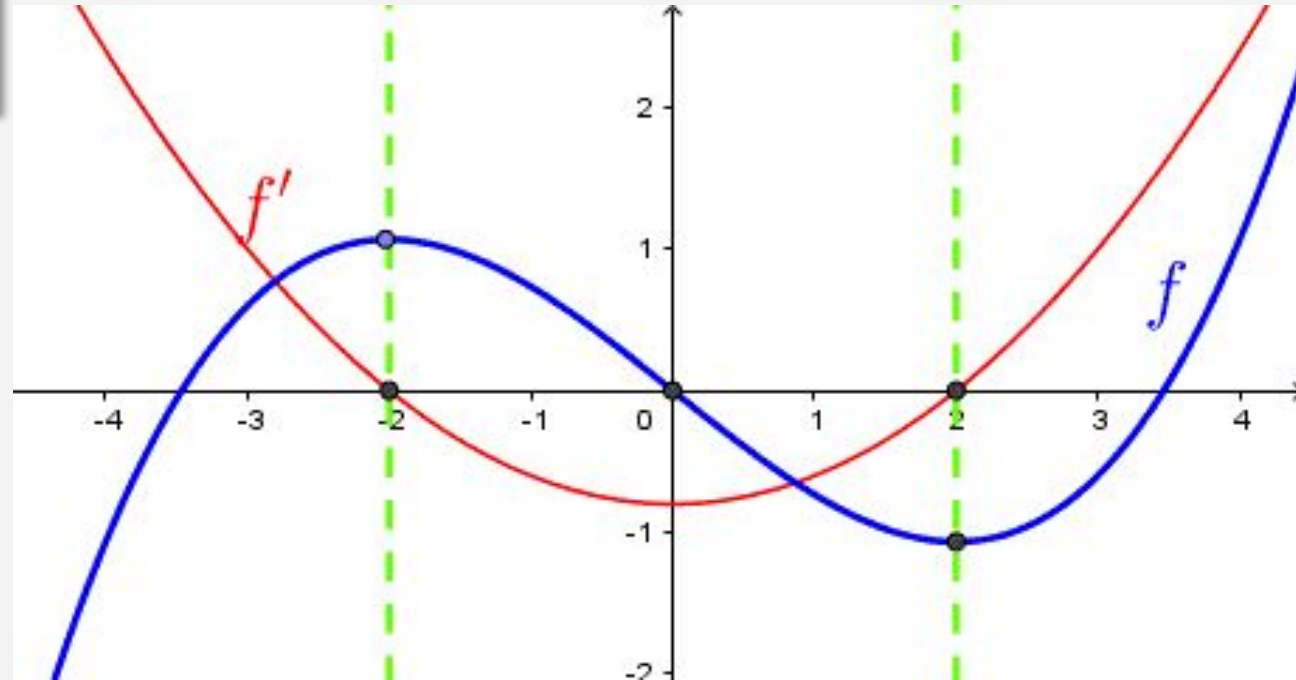
- Cociente incremental
 - Límite por izquierda y por derecha
 - Signo indica si crece o decrece
 - Solo alrededor del punto!



Derivada 1D como indicador de crecimiento/decrecimiento

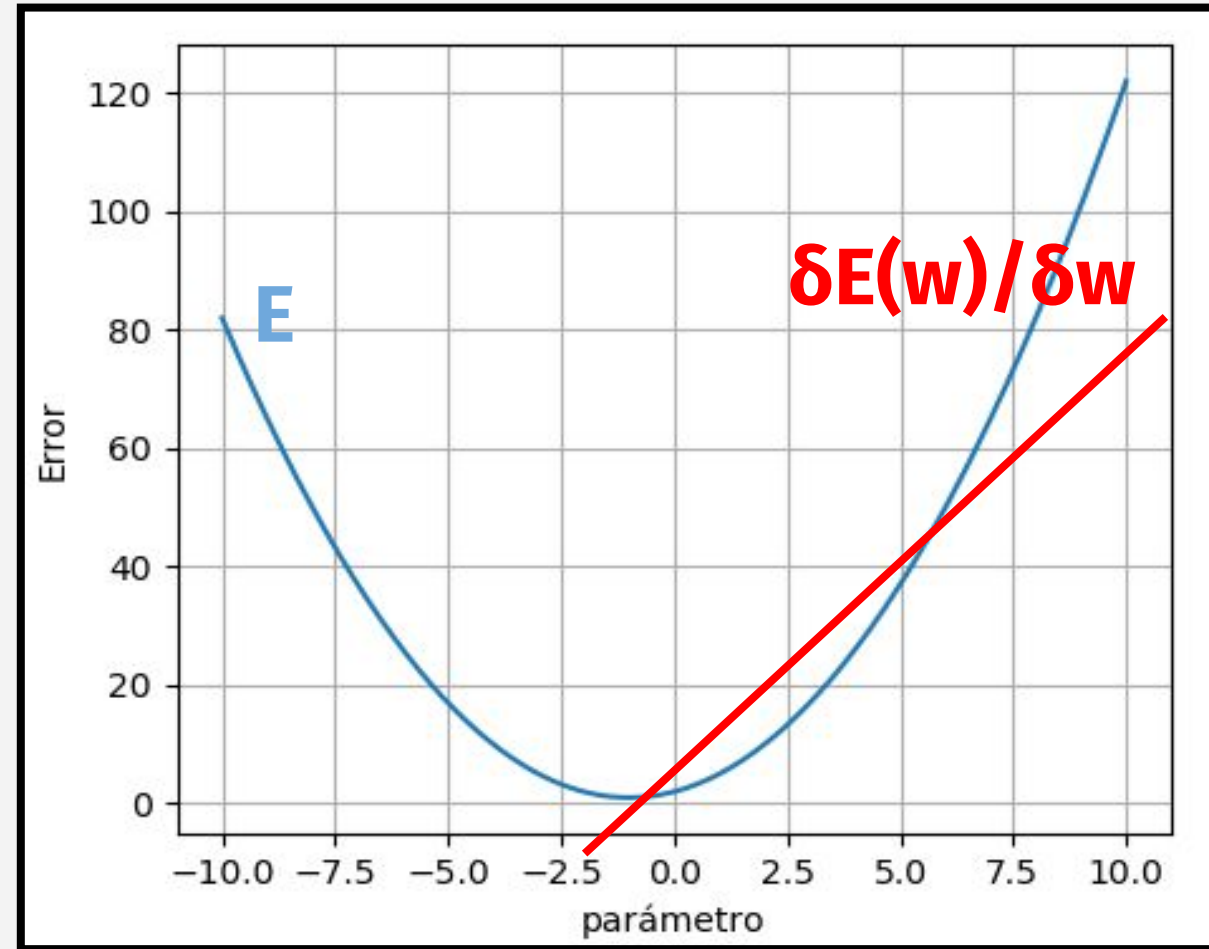
$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} = \begin{cases} > 0 & \text{si } f \text{ crece} \\ < 0 & \text{si } f \text{ decrece} \\ = 0 & \text{pto crítico} \end{cases}$$

- Cociente incremental
 - Límite por izquierda y por derecha
 - Signo indica si crece o decrece
 - Solo alrededor del punto!

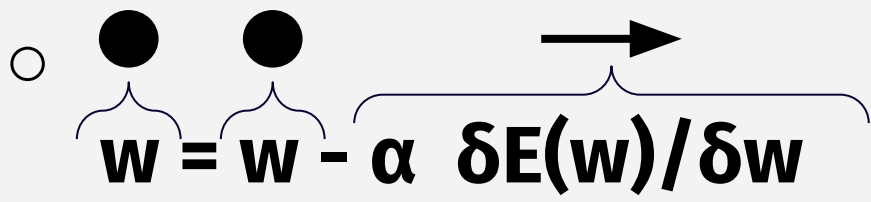


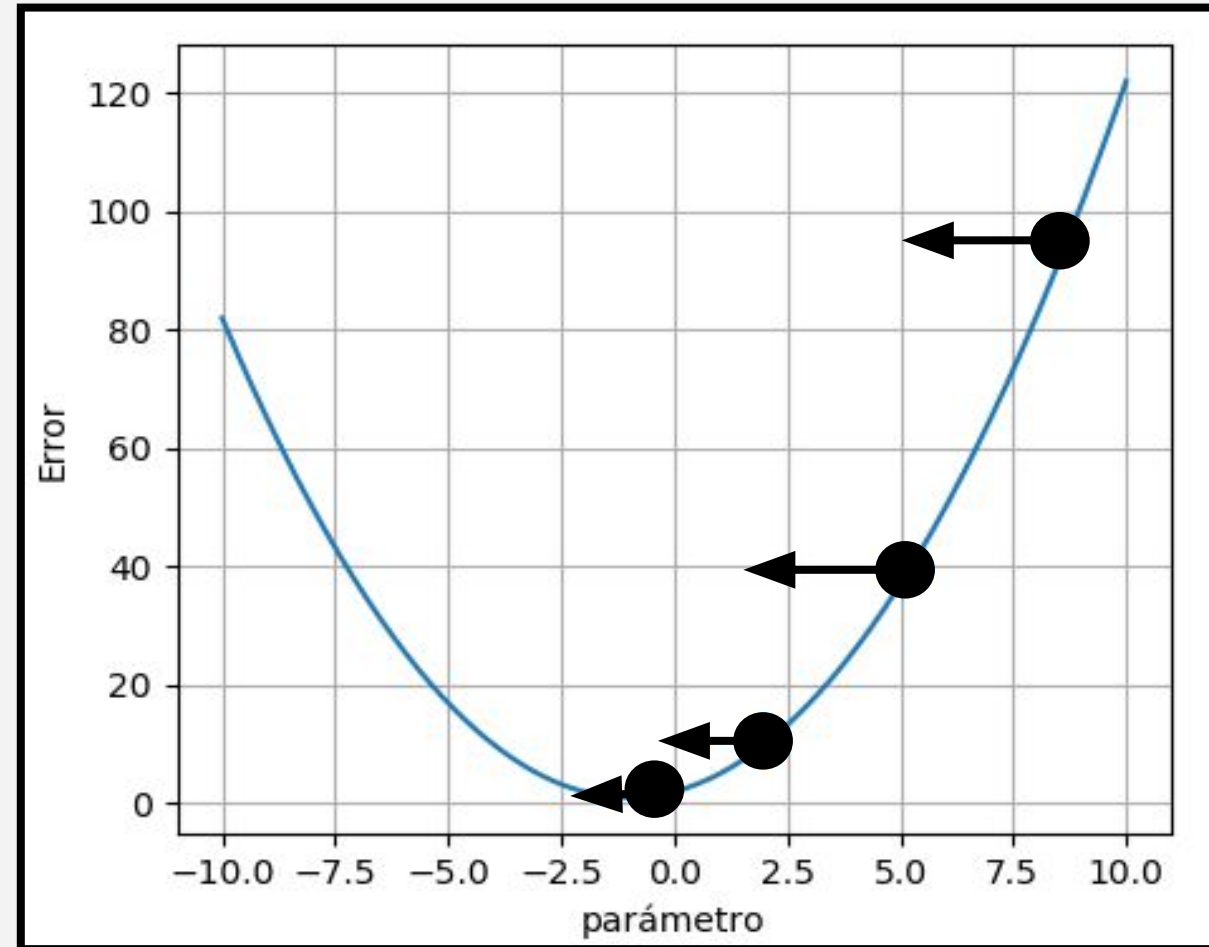
Descenso de gradiente en 1D

- Datos
 - Función E
 - De error u otra
 - Parámetro w
 - Valor inicial $w=w_0$ aleatorio
 - Derivada de E
 - Con respecto a $\mathbf{w}$
 - $\delta E(w)/\delta w$
 - Velocidad de aprendizaje α
 - Tamaño del **paso**
- Iterar
 - $\mathbf{w} = \mathbf{w} - \alpha \delta E(w)/\delta w$
 - Hasta converger



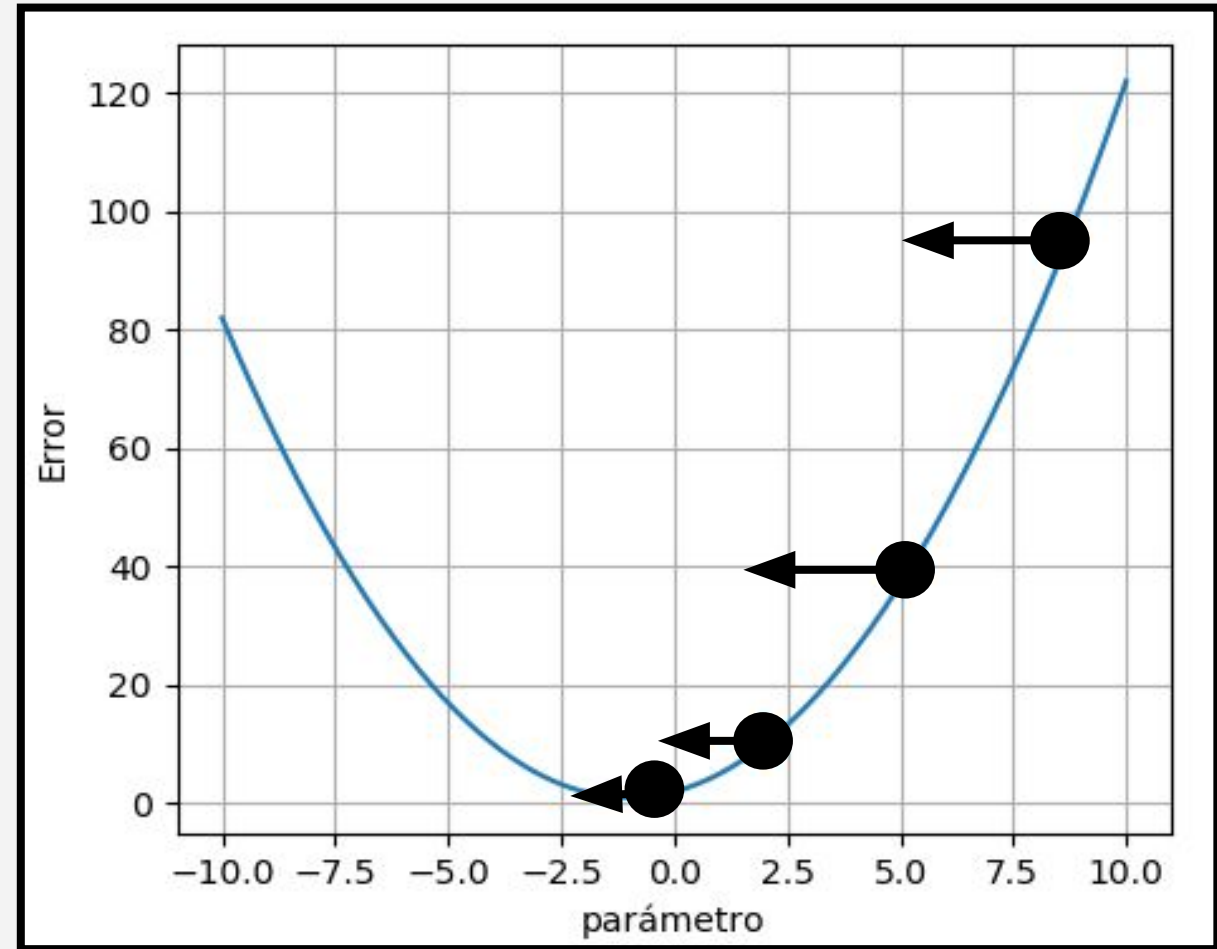
Descenso de gradiente en 1D

- Iterar
 - 
$$\mathbf{w} = \mathbf{w} - \alpha \frac{\delta E(\mathbf{w})}{\delta \mathbf{w}}$$
- Interpretamos
 - $w = w - \langle \text{algo} \rangle$
 - Modificamos w
 - $\delta E(w) / \delta w \rightarrow$
 - Dirección y magnitud del cambio
 - α —
 - Magnitud del cambio
 - $\alpha \delta E(w) / \delta w \rightarrow$
 - Dirección y magnitud del cambio

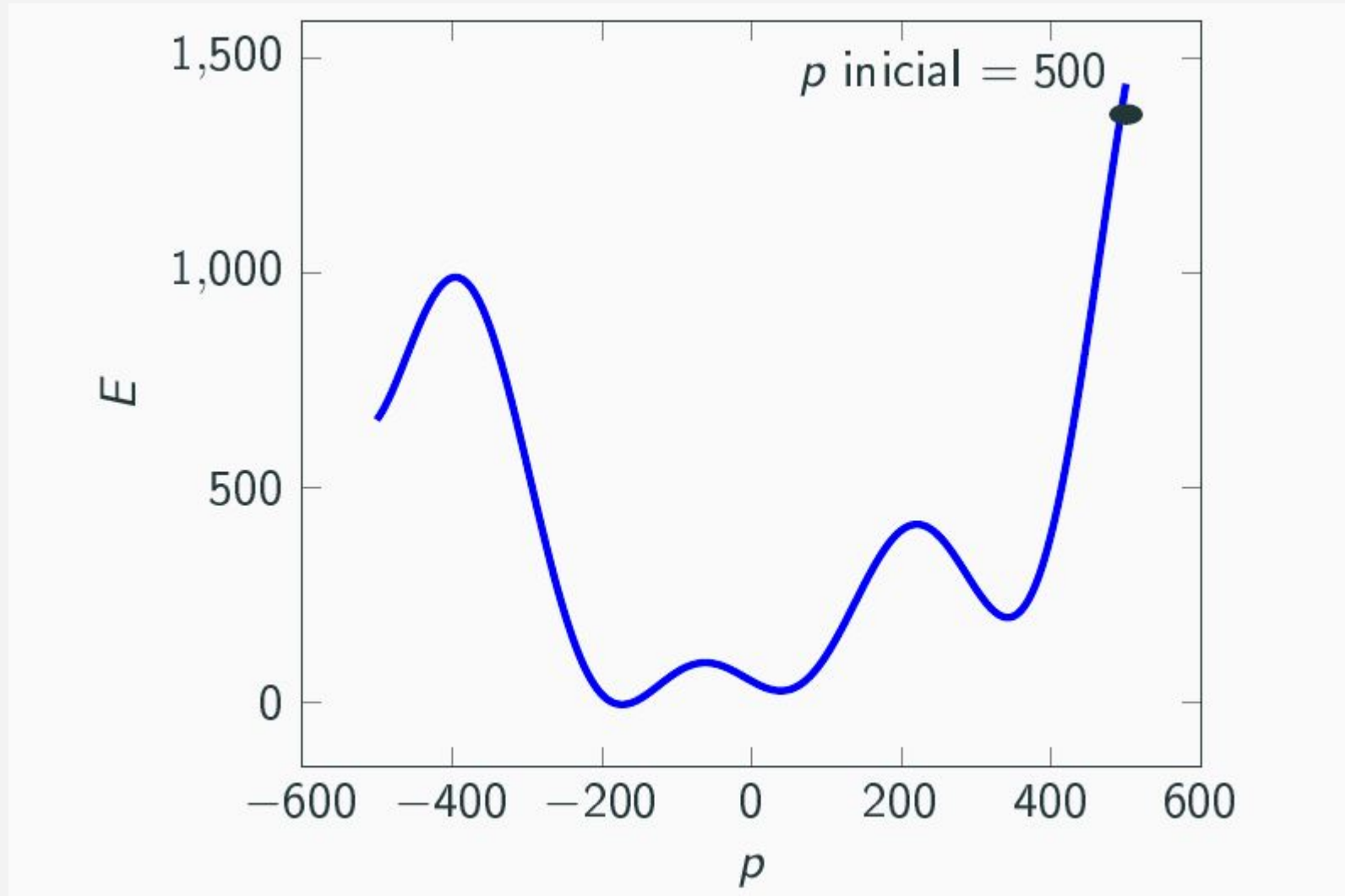


Descenso de gradiente en 1D

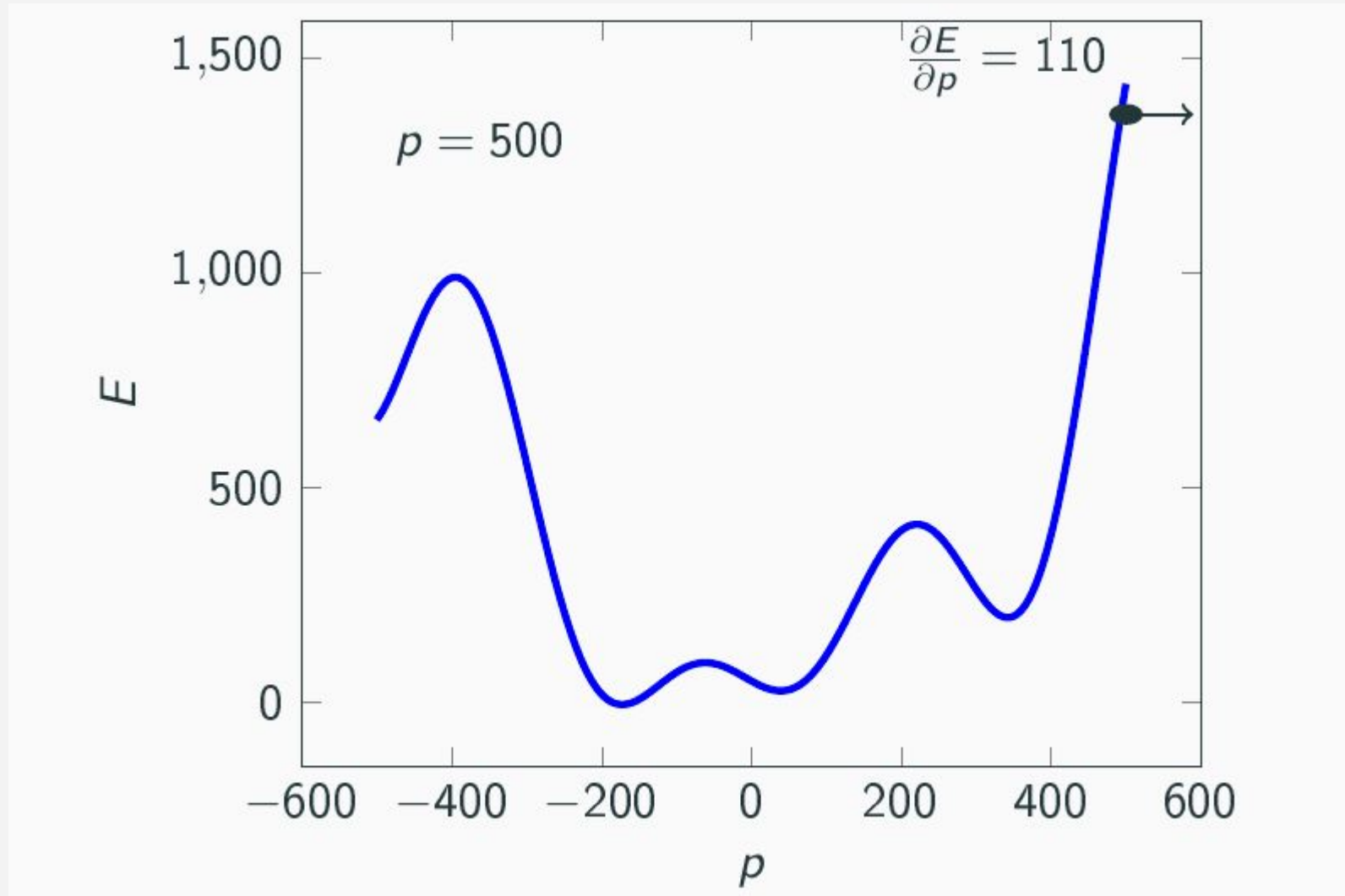
```
def descenso_gradiente(E,w,x,y):  
    w = ...  
    converge=False  
    while not converge:  
         $\delta E \delta w$  = derivada(E,w,x,y)  
        w = w -  $\delta E \delta w$   
        converge = ... (depende)  
    return w
```



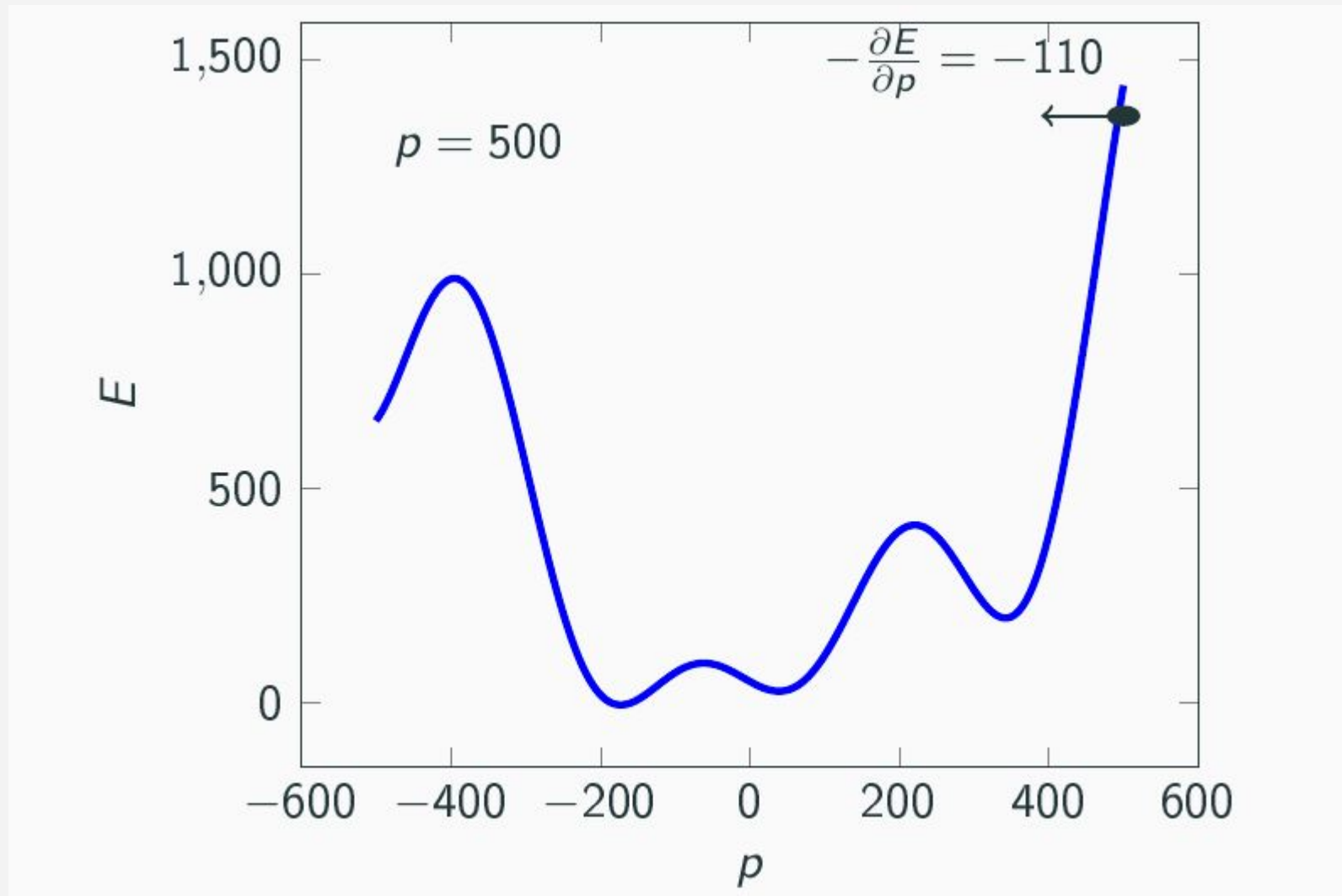
Descenso de gradiente en 1D



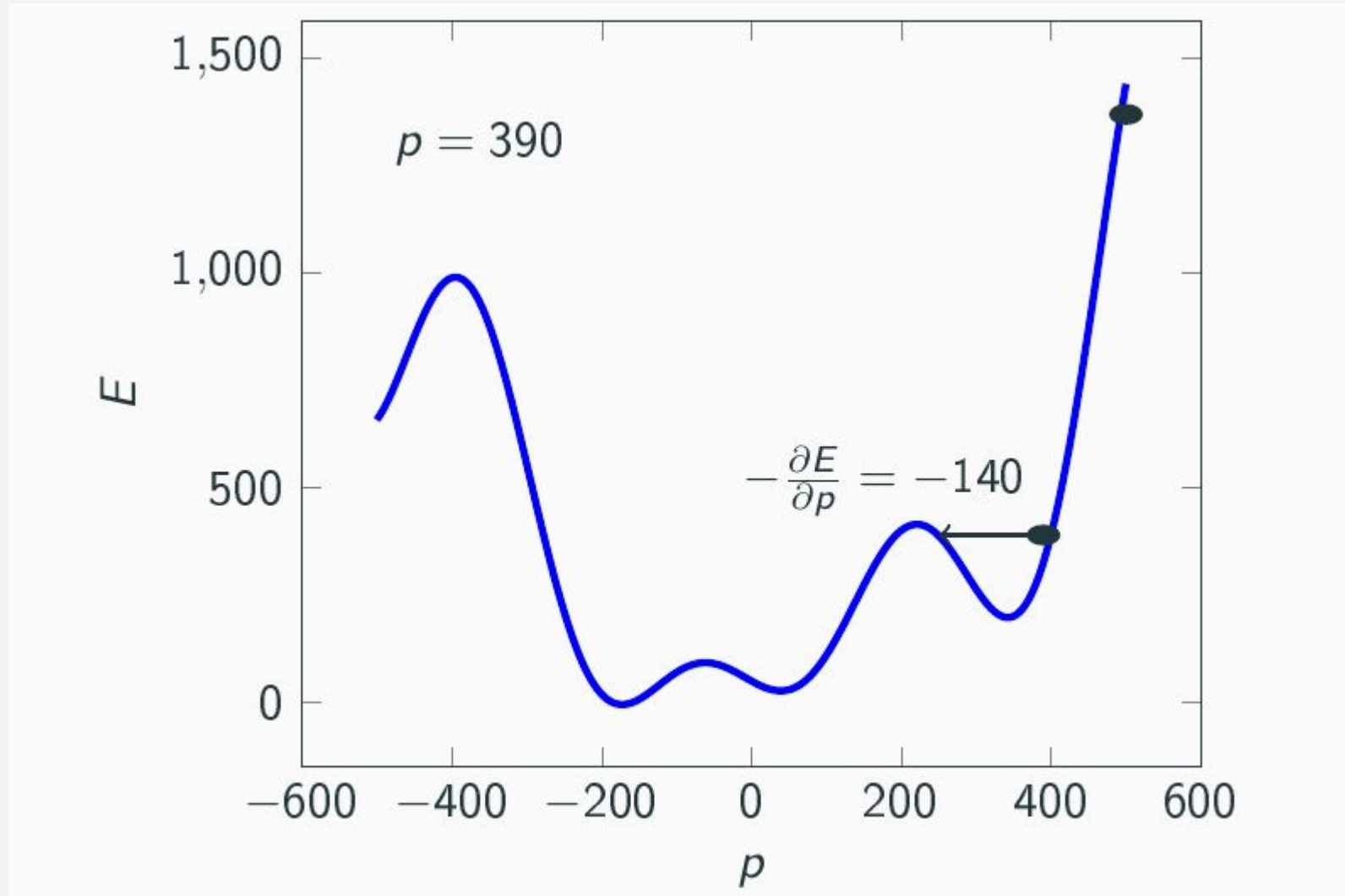
Descenso de gradiente en 1D



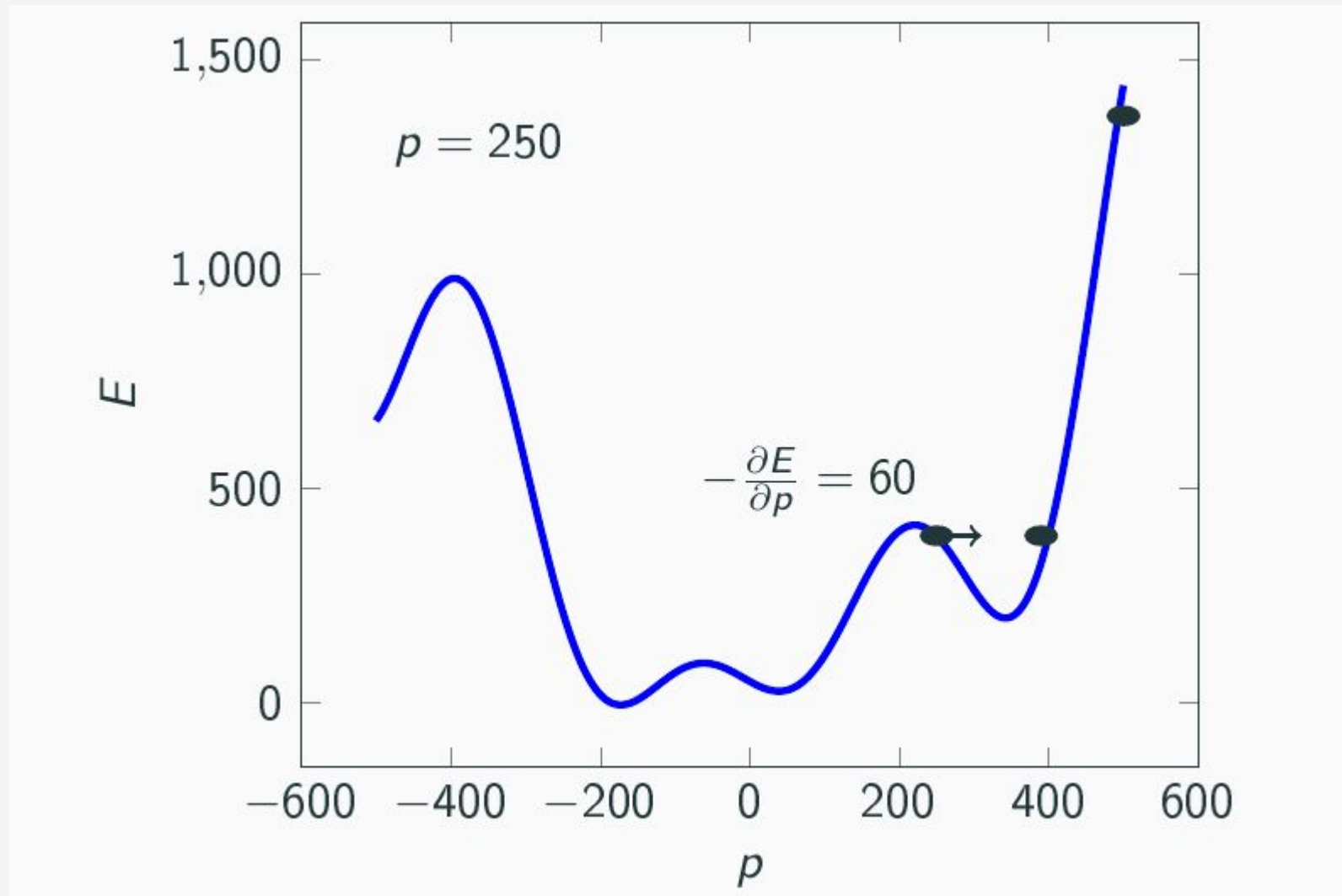
Descenso de gradiente en 1D



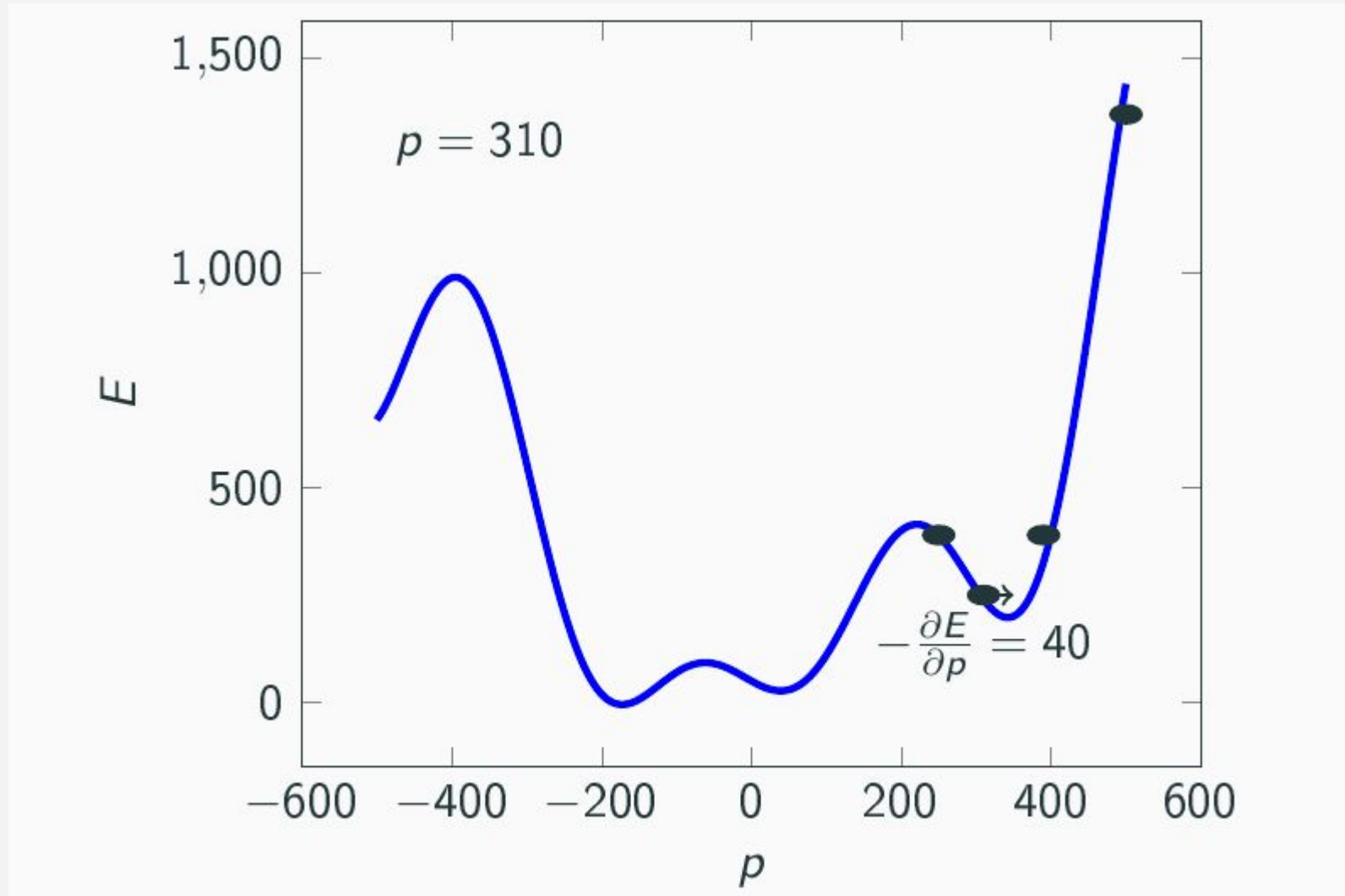
Descenso de gradiente en 1D



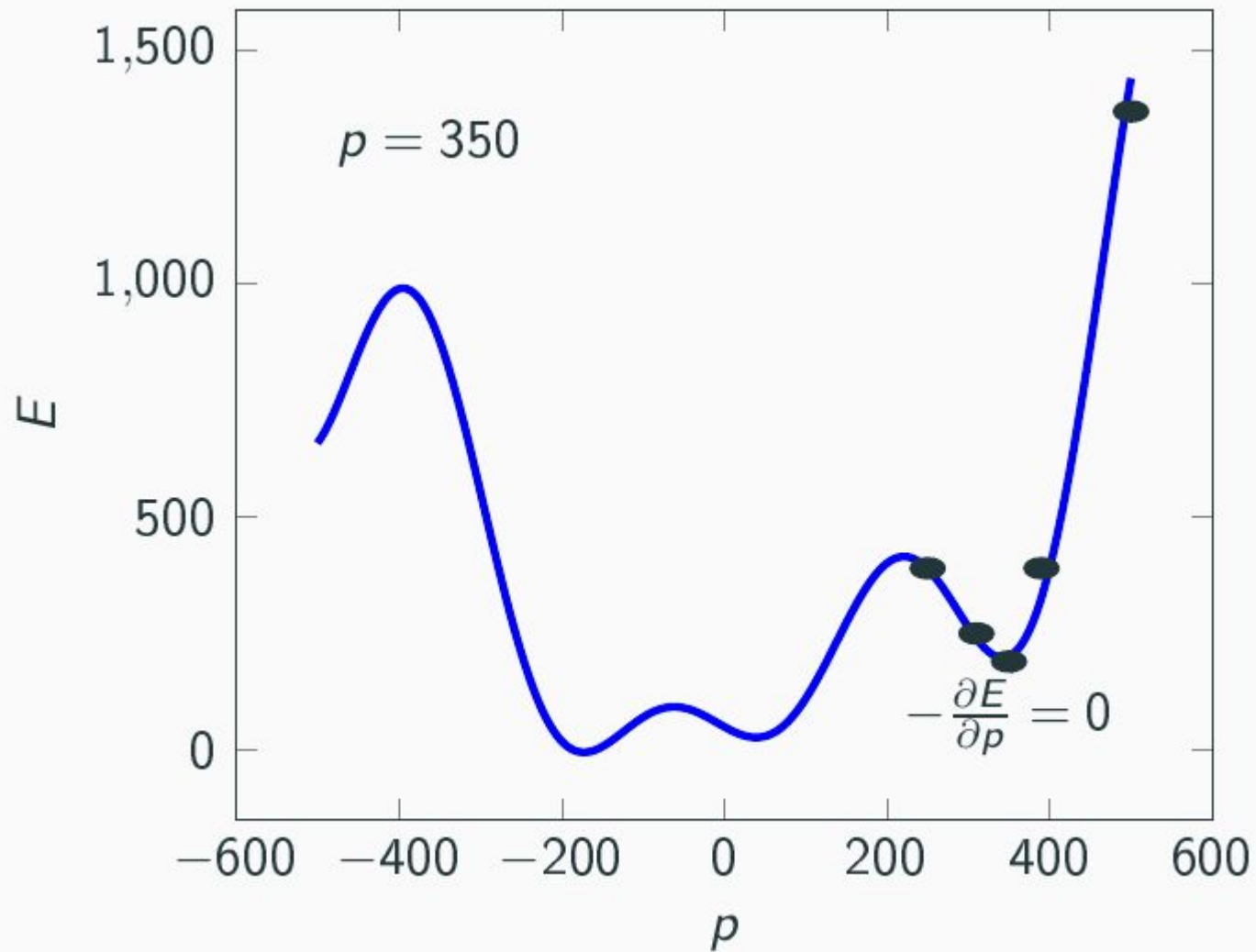
Descenso de gradiente en 1D



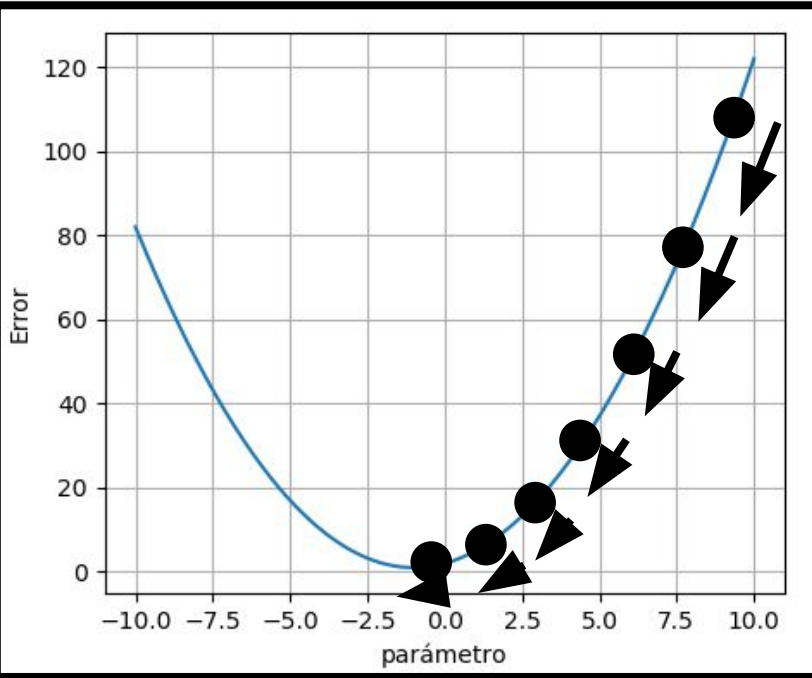
Descenso de gradiente en 1D



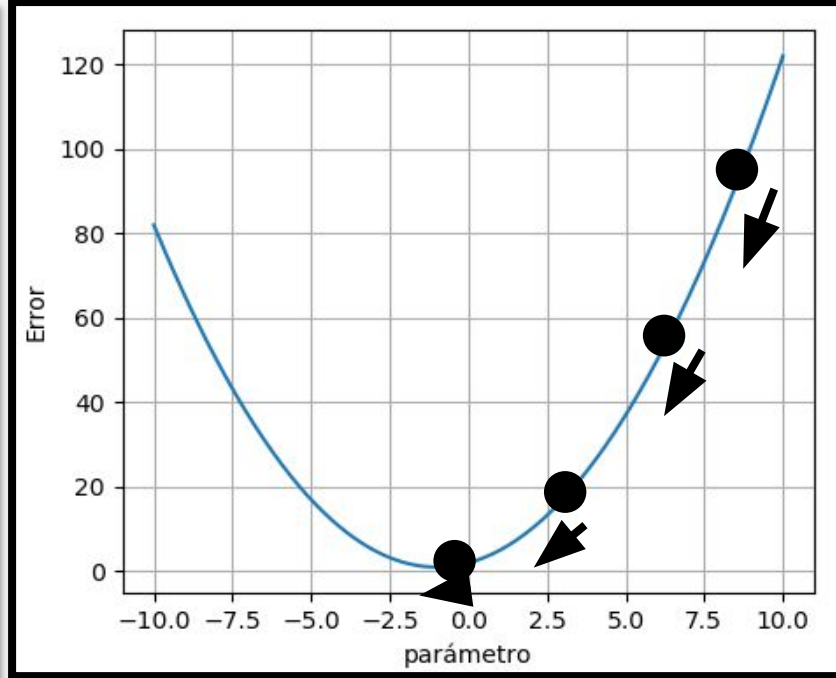
Descenso de gradiente en 1D



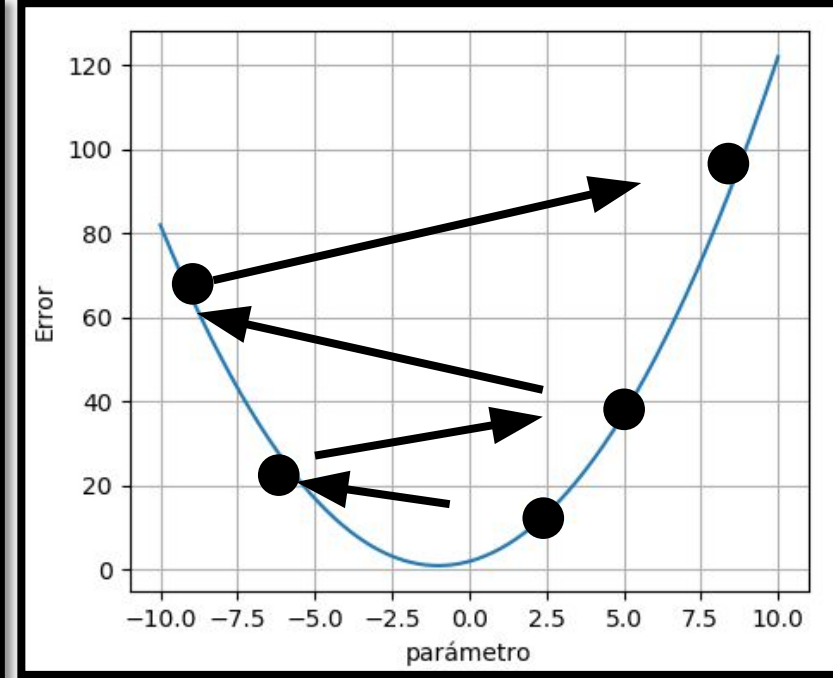
Efecto de α en $w := w - \alpha (\delta E(w)/\delta w)$



- α demasiado chico
 - Poco avance por iteración
 - Alto coste computacional



- α “correcto”
 - Buen avance por iteración
 - Razonable coste computacional

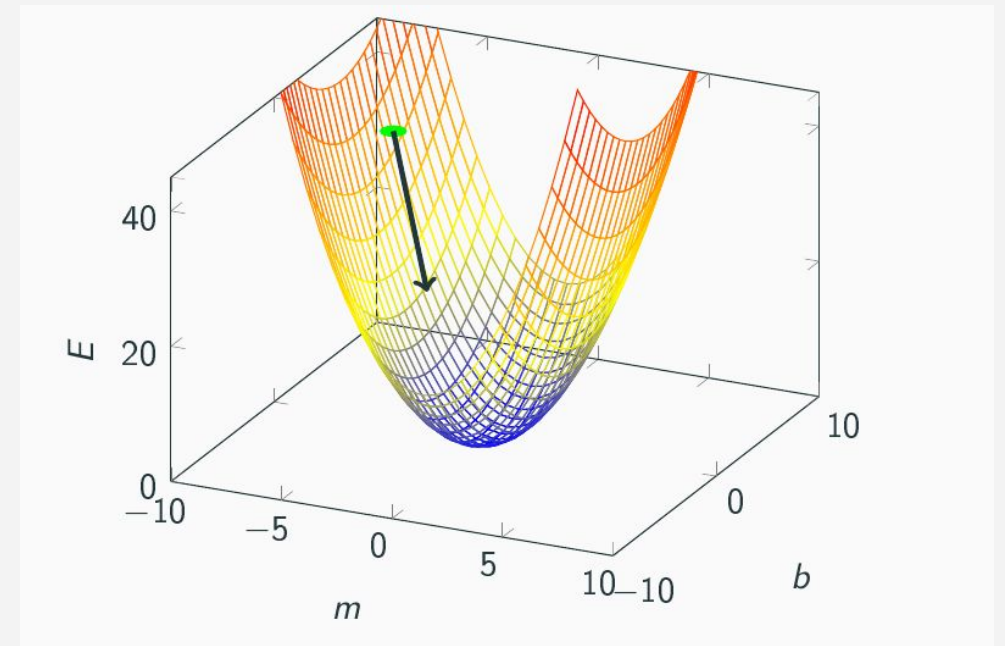


- α demasiado **grande**
 - “saltos” grandes
 - Puede no converger
 - Errores numéricos

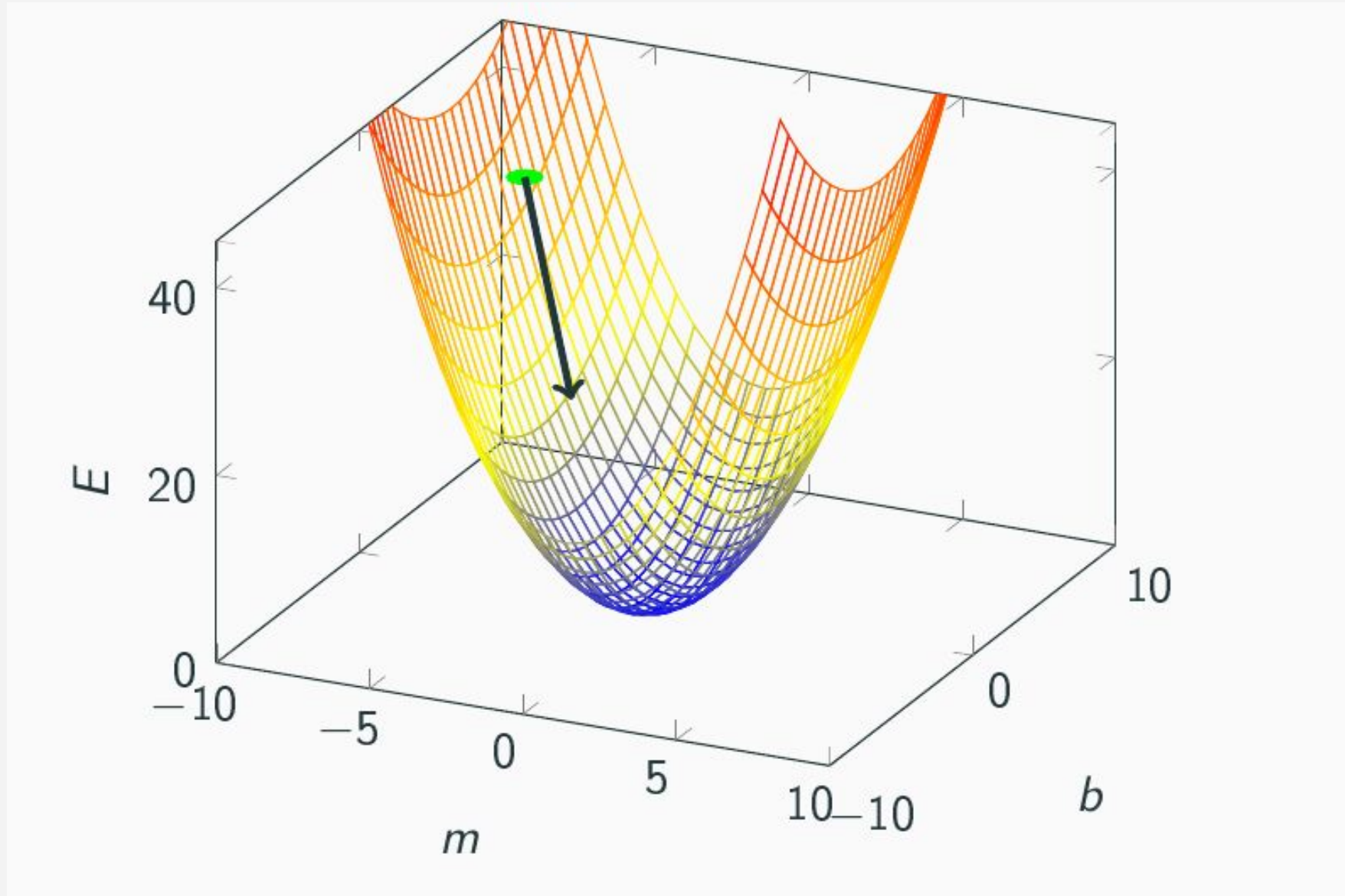
¿Por qué descenso de **gradiente**?

- Si tenemos 2 parámetros
 - w_1 y w_2
 - 2 derivadas parciales
 - $\delta E / \delta w_1$
 - $\delta E / \delta w_2$
 - **Gradiente ΔE**
 - Vector de derivadas parciales
 - $\Delta E = (\delta E / \delta w_1, \delta E / \delta w_2)$
- Con P parámetros
 - $w_1, w_2, \dots, w_P$
 - $\Delta E = (\delta E / \delta w_1, \delta E / \delta w_2, \dots, \delta E / \delta w_P)$

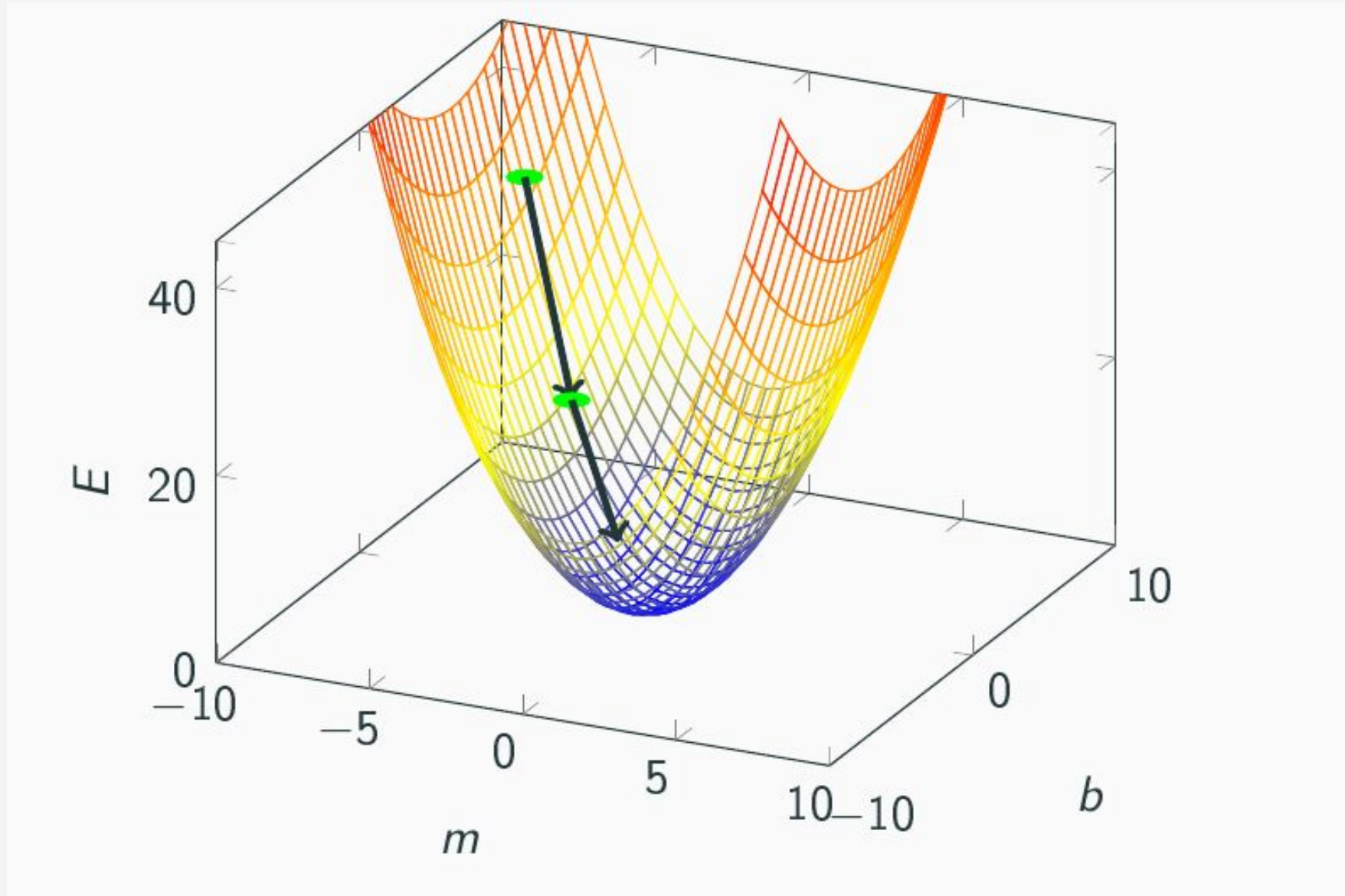
- 1 sólo parámetro
 - w
 - **$\Delta E = (\delta E / \delta w)$**
 - Gradiente = derivada



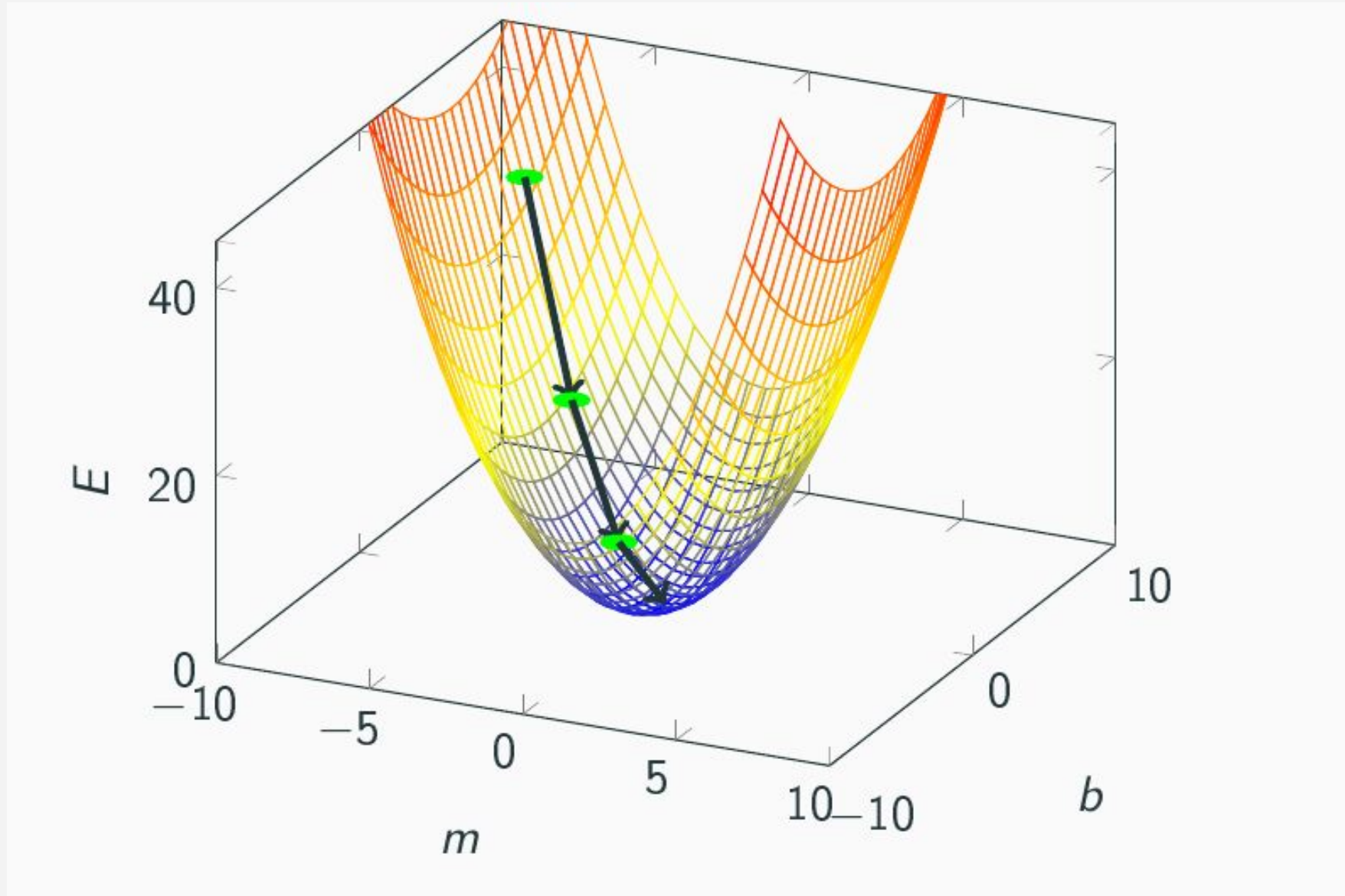
Descenso de gradiente en 2D



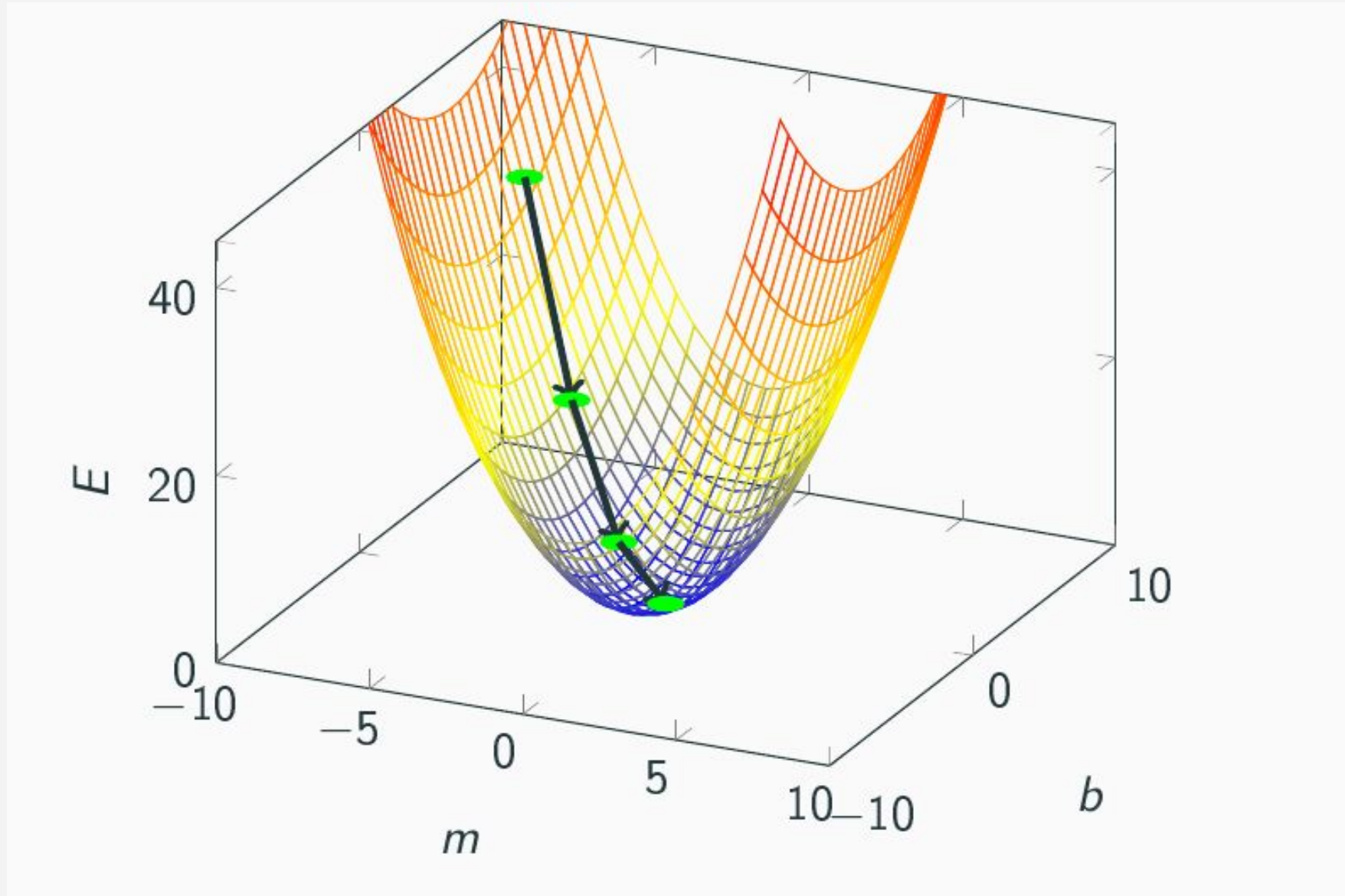
Descenso de gradiente en 2D



Descenso de gradiente en 2D

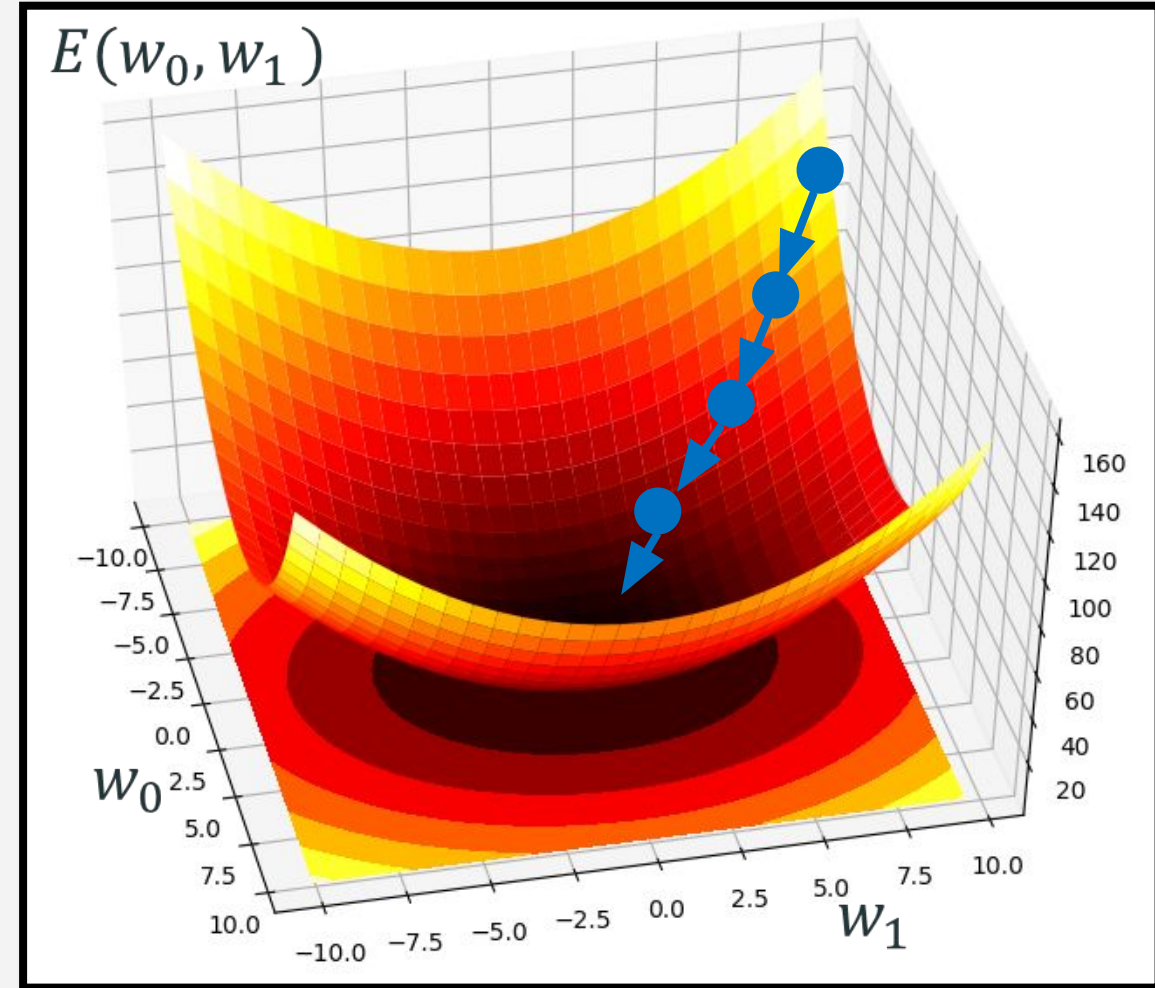


Descenso de gradiente en 2D



Descenso de gradiente en 2D

- 2D vs 1D
 - Mismo algoritmo
 - 2 parámetros
 - $E(w_1, w_2) = ..$
 - $\Delta \mathbf{E} = (\delta E / \delta w_1, \delta E / \delta w_2)$
- Ecuaciones
 - $w_1 = w_1 - \alpha \delta E(w_1, w_2) / \delta w_1$
 - $w_2 = w_2 - \alpha \delta E(w_1, w_2) / \delta w_2$
 - En general
 - $w_i = w_i - \alpha \delta E(w_1, w_2) / \delta w_i$



Descenso de gradiente en ND

Iteración 1

x_1	x_1	x_1	x_1	y
2	2	2	2	1
5	5	5	5	3.2
7	7	7	7	4.5
9	9	9	9	6
10	10	10	10	4
11	11	11	11	4.5
13.4	13.4	13.4	13.4	5.5
14	14	14	14	3
15	15	15	15	5

$$\begin{aligned} \delta E / \delta w_1 \\ \delta E / \delta w_2 \\ \dots \\ \delta E / \delta w_p \end{aligned}$$

$$\begin{aligned} w_1 &= w_1 - \delta E / \delta w_1 \\ w_2 &= w_2 - \delta E / \delta w_2 \\ \dots \\ w_p &= w_p - \delta E / \delta w_p \end{aligned}$$

Iteración 2

x_1	x_1	x_1	x_1	y
2	2	2	2	1
5	5	5	5	3.2
7	7	7	7	4.5
9	9	9	9	6
10	10	10	10	4
11	11	11	11	4.5
13.4	13.4	13.4	13.4	5.5
14	14	14	14	3
15	15	15	15	5

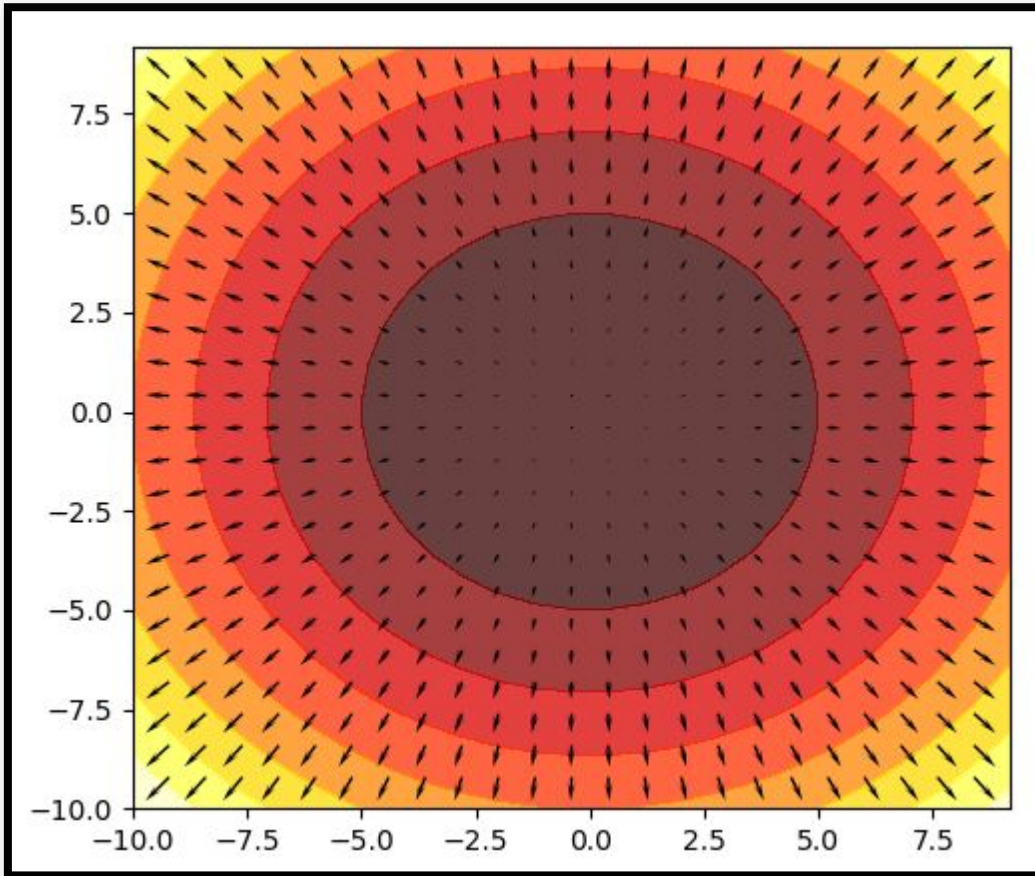
$$\begin{aligned} \delta E / \delta w_1 \\ \delta E / \delta w_2 \\ \dots \\ \delta E / \delta w_p \end{aligned}$$

$$\begin{aligned} w_1 &= w_1 - \delta E / \delta w_1 \\ w_2 &= w_2 - \delta E / \delta w_2 \\ \dots \\ w_p &= w_p - \delta E / \delta w_p \end{aligned}$$

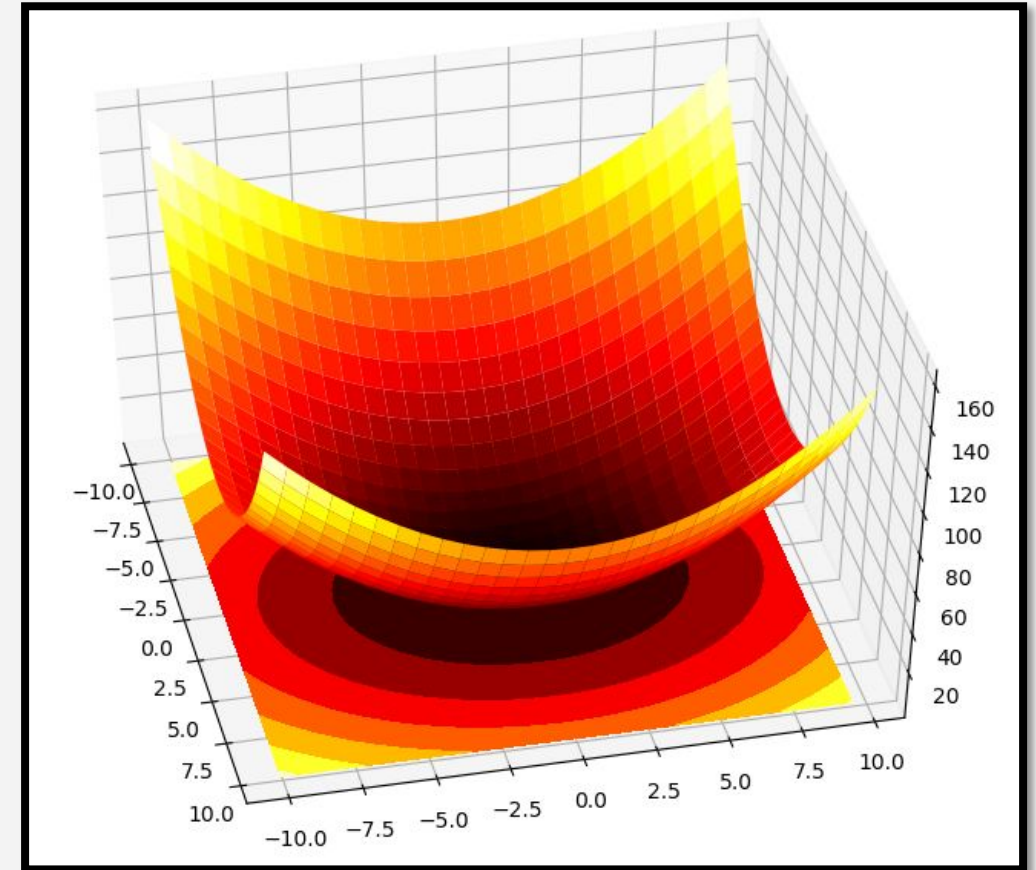
Diagramas de contorno y gradientes

```
W1 = np.arange(-10, 10, 0.8)
W2 = np.arange(-10, 10, 0.8)
W1, W2 = np.meshgrid(W1, W2)
E = (W1**2 + W2**2)
```

- Flechas: Vectores gradiente



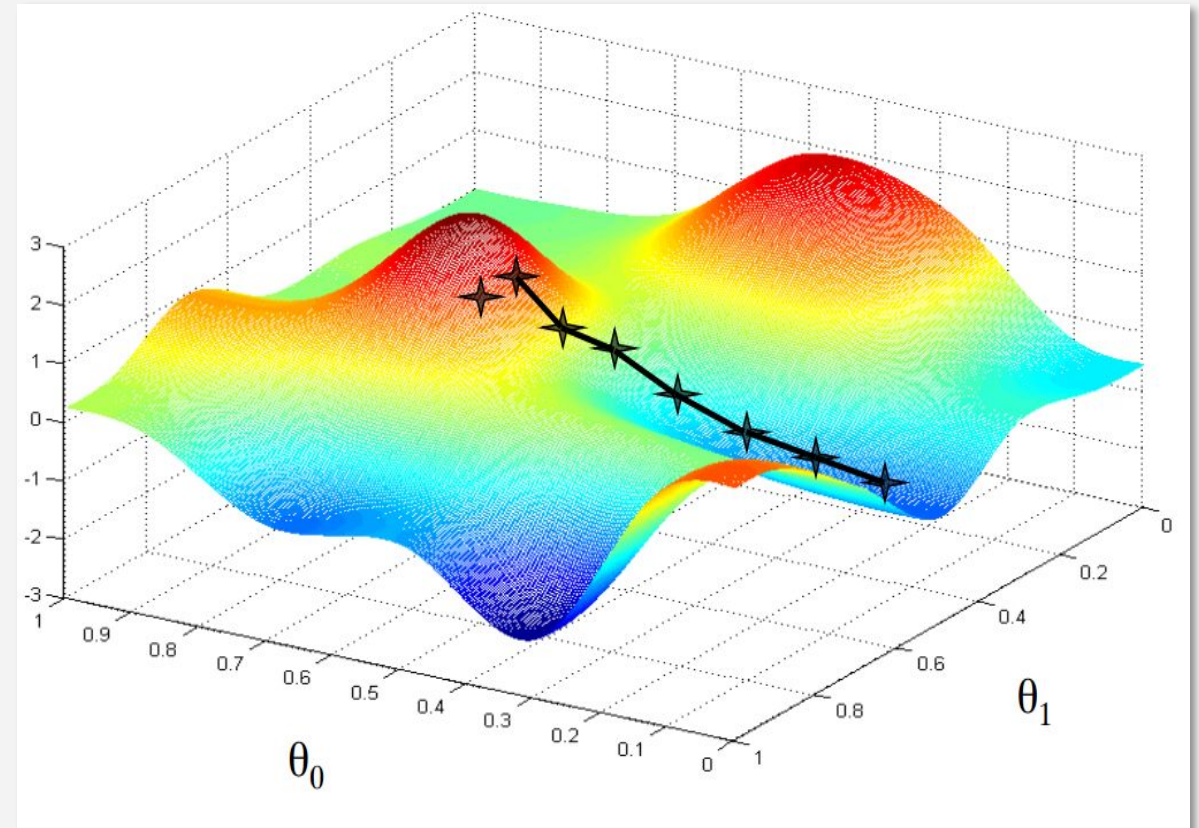
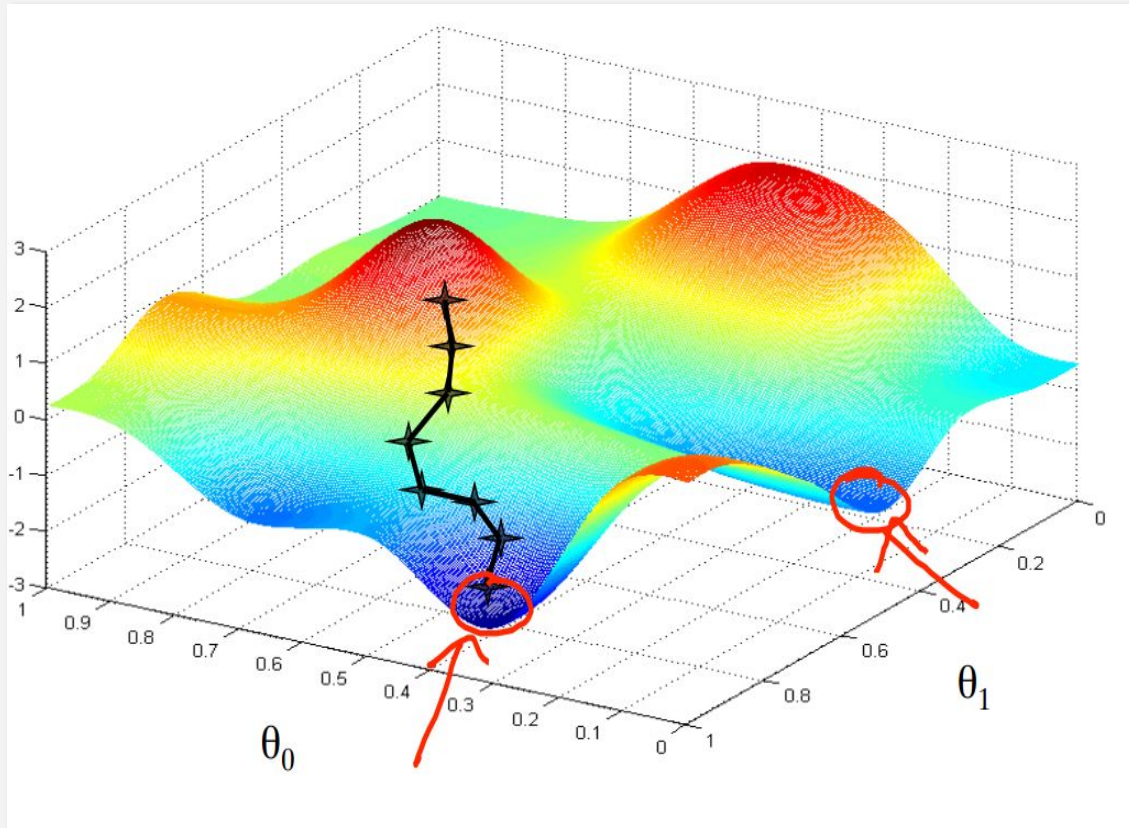
```
plt.contour(W1, W2, E)
dEdW1, dEdW2 = np.gradient(E)
plt.quiver(W1, W2, dEdW1, dEdW2)
```



```
ax = Axes3D(fig)
ax.plot_surface(W1, W2, E)
ax.contourf(W1, W2, E)
```

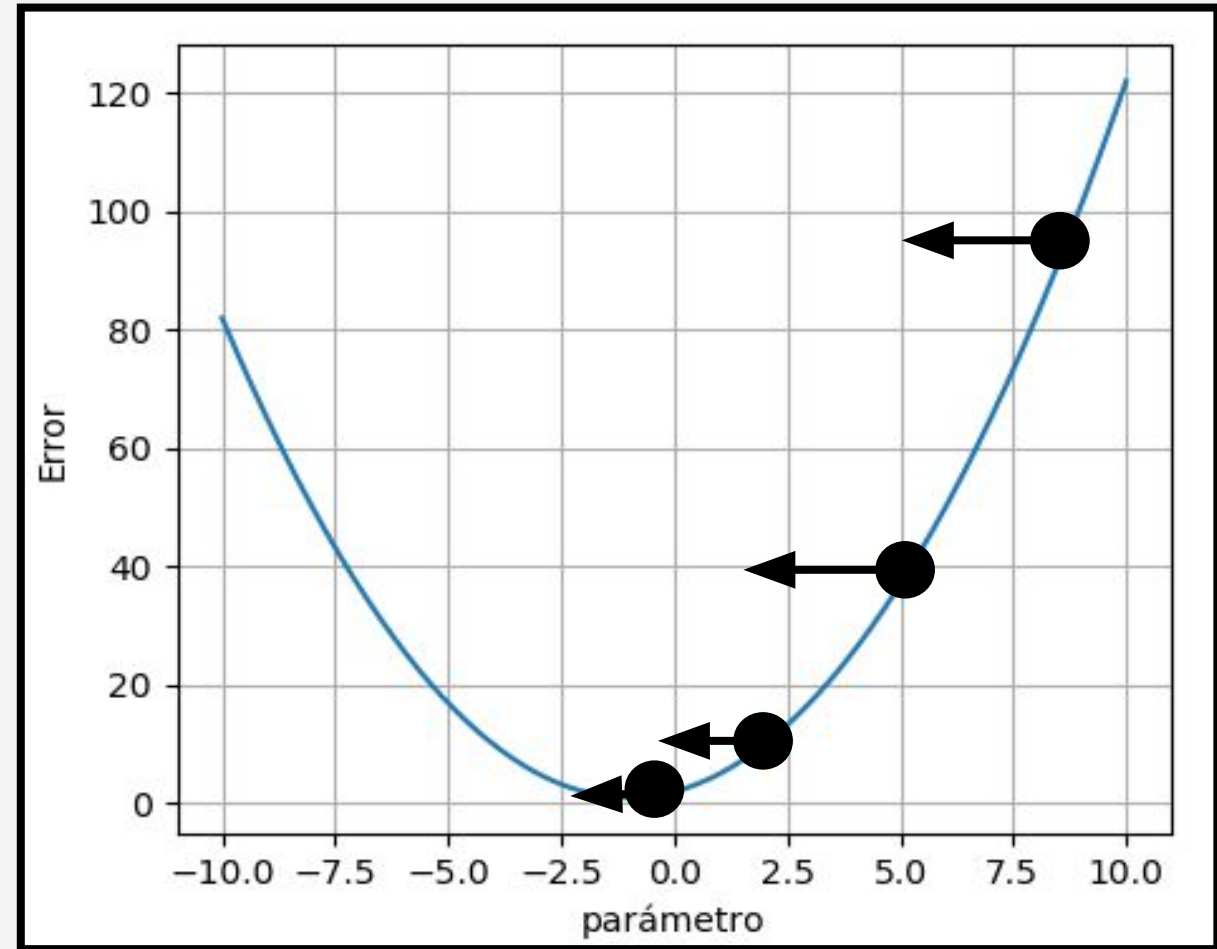
Funciones no convexas

- Redes Neuronales
 - Varían parámetros iniciales
 - Varían parámetros finales
 - Función de error no convexa → mínimos locales



Resumen

- Descenso de gradiente
 - Iterativo
 - Generalizable
 - Requiere f derivable
 - Escalable
 - Millones de ejemplos
 - Con modificaciones
 - Ecuación **fundamental**
 - $\mathbf{w} := \mathbf{w} - \alpha (\delta E(\mathbf{w}_i) / \delta \mathbf{w}_i)$
 - Mueve a $\mathbf{w}_i$ en la dirección que minimiza E
 - α indica el tamaño del paso

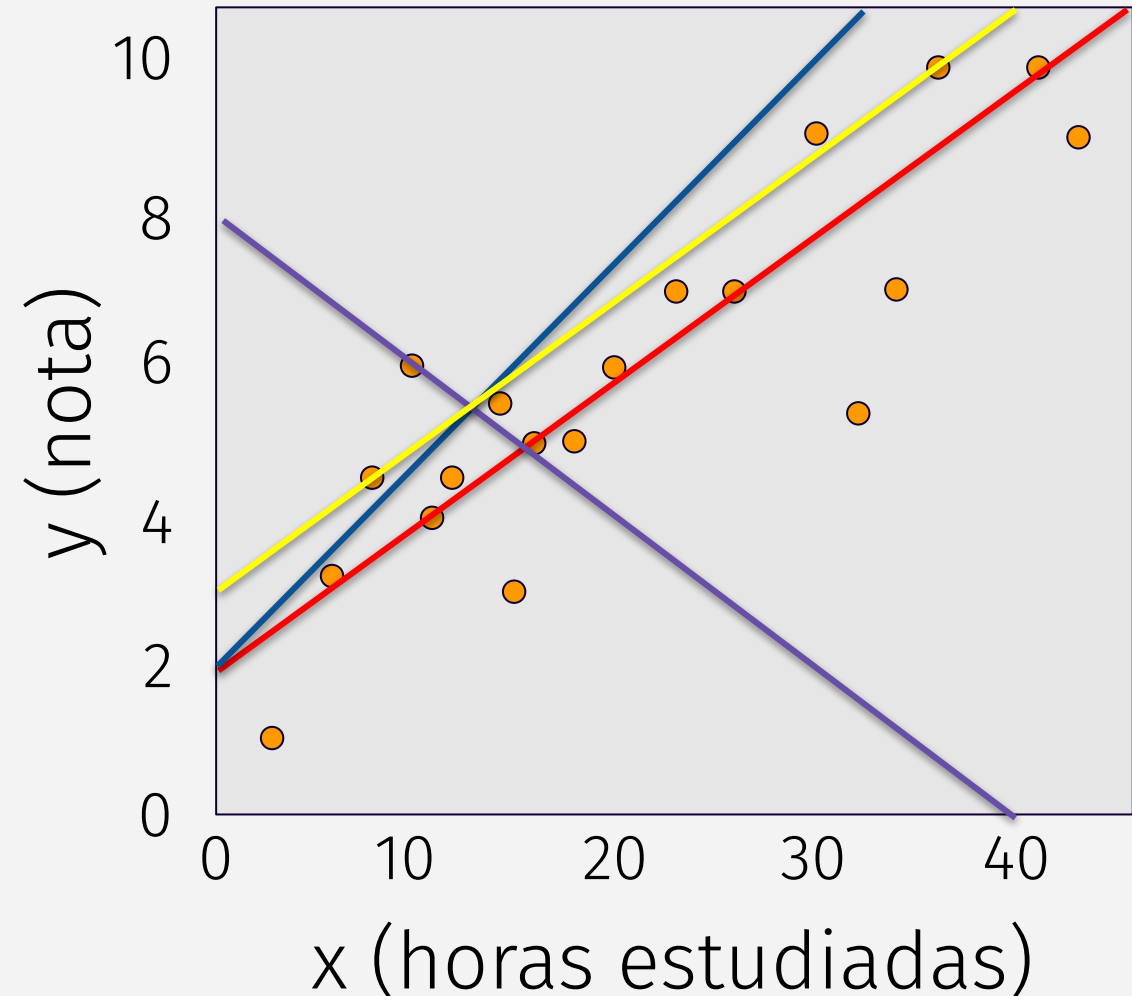


Regresión Lineal

Descenso de Gradiente

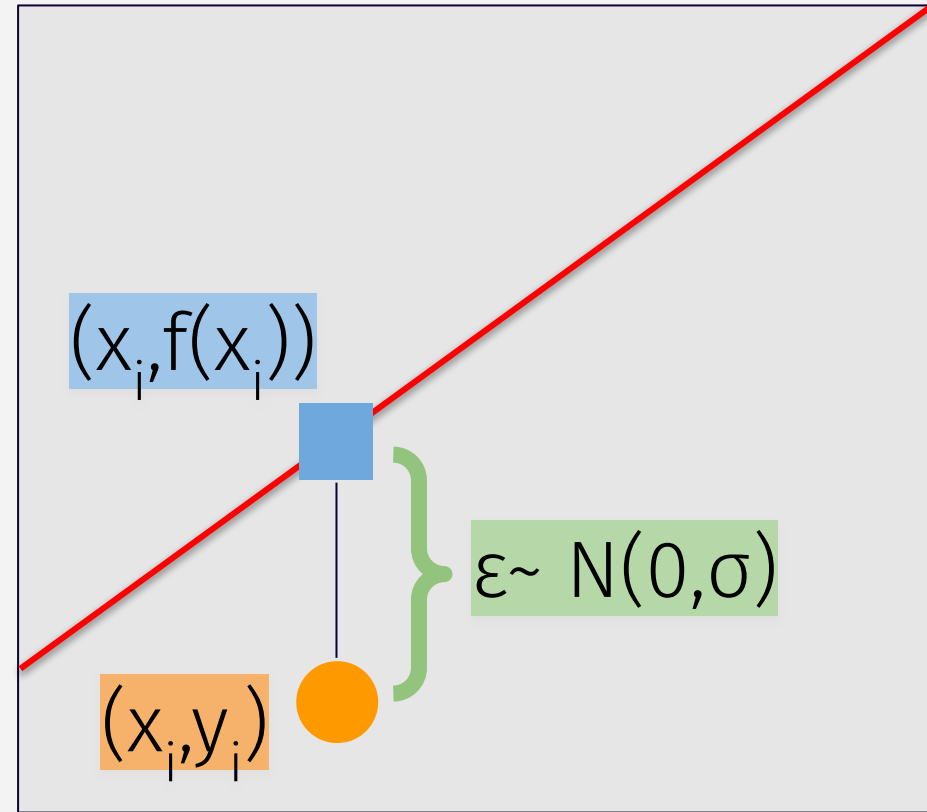
Optimización del Error **E**

- Problema:
 - Datos: ejemplos $(\mathbf{x}_i, \mathbf{y}_i)$
 - Parámetros: **m** y **b**
 - Función de error: **E**
 - Encontrar **m** y **b** que minimicen $\mathbf{E}(\mathbf{m}, \mathbf{b}, \mathbf{x}, \mathbf{y})$
 - **E** es derivable
 - Respecto de **m** y **b**



Regresión Lineal con MLE

- MLE: Maximum Likelihood Estimation
 - (Método de Máxima Verosimilitud)
 - Método estadístico
- Asumo $y = f(x) + \varepsilon = m x + b + \varepsilon$
 - $\varepsilon \sim N(0, \sigma)$
 - m, b y σ son **parámetros**
 - Busco estimadores m', b', σ'
 - $\varepsilon = f(x) - y$
 - $(f(x) - y) \sim N(0, \sigma)$
 - Función de verosimilitud L
 - $L(m, b, \sigma) = \prod_i^n \text{Gaussiana}[\mu=0, \sigma,](f(x_i) - y_i)$
 - MLE $\rightarrow$ fórmula para m', b' y σ'

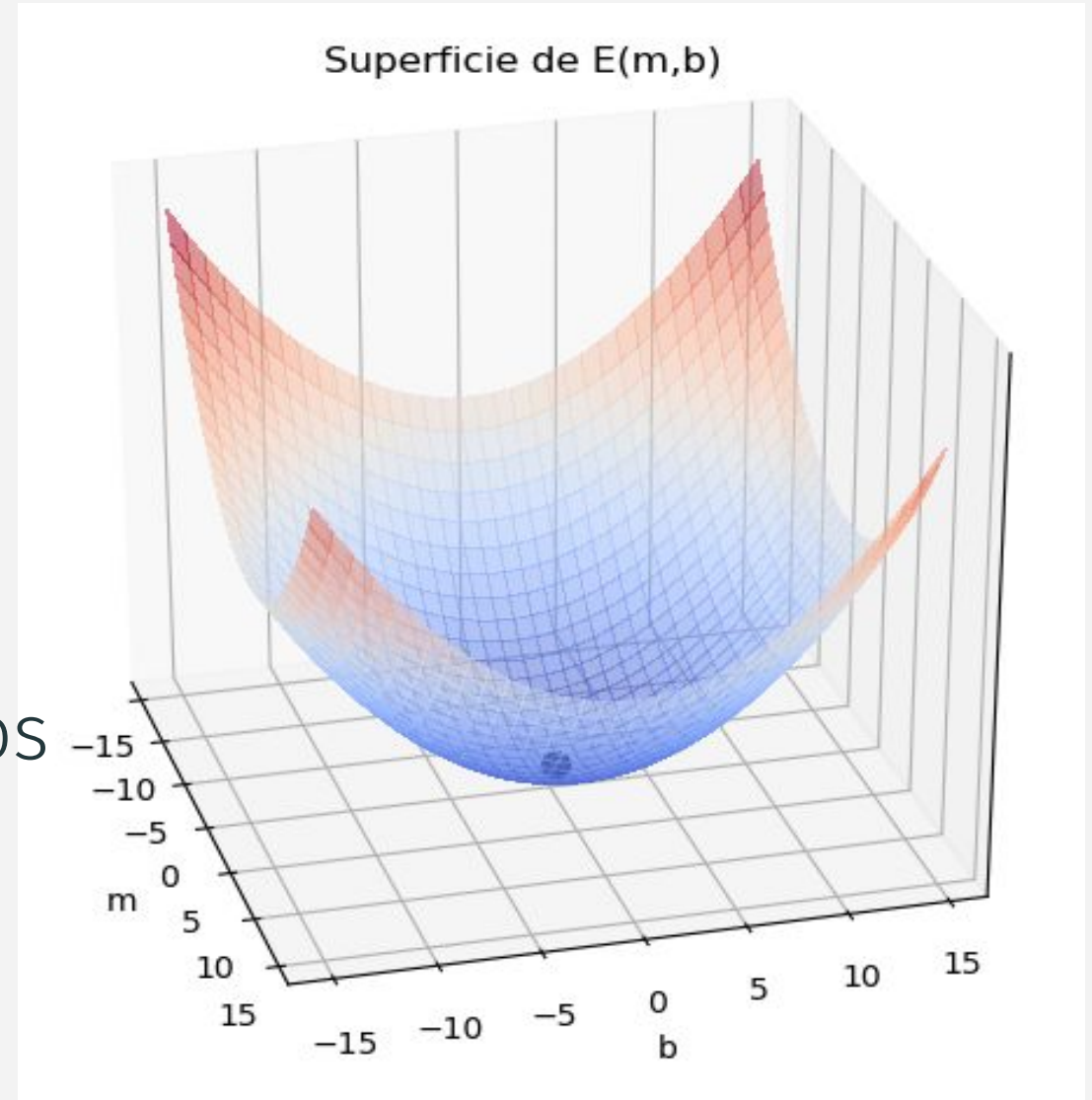


Métodos clásicos de Optimización para RL

Métodos clásicos (analíticos)

- Cálculo: $\frac{\partial E}{\partial b} = 0$ y $\frac{\partial E}{\partial m} = 0$, despejo m y b .
- Álgebra lineal: $Y = mX + b = \begin{pmatrix} 1 & x \end{pmatrix} \begin{pmatrix} b \\ m \end{pmatrix}$, proyecto.
- Probabilidades: $y = mx + b + e$ con $e \sim \mathcal{N}(0, \sigma)$, estimo m y b con MLE

- Ventajas
 - Solución analítica, simple
- Desventajas
 - Poco eficiente con muchos datos
 - $n \geq 10^6$
 - Problemas numéricos
- **Alternativa**
 - Descenso de gradiente

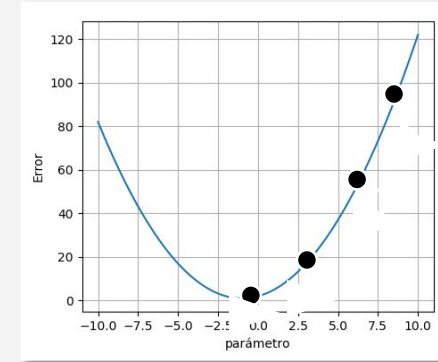
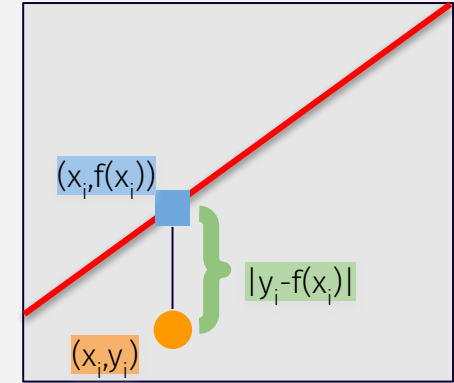
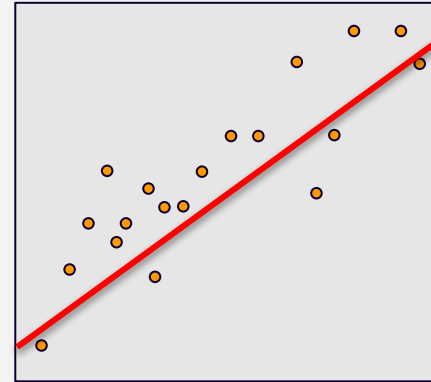


Descenso de gradiente para Regresión Lineal

- Modelo
 - $f(x) = m \times x + b$
 - $E(m,b) = (1/n) \sum_i^n E_i(m,b)$
 - $E_i(m,b) = (y_i - m x_i + b)^2$
- Ecuaciones de Descenso de Gradiente
 - $b = b - \alpha \delta E(m,b)/\delta b$
 - $m = m - \alpha \delta E(m,b)/\delta m$

■ Derivadas parciales?

- $\delta E(m,b)/\delta b = 1/n \sum_i^n 2 (y_i - f(x_i))$
- $\delta E(m,b)/\delta m = 1/n \sum_i^n 2 (y_i - f(x_i)) x_i$
 - ¿Cómo las obtengo?



Repaso de derivadas

$$1. \delta \mathbf{f(x)+g(f)} / \delta x = \delta \mathbf{f(x)} / \delta x + \delta \mathbf{g(x)} / \delta x$$

$$2. \delta \mathbf{c} / \delta x = 0$$

$$a. \delta \mathbf{f(x)-g(f)} / \delta x = \delta \mathbf{f(x)} / \delta x - \delta \mathbf{g(x)} / \delta x$$

$$b. \delta \mathbf{f(x)+c} / \delta x = \delta f(x) / \delta x$$

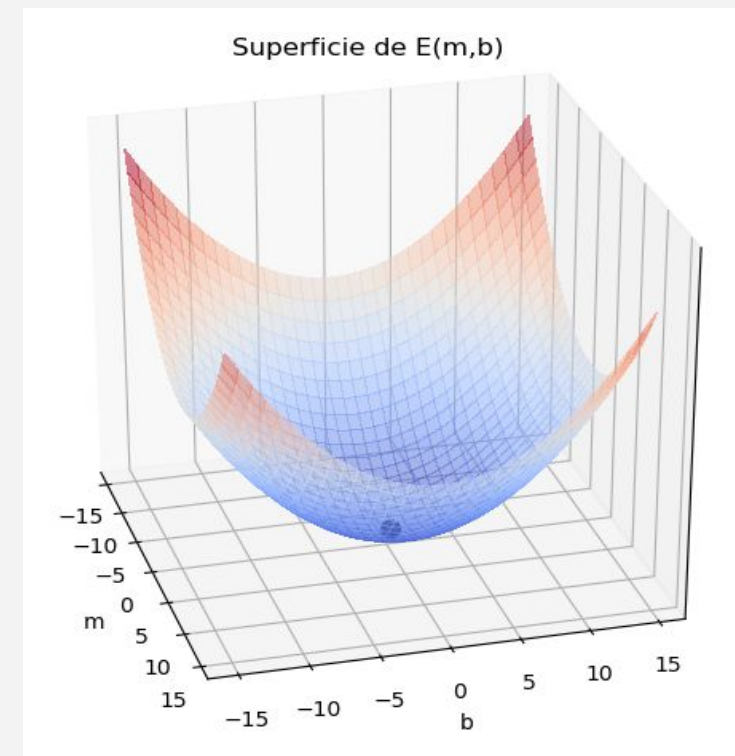
$$c. \delta [\Sigma_i^n \mathbf{f(x_i)}] / \delta x = \Sigma_i^n [\delta \mathbf{f(x_i)} / \delta x]$$

$$3. \delta \mathbf{f(g(x))} / \delta x = \delta \mathbf{f(g(x))} / \delta \mathbf{g(x)} \delta \mathbf{g(x)} / \delta x$$

$$\begin{aligned} a. \delta \mathbf{(3x-4)^2} / \delta x &= \delta \mathbf{(3x-4)^2} / \delta \mathbf{(3x-4)} \delta \mathbf{(3x-4)} / \delta x \\ &= 2(3x-4) 3 = 6 (3x-4) \end{aligned}$$

Derivada del error respecto de b $\delta E(\mathbf{m}, \mathbf{b}) / \delta b$

$$\begin{aligned}\delta E(\mathbf{m}, \mathbf{b}) / \delta b &= \delta(1/n \sum_i^n E_i(\mathbf{m}, \mathbf{b})) / \delta b \\&= 1/n \sum_i^n \delta E_i(\mathbf{m}, \mathbf{b}) / \delta b \\&= 1/n \sum_i^n \delta((y_i - f(x_i))^2) / \delta b \\&= 1/n \sum_i^n 2 (y_i - f(x_i)) \delta(y_i - f(x_i)) / \delta b \\&= 1/n \sum_i^n 2 (y_i - f(x_i)) \delta(y_i - m x_i + b) / \delta b \\&= 1/n \sum_i^n 2 (y_i - f(x_i)) \delta b / \delta b \\&= 1/n \sum_i^n 2 (y_i - f(x_i))\end{aligned}$$



Derivada del error respecto de m $\delta E(m,b)/\delta m$

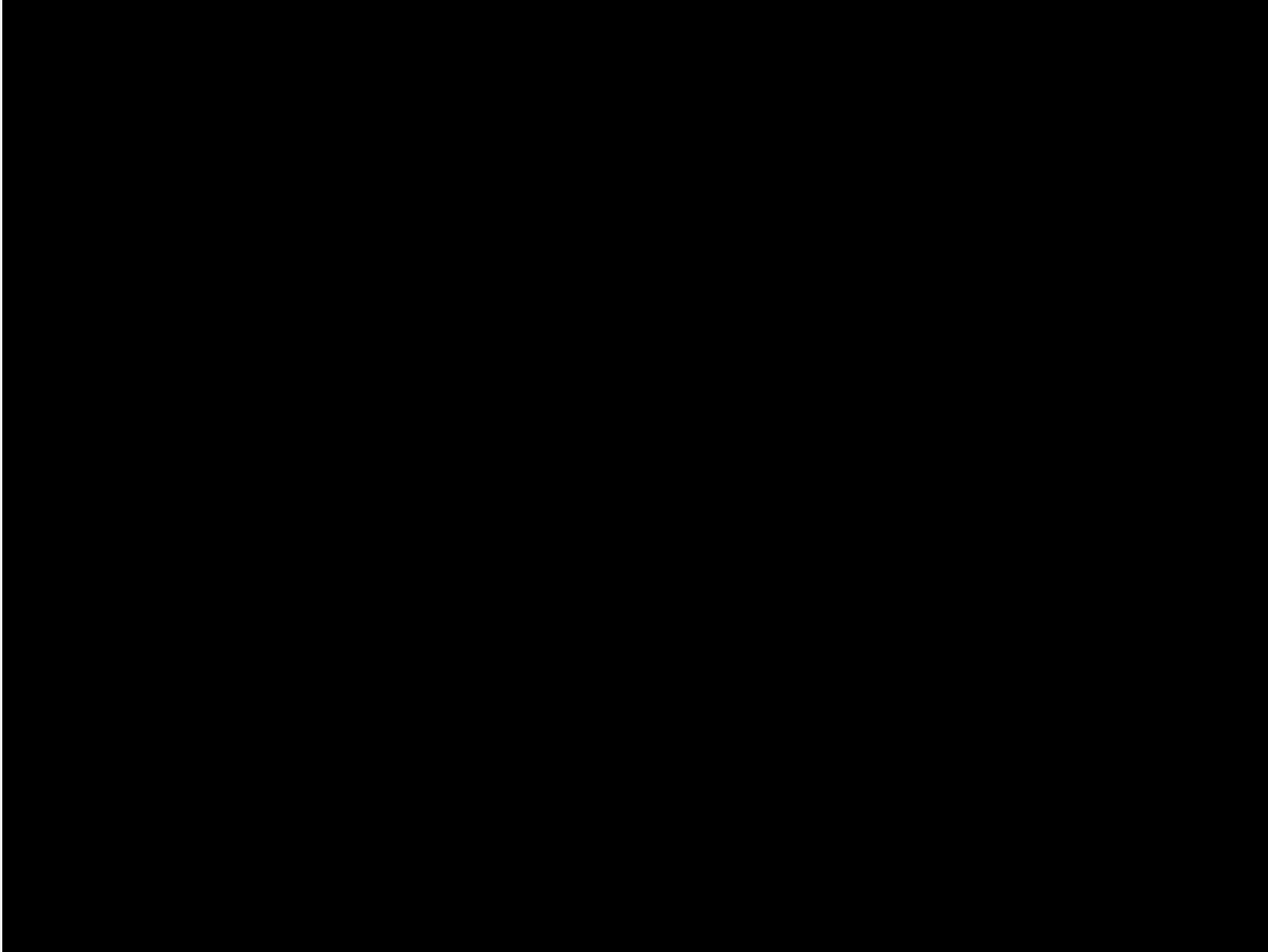
$$\begin{aligned}\delta E(m,b)/\delta m &= \delta(1/n \sum_i^n E_i(m,b))/\delta m \\&= 1/n \sum_i^n \delta E_i(m,b)/\delta m \\&= 1/n \sum_i^n \delta((y_i - f(x_i))^2)/\delta m \\&= 1/n \sum_i^n 2 (y_i - f(x_i)) \delta(y_i - f(x_i))/\delta m \\&= 1/n \sum_i^n 2 (y_i - f(x_i)) \delta(y_i - m x_i + b)/\delta m \\&= 1/n \sum_i^n 2 (y_i - f(x_i)) \delta(-m x_i)/\delta m \\&= 1/n \sum_i^n 2 (y_i - f(x_i)) x_i\end{aligned}$$

Igual a δb
pero con
 δm

Pseudocódigo

- **descenso_gradiente_rl**(x,y,α,iteraciones)
 - Inicializar m y b de forma aleatoria
 - for i in iteraciones:
 - Calcular derivadas
 - $\delta E(m,b)/\delta b = 1/n \sum_i^n 2 (y_i - f(x_i))$
 - $\delta E(m,b)/\delta m = 1/n \sum_i^n 2 (y_i - f(x_i)) x_i$
 - $b = b - \alpha \delta E(m,b)/\delta b$
 - $m = m - \alpha \delta E(m,b)/\delta m$
 - $E = (1/n) \sum_i^n (y_i - m x_i + b)^2$
 - retornar m,b

Ejemplo con simulador

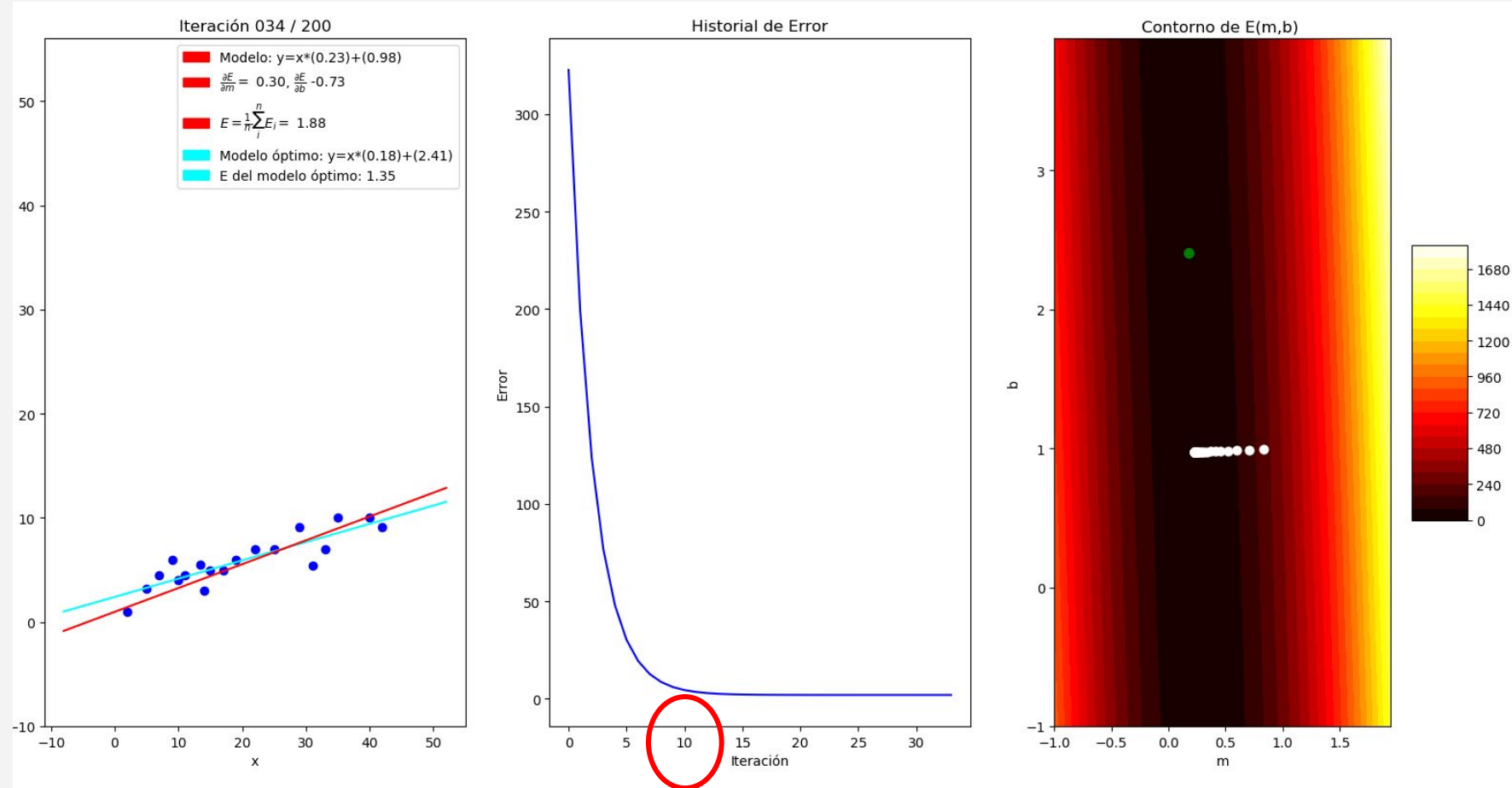


Cuestiones prácticas

- m_0 y $b_0 \rightarrow$ Valores iniciales de m y b
 - Afectan a la optimización.
 - Aprovechar la experticia del dominio.
 - Ejemplo de las notas,
 - ¿valores sensatos?
 - $y_{\min} = 2 \rightarrow$ Nadie tiene una nota menor a 2
 - $b_0 = y_{\min} = 2$
 - $x_{\max} = 40 \rightarrow$ Nadie estudia más de 40 horas
 - $y_{\max} = 10 \rightarrow$ Nota máxima 10
 - $m_0 = (y_{\max} - y_{\min}) / x_{\max} = (10 - 2) / 40 = 8 / 40 = 0.2$
 - Normalización de variables
 - Escala de notas de 0 al 10 vs 0 al 100
 - ¿Afecta al descenso?

Descenso de gradiente con **variables sin normalizar**

- Variables sin normalizar
 - Diferentes escalas
 - **Idem parámetros**
 - Dificulta encontrar el mínimo
 - Tarda más tiempo.



Normalización de variables

Original

Horas	Nota
2	1
5	3.2
7	4.5
9	6
10	4
11	4.5
13.4	5.5
14	3
15	5

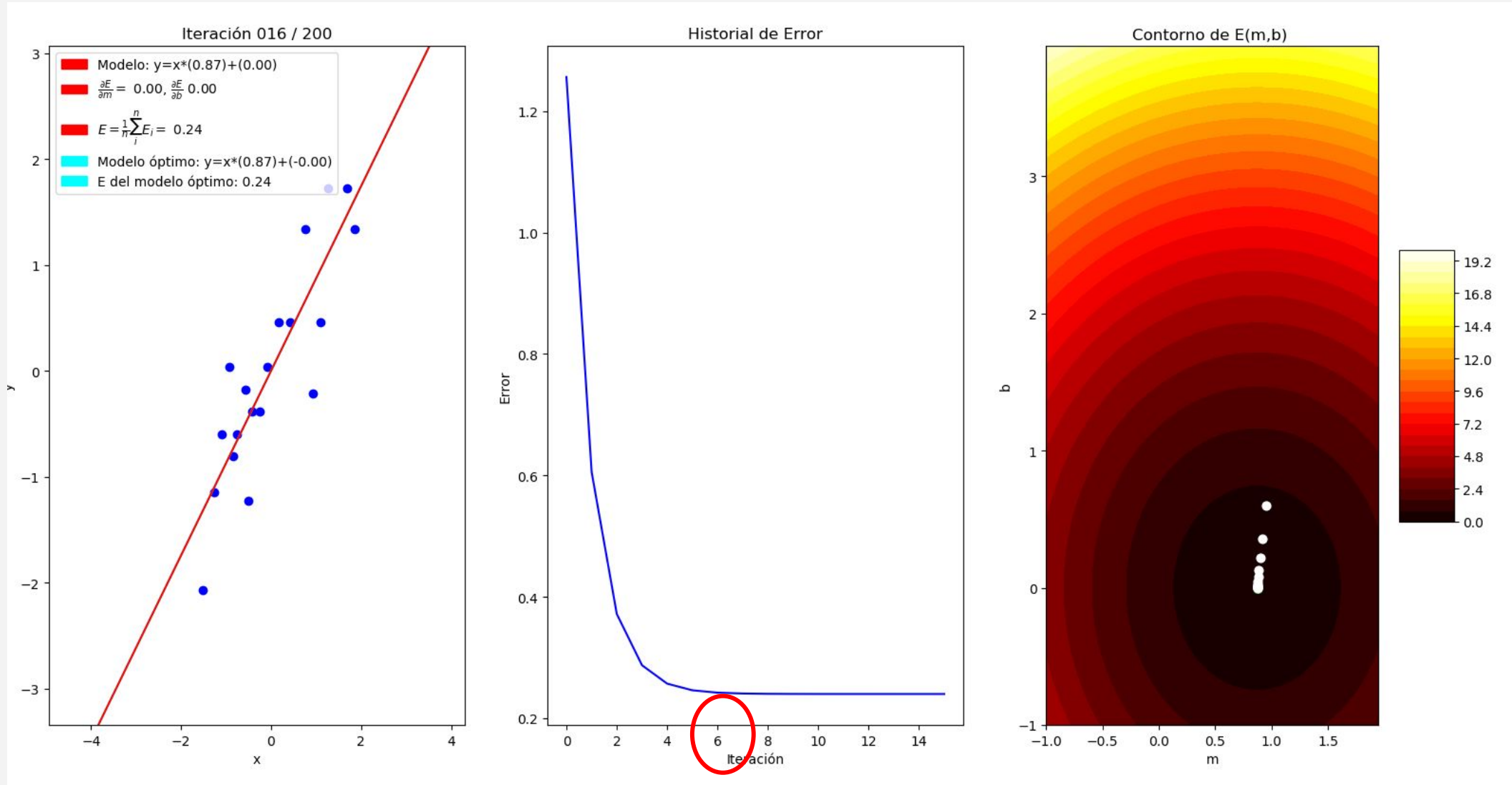
Normalización μ/σ

Horas	Nota
-1.75	-2.03
-1.06	-0.58
-0.60	0.28
-0.14	1.27
0.09	-0.05
0.32	0.28
0.87	0.94
1.01	-0.71
1.24	0.61

Normalización min/max

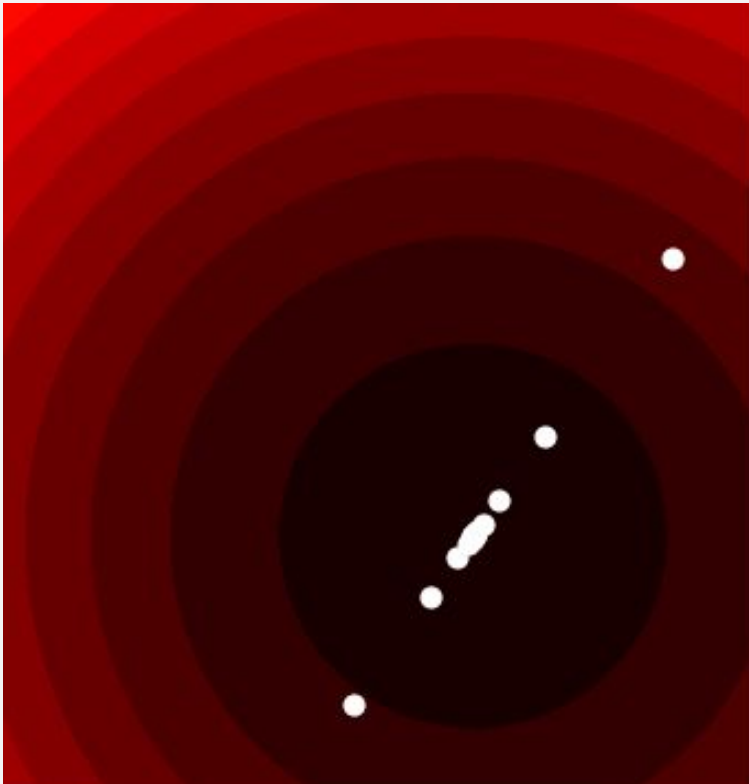
Horas	Nota
0	0
0.23	0.44
0.38	0.7
0.54	1
0.62	0.6
0.69	0.7
0.88	0.9
0.92	0.4
1.00	0.8

Descenso de gradiente con **variables normalizadas**

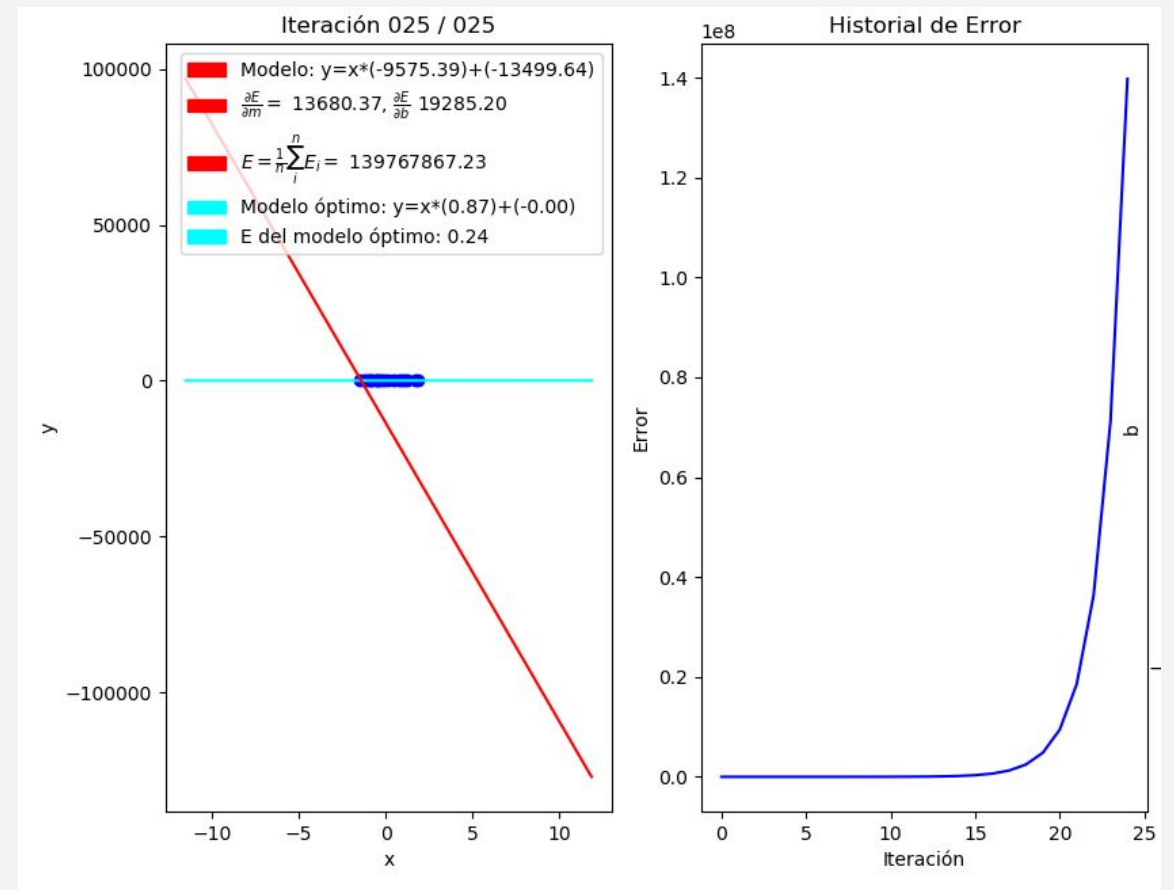


Divergencia con α muy grande

Saltos erráticos



El algoritmo diverge



Resumen

- Ecuaciones de Descenso de Gradiente
 - Especializadas para Regresión Lineal
 - $b = b - \alpha \delta E(m,b)/\delta b$
 - $m = m - \alpha \delta E(m,b)/\delta m$
 - Derivadas parciales
 - $\delta E(m,b)/\delta b = 1/n \sum_i^n 2 (y_i - f(x_i))$
 - $\delta E(m,b)/\delta m = 1/n \sum_i^n 2 (y_i - f(x_i)) x$
- Cuestiones prácticas
 - Valores iniciales de **m** y **b**
 - Valor de **α**
 - Normalización de las variables
 - Convergencia y velocidad

Ejercicio: Archivo **Regresión Lineal -**

Aprendizaje.ipynb

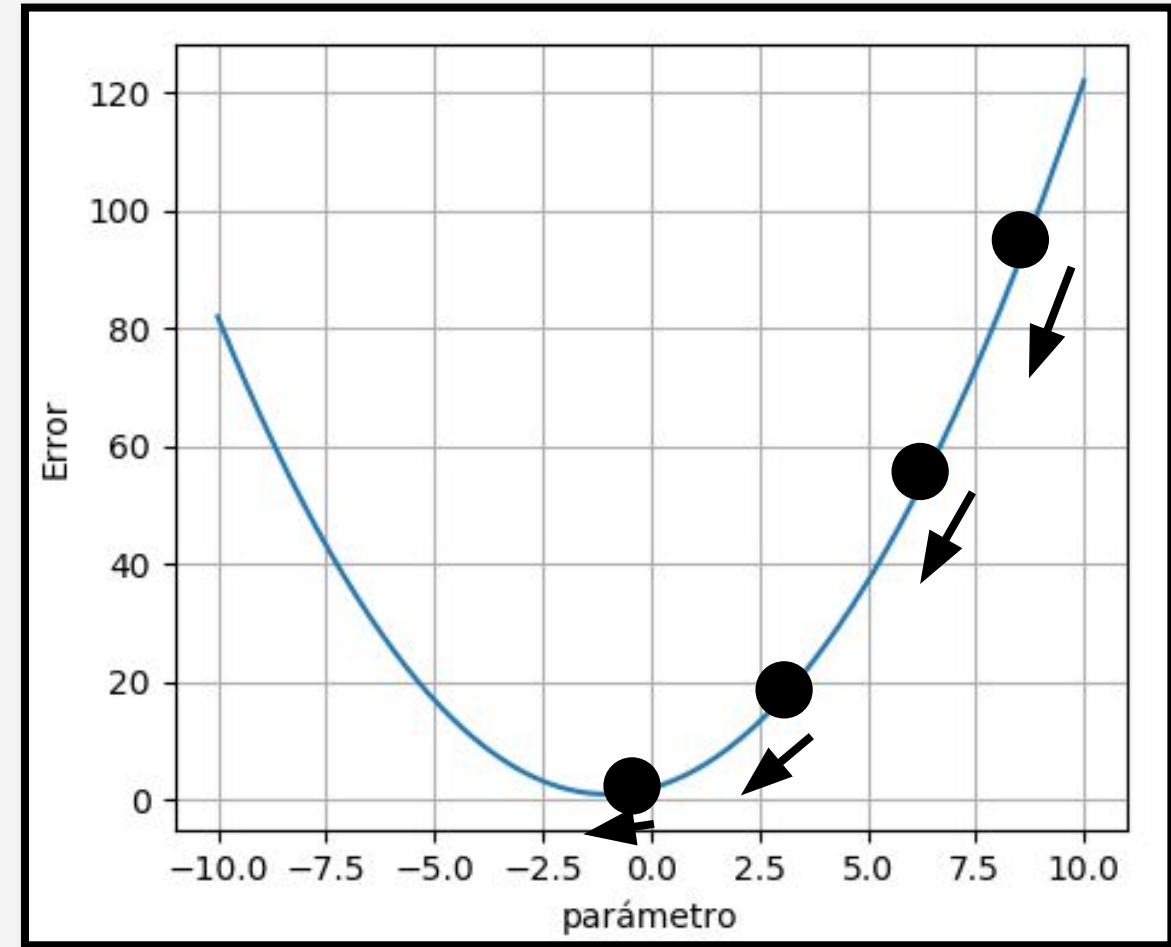
- **Probar e interpretar**

- Ejecutar el código y ver como se entrena el modelo
- Cambiar los valores iniciales de **m** y **b**
 - ¿Cómo afecta esto al entrenamiento?
- Cambiar el valor de **α**
 - ¿Cómo afecta esto al entrenamiento?
 - ¿Qué sucede si utilizo un α muy chico?
 - ¿y uno muy grande?
- ¿Son comparables los valores $\delta E / \delta m$ y $\delta E / \delta b$?
 - ¿De qué depende su magnitud?
- Cambiar el conjunto de datos utilizado por **anscombe1.csv**, **anscombe2.csv**, **anscombe3.csv** o **anscombe4.csv** y repetir.

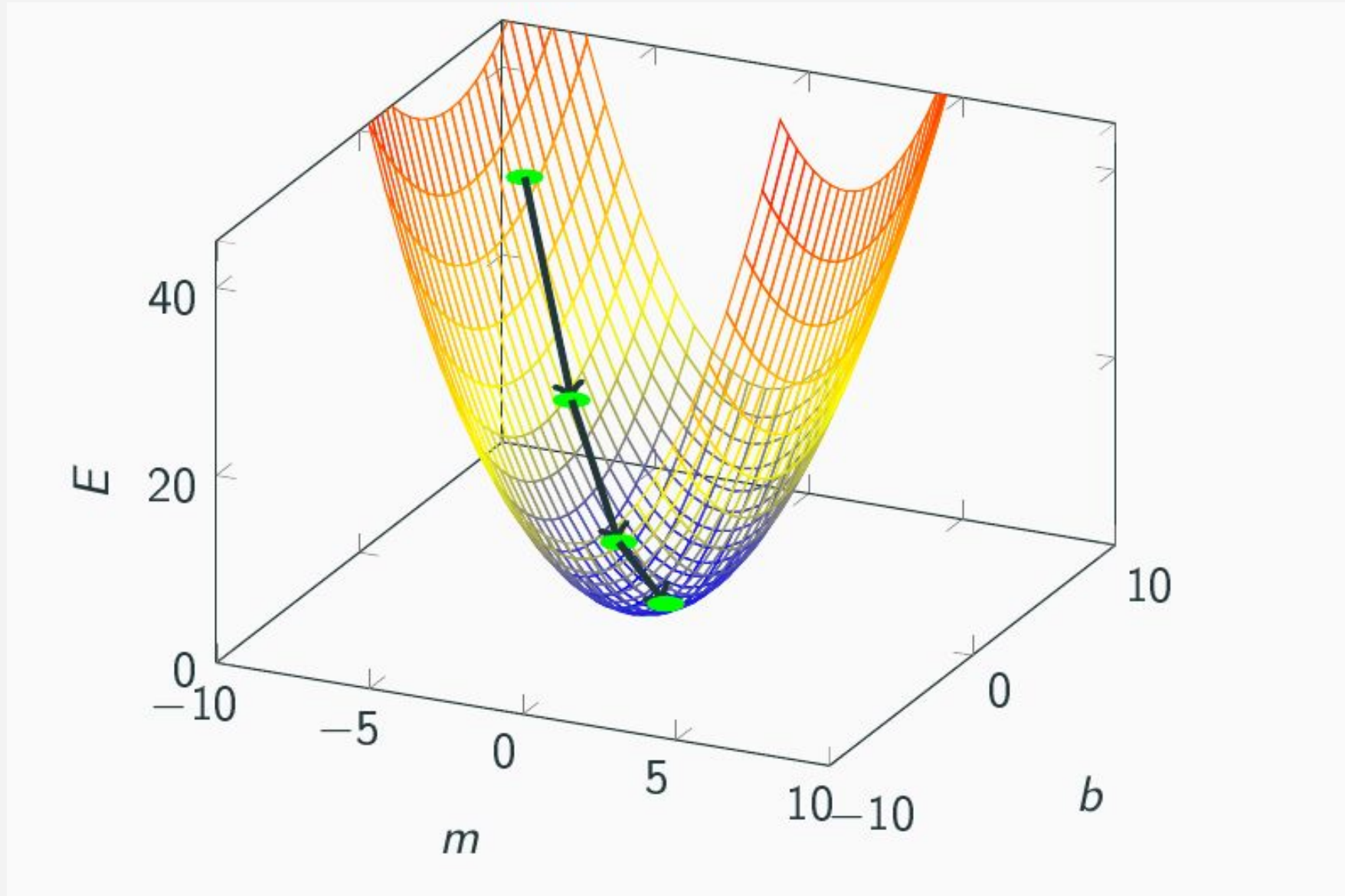
Monitoreo y Terminación de Descenso de Gradiente

Descenso de gradiente

- Descenso de gradiente
 - Iterativo
 - Generalizable
 - Regresión Lineal
 - Regresión Logística
 - Redes Neuronales
 - Máquinas de Vectores de Soporte
 - Cualquier modelo con E derivable
 - Escalable (con modificaciones)
 - Millones de ejemplos

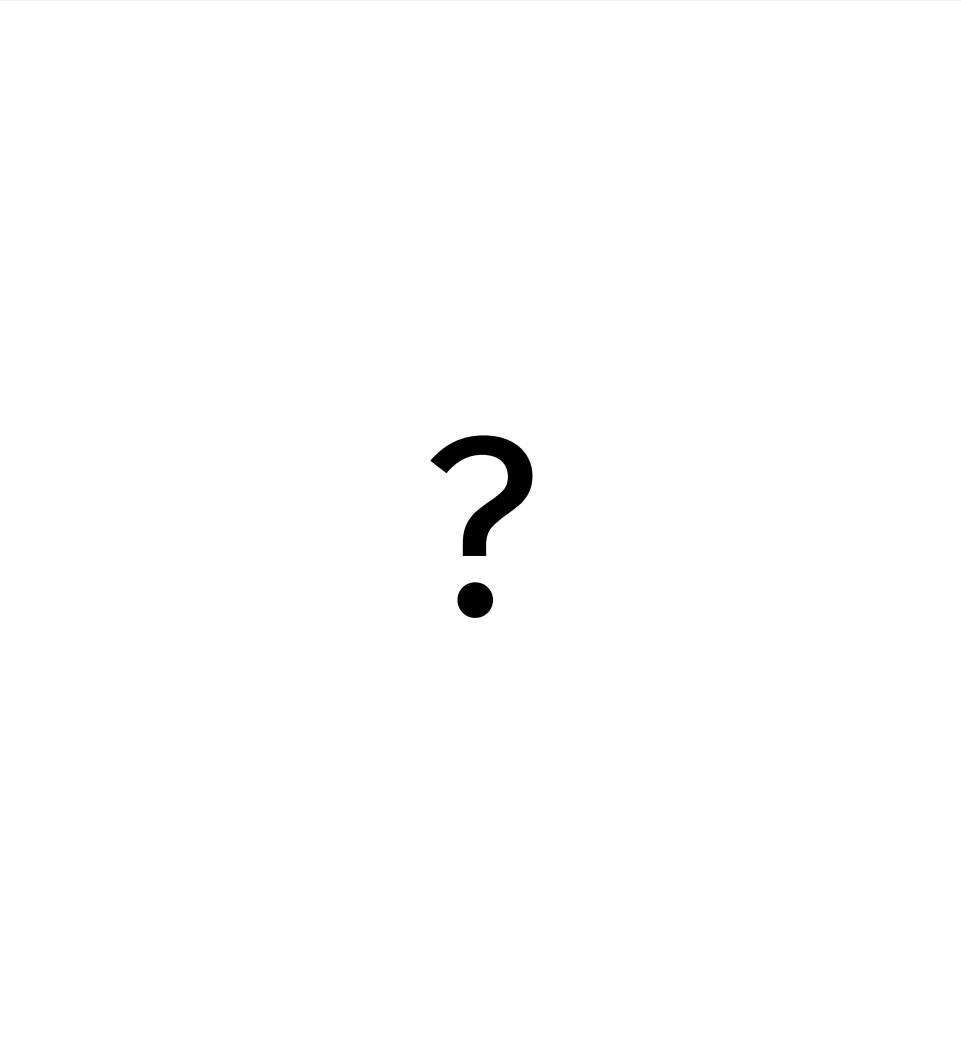


Descenso de gradiente con 2 parámetros



Descenso de gradiente con $n > 2$ parámetros

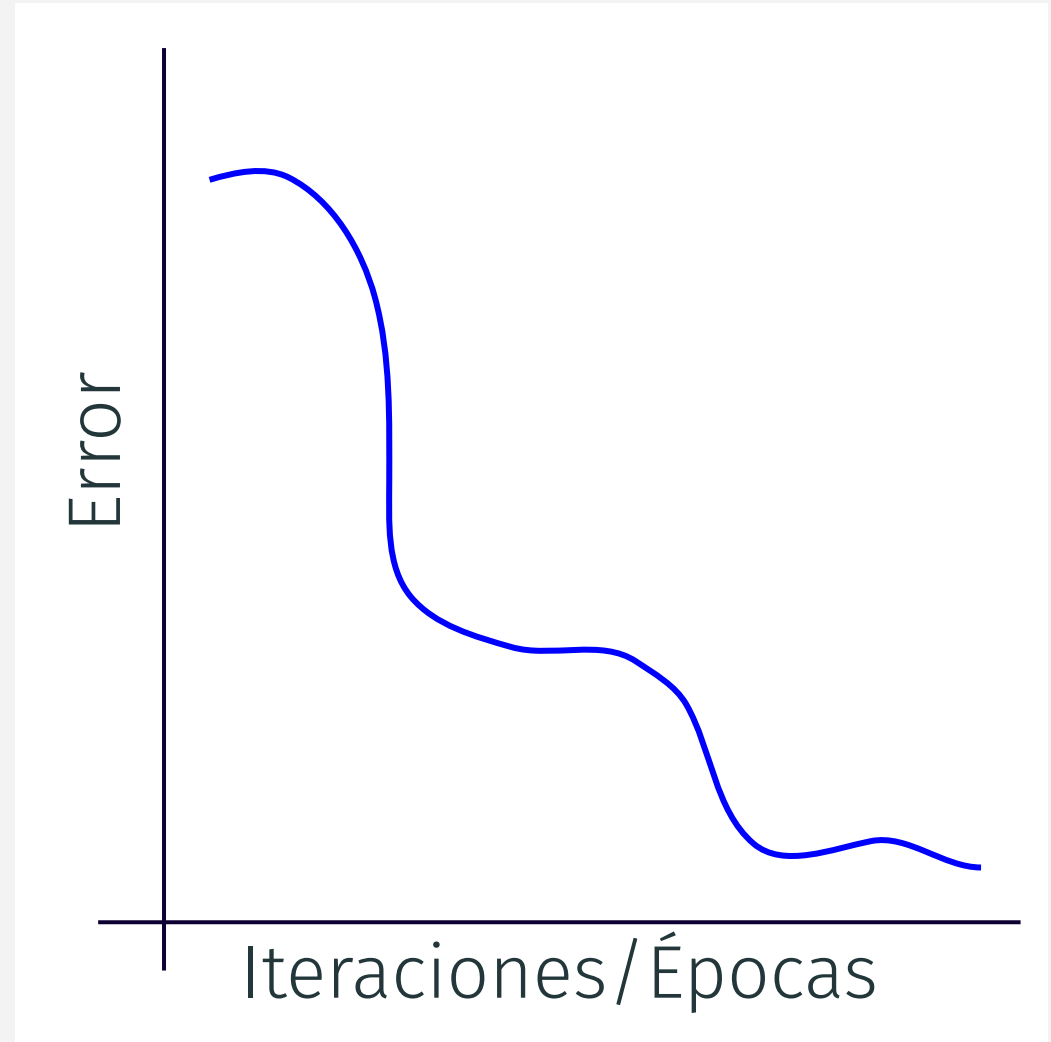
- ¿Cuándo terminar?
 - Iteraciones fijas
 - Tolerancia de error
- ¿Cómo determinar?
 - Monitorear parámetros vs error
 - Ver si se llega al mínimo
 - No se puede visualizar E con $n > 2$ parám.



Error vs Parámetros

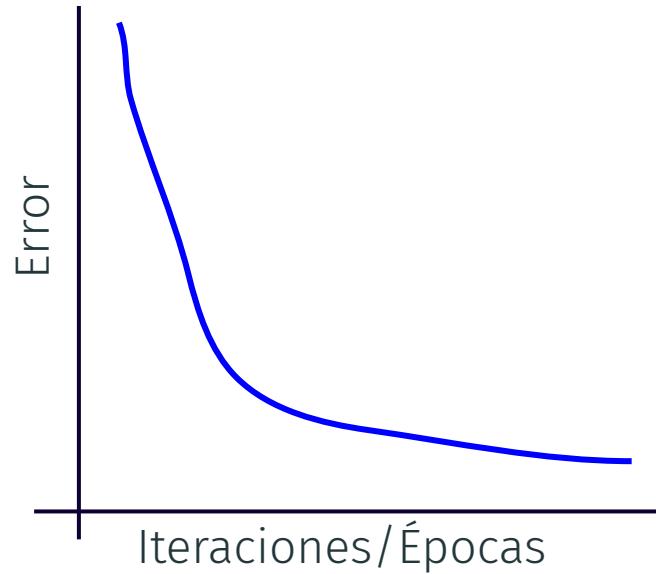
Curvas de error

- Monitorear Error vs Iteración
 - No requiere 1D o 2D
 - Funciona si el error no es convexo
- Cada Iteración
 - Calcular error
 - Actualizar gráfico
 - (o guardar en vector y dibujar al final)
 - (idem en un archivo)
- Sirve para otros métodos iterativos

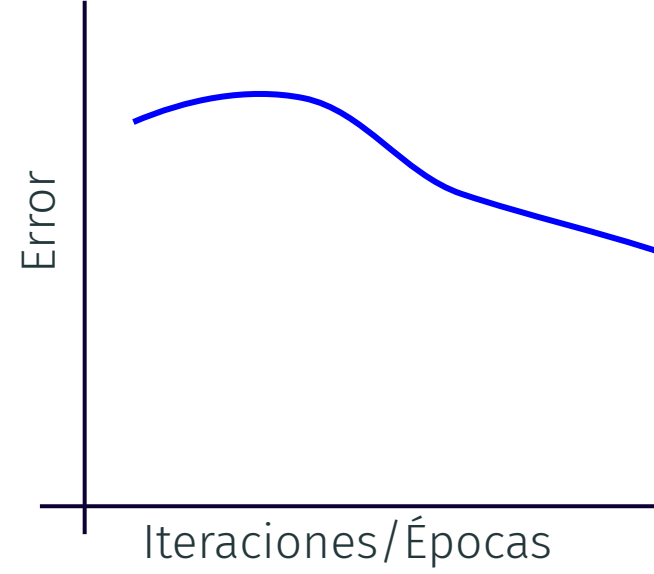
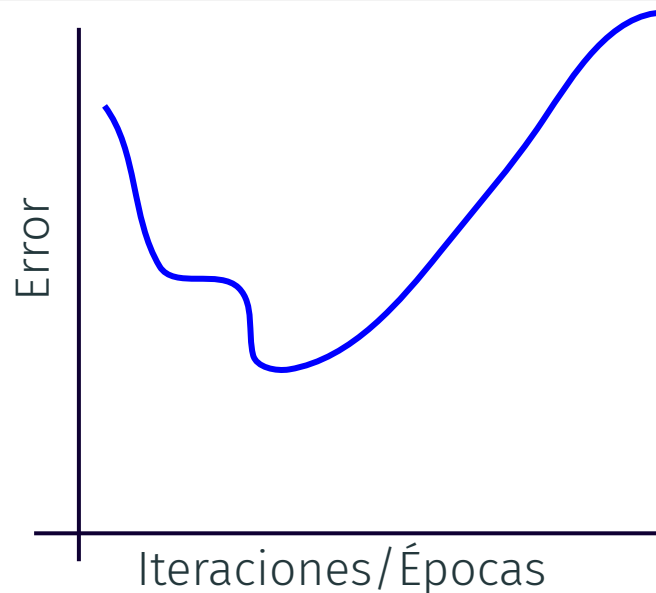


Curvas de error - Posibles síntomas

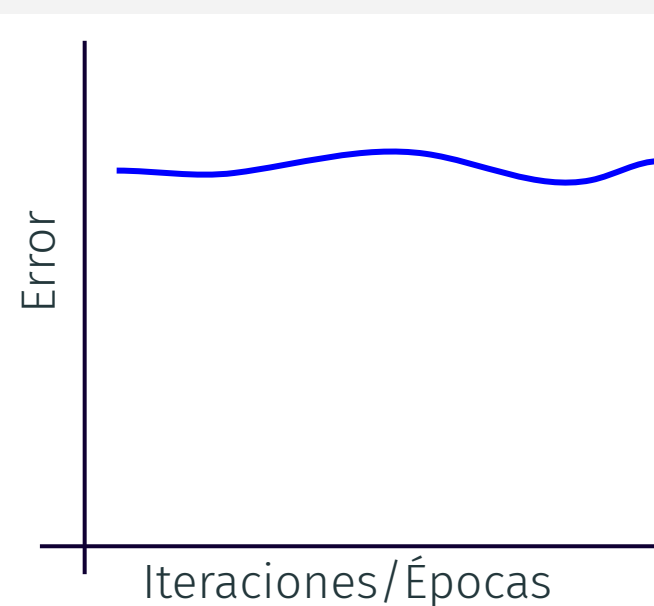
- Convergencia Rápida
- Ajustes menores
- Típico



- Divergencia
- Posibilidades
- 1) α muy alto
- 2) ej. sin normalizar



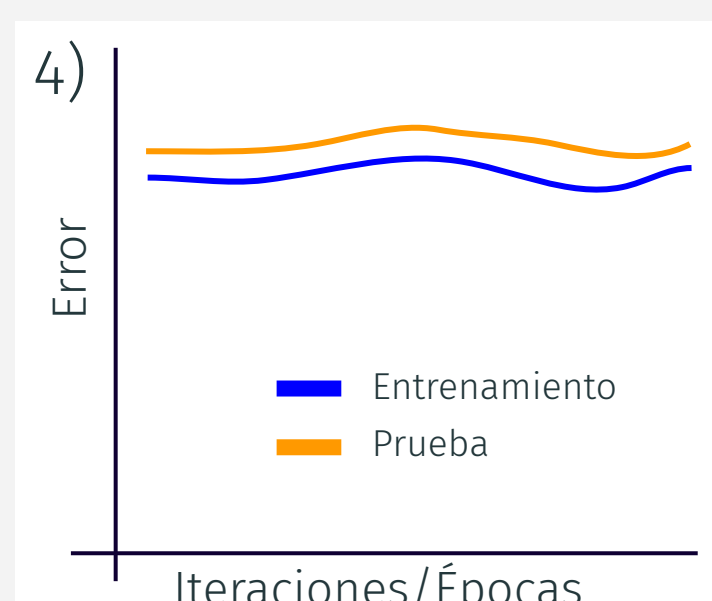
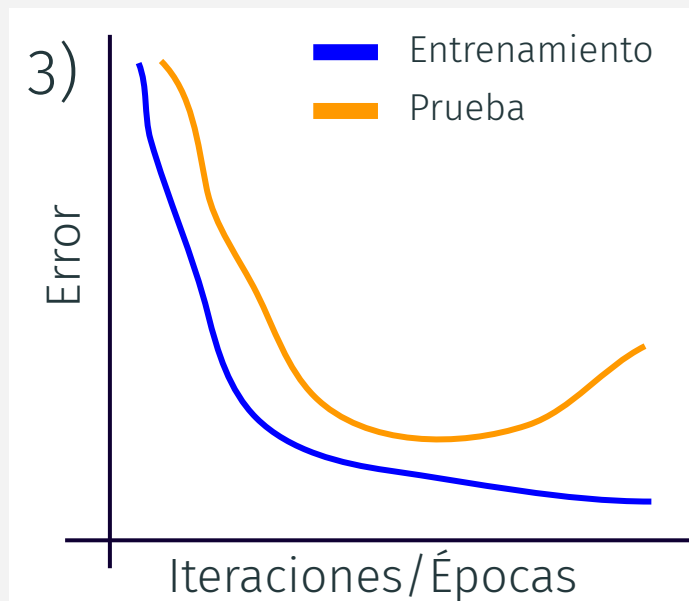
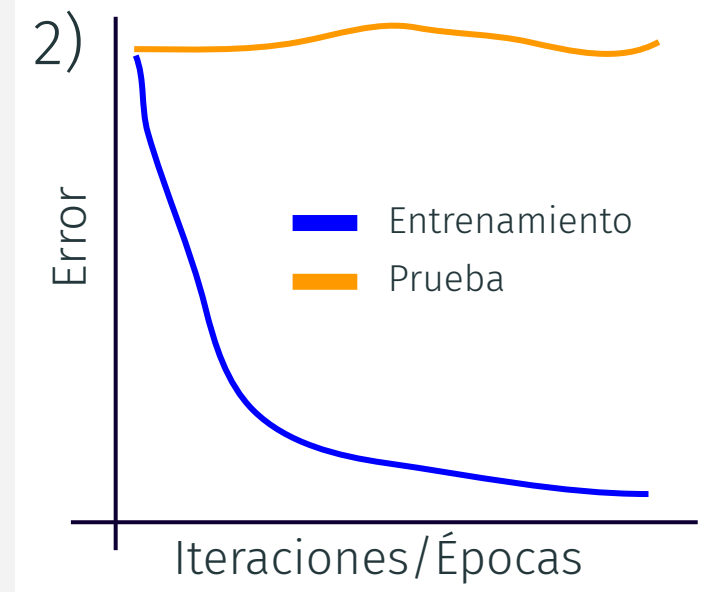
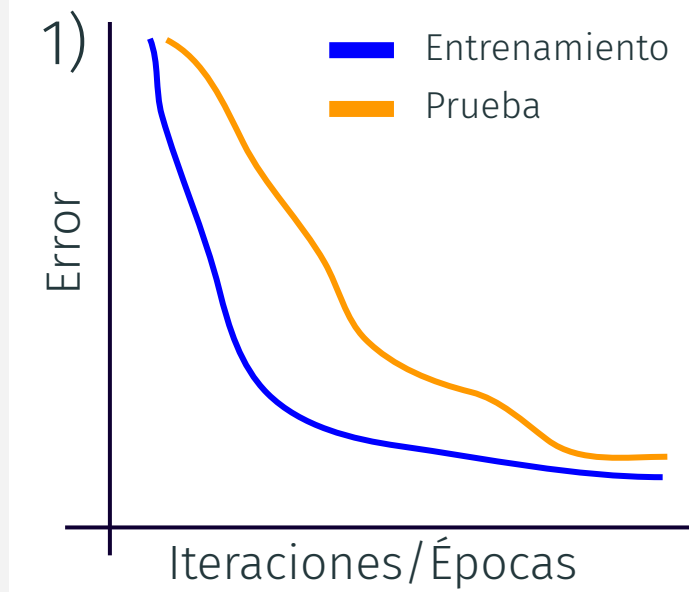
- Ajustes constantes
- Posibilidades
- 1) Faltan iteraciones
- 2) α muy chico



- Modelo no aprende
- Problema muy complejo
- α muy chico

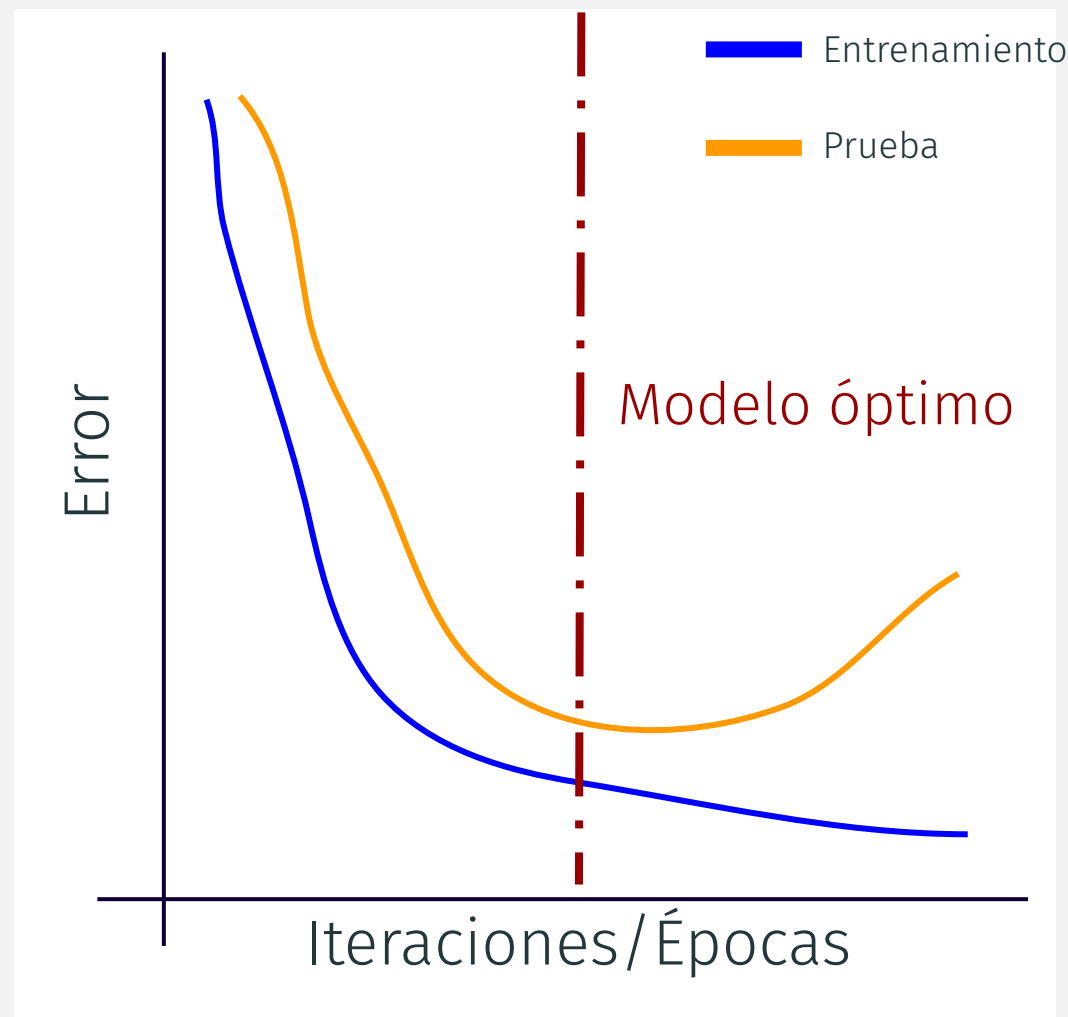
Curvas de error entrenamiento/prueba

- 1)
 - Buen entrenamiento
 - Buena generalización
- 2)
 - Buen entrenamiento
 - Mala generalización
- 3)
 - Buen entrenamiento
 - Buena generalización
 - Mala a lo último
 - Parar antes
- 4)
 - Mal entrenamiento
 - Mala generalización
 - Cambiar modelo



Para antes (early stopping)

- Entrenar más tiempo
 - Aprender mejor
 - Conjunto entrenamiento
- Típico
 - K iteraciones
 - Error de prueba ↓
 - K+1 en adelante
 - Error de prueba ↑
- Early Stopping
 - No esperar a que error en entrenamiento se estanque
 - Monitorear error en prueba (o val)

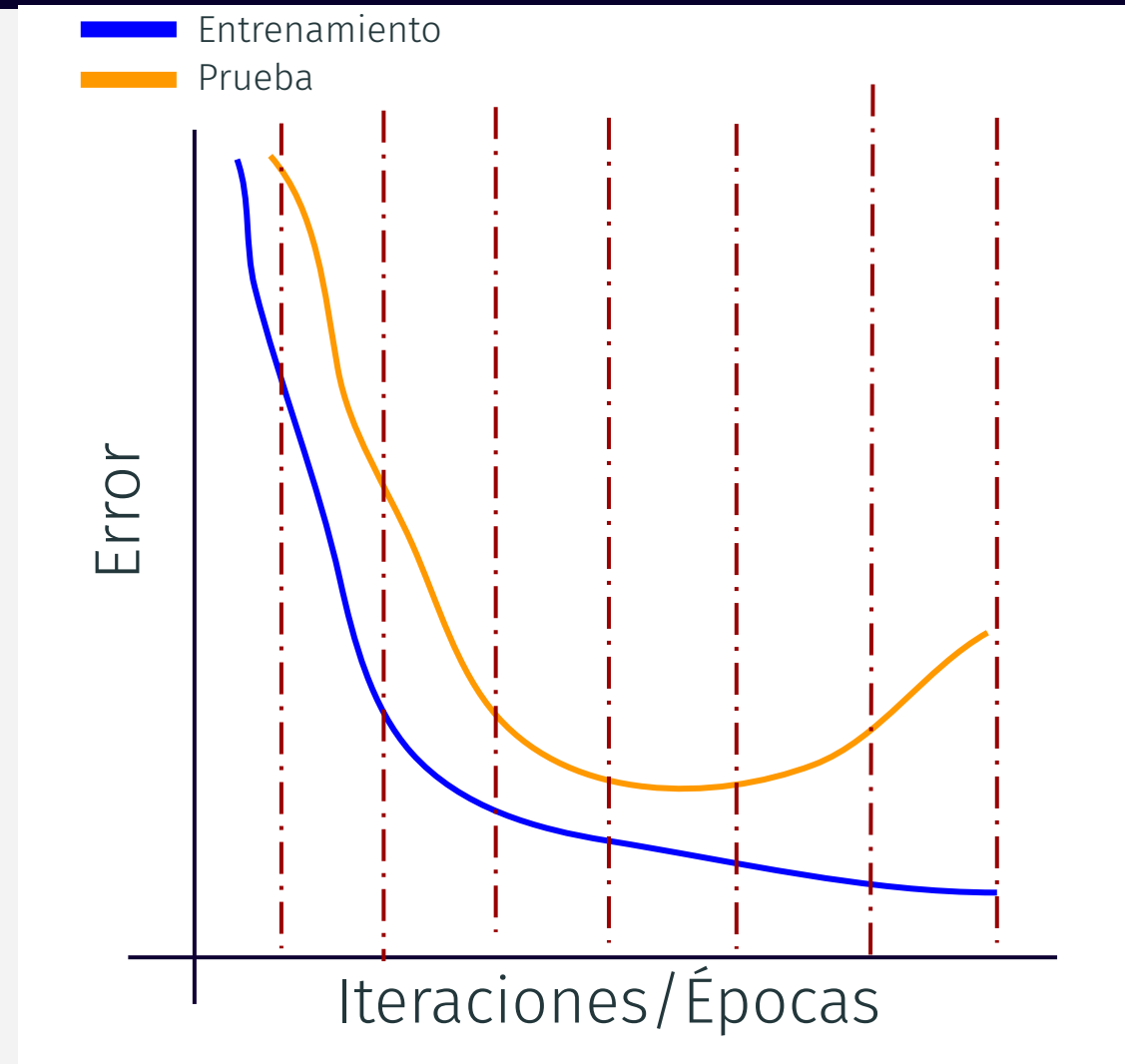


Descenso de gradiente con early stopping

- **descenso_gradiente**($x_{\text{train}}, y_{\text{train}}, x_{\text{test}}, y_{\text{test}}, \alpha, \text{ite}$)
 - p = parametros aleatorios
 - $E = \text{error}(x_{\text{test}}, y_{\text{test}}, p)$
 - $i = 0$
 - Repetir
 - Calcular $\delta E / \delta p$
 - $p = p - \alpha \delta E / \delta p$
 - $E_{\text{anterior}} = E$
 - $E = \text{error}(x_{\text{test}}, y_{\text{test}}, p)$
 - $i = i + 1$
 - Hasta que $\text{abs}(E - E_{\text{anterior}}) \simeq 0$ o $i = \text{ite}$
 - retornar p

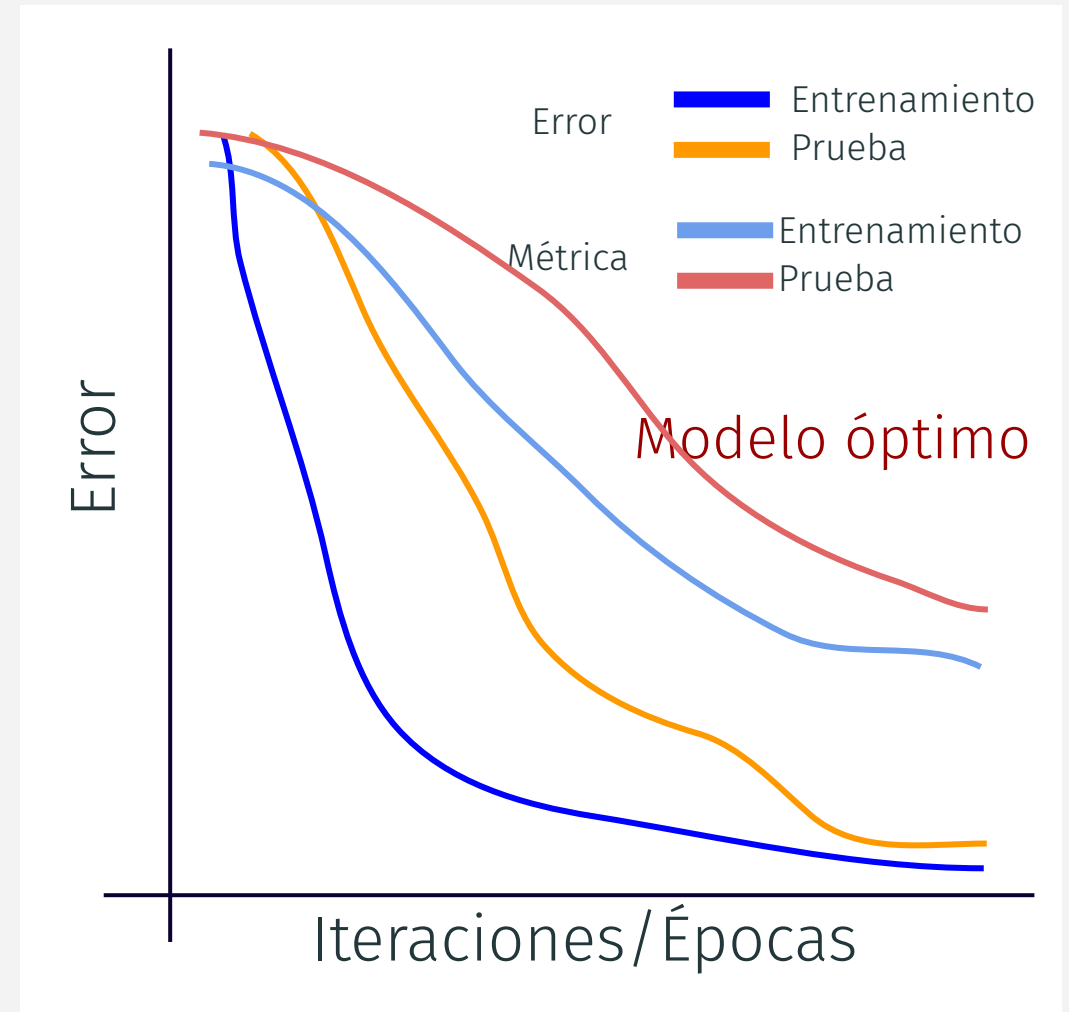
Para antes (early stopping)

- Alternativamente
 - Entrenar hasta el final
 - Guardar *checkpoints* del modelo
 - Valores de los parámetros
 - En distintas iteraciones
 - Elegir el *checkpoint* con menor error de prueba



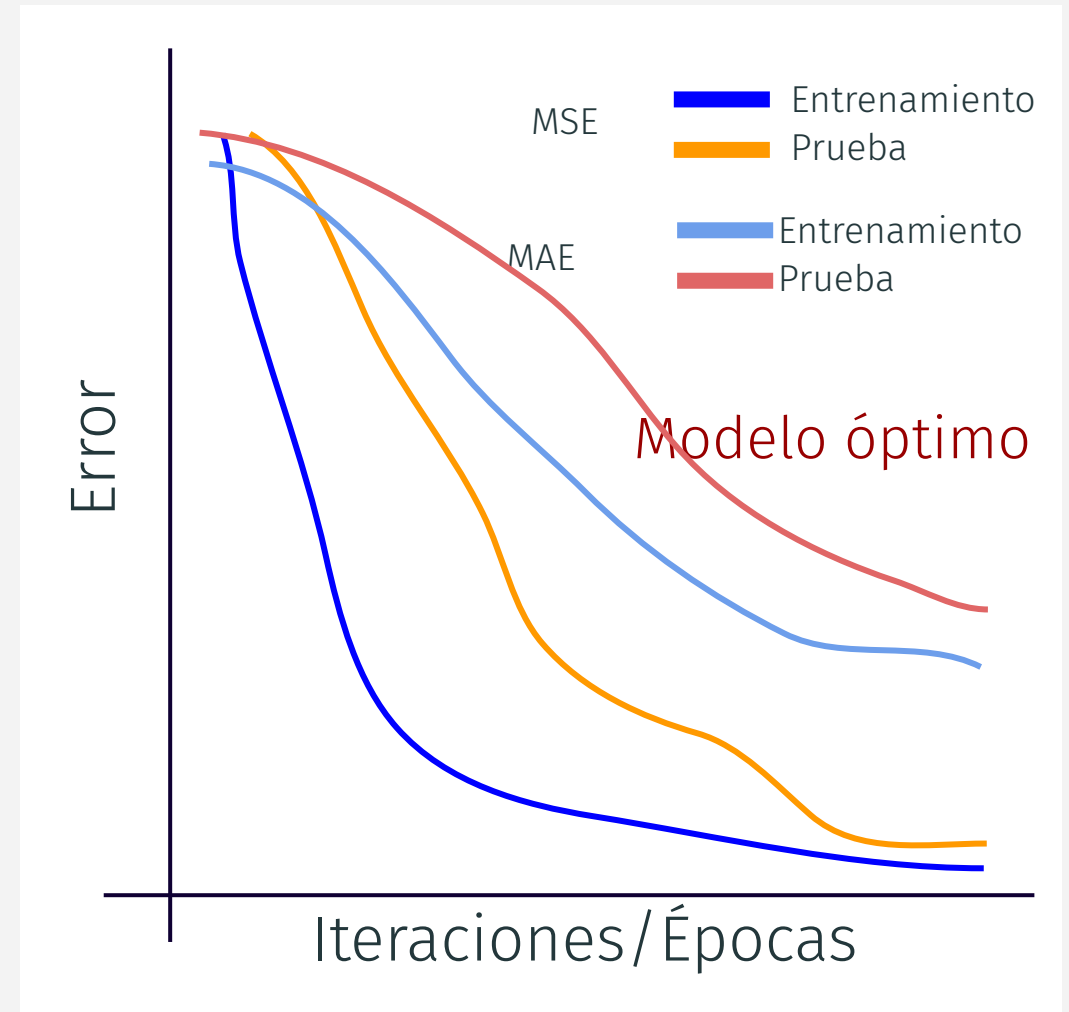
Métricas de error

- Función de error E
 - Diseñada para optimizar
 - Ejemplo MSE
 - $MSE_i = (y_i - f(x_i))^2$
 - $MSE = (1/n) \sum_i^n MSE_i$
 - Difícil de interpretar MSE_i
 - Difícil de interpretar MSE
 - Si **MSE = 12** ¿qué significa?
- Métricas
 - Similares a funciones de error
 - No tienen que ser optimizables
 - Más fáciles de interpretar
 - Fáciles de comparar entre modelos / problemas



Métrica Error Absoluto Medio

- Métrica MAE
 - Mean Absolute Error
 - Para problemas de Regresión
 - $MAE_i = \text{abs}(y_i - f(x_i))$
 - $MAE = (1/n) \sum_i^n MAE_i$
 - No es derivable
 - $\text{abs}()$ no es derivable en 0
- Fácil de interpretar
 - Mismas unidades que la salida **y**



Ejemplo: Error Absoluto Medio

- Métrica MAE
 - $MAE_i = \text{abs}(y_i - f(x_i))$
 - $MAE = (1/n) \sum_i^n MAE_i$
- Error MSE
 - $MSE_i = (y_i - f(x_i))^2$
 - $MSE = (1/n) \sum_i^n MSE_i$

Salida	Esperado	norm ²	MSE	abs(norm)	MAE
3	5	4	10	2	3
3	5	4		2	
1	5	16		4	
3	-1	16		4	

MAE: En promedio, el modelo tiene un error de 3 puntos